

PATHCAST: A Process-Aware Probabilistic Forecasting Method for Software Development Processes

Rodrigo Almeida de Oliveira

SUPERVISOR

Edson E. Scalabrin

PATHCAST: A Process-Aware Probabilistic Forecasting Method for Software Development Processes

Rodrigo Almeida de Oliveira

Thesis Project presented to the *Programa de Pós-Graduação em Informática* as a partial requirement for the degree of Doctor in Informatics.

MAJOR FIELD: Computer Science

SUPERVISOR: EDSON E. SCALABRIN

Espaço dedicado para a ficha catalográfica. Não alterar.

Espaço dedicado para a declaração de conclusão emitida pelo programa. Não
alterar.

Dedicatória vai aqui!

Citação bonita vai aqui! - Jean Paul Barddal, 2022

Abstract

Software development organizations generate large volumes of process data through *issue* management systems, version control repositories, continuous integration *pipelines*, and code review platforms. Despite this extensive instrumentation, the prediction of software development process behavior still relies predominantly on aggregated metrics and heuristic estimation methods that treat development workflows as black boxes. As a result, existing approaches provide a limited understanding of the structural dynamics and path-dependent variability that characterize real software development processes. This thesis investigates whether models derived from process mining can provide a more informative basis for the probabilistic forecasting of software development processes. In particular, the research tests the hypothesis that process-derived features and explicit stochastic models can improve predictive performance in comparison with approaches based exclusively on traditional software repository metrics. To address this problem, the thesis proposes PATHCAST (PATH-aware foreCASTing), a computational method that integrates process mining, Markov chain modeling, Monte Carlo simulation, and machine learning into a unified analytical *pipeline*. The approach transforms heterogeneous data from software repositories into structured event logs, from which process models are discovered to represent actual development flows. These models are then used to derive stochastic state-transition models that enable the probabilistic simulation of process execution trajectories and their outcome distributions. The research follows the *Design Science Research* methodology, in which the proposed method is implemented and empirically evaluated based on historical data from software projects. The evaluation compares the predictive performance of PATHCAST with baseline models based on repository metrics and conventional forecasting techniques. The expected results are that the explicit incorporation of process structure into forecasting models will improve both predictive accuracy and interpretability, thereby contributing to the advancement of process intelligence applied to software engineering.

Key-words: Process Mining, Probabilistic Forecasting, Markov Chains, Monte Carlo Simulation, Machine Learning, Design Science Research

Resumo

Organizações de desenvolvimento de software geram grandes volumes de dados de processo por meio de sistemas de gerenciamento de *issues*, repositórios de controle de versão, *pipelines* de integração contínua e plataformas de revisão de código. Apesar dessa extensa instrumentação, a previsão do comportamento de processos de desenvolvimento de software ainda se baseia predominantemente em métricas agregadas e métodos heurísticos de estimativa que tratam os fluxos de trabalho de desenvolvimento como caixas-pretas. Como consequência, as abordagens existentes fornecem uma compreensão limitada da dinâmica estrutural e da variabilidade dependente do caminho que caracterizam os processos reais de desenvolvimento de software. Esta tese investiga se modelos derivados de mineração de processos podem fornecer uma base mais informativa para a previsão probabilística de processos de desenvolvimento de software. Em particular, a pesquisa testa a hipótese de que características derivadas do processo e modelos estocásticos explícitos podem melhorar o desempenho preditivo em comparação com abordagens baseadas exclusivamente em métricas tradicionais de repositórios de software. Para abordar esse problema, a tese propõe o PATHCAST (PATH-aware foreCASTing), um método computacional que integra mineração de processos, modelagem por cadeias de Markov, simulação de Monte Carlo e aprendizado de máquina em um *pipeline* analítico unificado. A abordagem transforma dados heterogêneos provenientes de repositórios de software em logs de eventos estruturados, a partir dos quais são descobertos modelos de processo que representam os fluxos reais de desenvolvimento. Esses modelos são então utilizados para derivar modelos estocásticos de transição de estados que permitem a simulação probabilística de trajetórias de execução do processo e de suas distribuições de resultados. A pesquisa segue a metodologia *Design Science Research*, na qual o método proposto é implementado e avaliado empiricamente com base em dados históricos de projetos de software. A avaliação compara o desempenho preditivo do PATHCAST com modelos de referência baseados em métricas de repositórios e em técnicas convencionais de previsão. Espera-se que os resultados demonstrem que a incorporação explícita da estrutura do processo em modelos de previsão melhore tanto a precisão preditiva quanto a interpretabilidade, contribuindo, assim, para o avanço da inteligência de processos aplicada à engenharia de software.

Palavras-chave: Mineração de Processos, Previsão Probabilística, Cadeias de Markov, Simulação de Monte Carlo, Aprendizado de Máquina, Design Science Research

Acknowledgements

Agradecimentos vão aqui.

Contents

List of Figures	xi
List of Tables	xii
List of Charts	xiii
List of Algorithms	xiv
List of Acronyms	xv
1 Introduction	1
1.1 Research Problem	2
1.2 Objectives	3
1.2.1 General Objective	3
1.2.2 Specific Objectives	3
1.3 Research Questions	4
1.4 Justification and Research Gap	5
1.5 Expected Contributions	6
1.6 Research Methodology	7
1.7 Structure of the Document	8
2 Theoretical Foundation	10
2.1 Process Mining	10
2.2 Mining Software Repositories	11
2.3 Stochastic Process Modeling with Markov Chains	12
2.4 Monte Carlo Simulation and Probabilistic Forecasting Metrics	13
2.5 Machine Learning for Software Engineering	14
2.6 Variant Entropy	15
2.7 Chapter Synthesis	15

3	Systematic Literature Review	16
3.1	Introduction	16
3.2	Review Protocol	16
3.2.1	Research Questions	17
3.2.2	Search Strategy	17
3.2.3	Inclusion and Exclusion Criteria	20
3.2.4	Quality Assessment	21
3.2.5	Data Extraction	21
3.3	Search Results and Study Selection	21
3.3.1	Database Retrieval and Deduplication	21
3.3.2	Operational Working Set Construction	25
3.3.3	Title/Abstract Screening	25
3.3.4	Full-Text Screening	26
3.4	Results and Synthesis	29
3.4.1	Overview of Included Studies	29
3.4.2	Inclusion Criterion Coverage	30
3.4.3	Prevalent Exclusion Patterns	30
3.4.4	Full-Text Extraction and PDF-Based Triangulation	31
3.4.5	RQ1: Application Landscape	31
3.4.6	RQ2: Techniques and Prediction	34
3.4.7	RQ3: Integration and Gaps	38
3.5	Discussion	39
3.5.1	Research Gap Identification	39
3.5.2	Positioning of PATHCAST	43
3.5.3	Proposed Taxonomy: SPMF	43
3.5.4	Research Agenda	44
3.5.5	Implications for Practitioners	45
3.6	Threats to Validity of the SLR	45
3.6.1	Construct Validity	45
3.6.2	Internal Validity	46
3.6.3	External Validity	48
3.6.4	Conclusion Validity	51
3.6.5	Replication Package	51
3.7	Cross-Domain Synthesis and Gap Map	52
3.7.1	Process Mining Applied to SDLC	52
3.7.2	Stochastic Forecasting and ML for SE	53

3.7.3	Gap Map	53
3.8	Chapter Summary	54
4	The Proposed Method: PATHCAST	57
4.1	Architectural Overview	57
4.1.1	Pipeline Definition	57
4.1.2	Pipeline Architecture	58
4.1.3	Design Principles	58
4.1.4	Component View	60
4.1.5	Stage Contracts	60
4.2	Stage 1: Event Log Construction	61
4.2.1	Event Extraction	62
4.2.2	Case Correlation	62
4.2.3	Activity Mapping	63
4.2.4	Event Log Formatting and Filtering	64
4.2.5	Log Quality Assessment	65
4.3	Stage 2: Process Mining	65
4.3.1	Variant Entropy	65
4.3.2	Process Discovery	65
4.3.3	Conformance Checking	66
4.3.4	Performance Enhancement	66
4.3.5	State Space Extraction	66
4.4	Stage 3: Stochastic Modeling	67
4.4.1	Transition Counting	67
4.4.2	Maximum Likelihood Estimation	67
4.4.3	Confidence Assessment	68
4.4.4	Absorbing Chain Decomposition	68
4.4.5	Empirical Sojourn Time Distributions	69
4.4.6	Stationary Distribution and WIP Analysis	69
4.4.7	Rework Analysis	69
4.4.8	Numerical Example	69
4.5	Stage 4: Monte Carlo Simulation	70
4.5.1	Simulation Algorithm	70
4.5.2	Forecast Extraction	71
4.5.3	Convergence and Simulation Budget	72
4.5.4	Reproducibility	72

4.5.5	Comparison with Throughput-Based Monte Carlo	72
4.6	Machine Learning Enhancement Component	72
4.6.1	Feature Engineering	72
4.6.2	Label Construction	73
4.6.3	Training Protocol	73
4.6.4	Integration with Monte Carlo Forecasts	74
4.7	Pipeline Integration	74
4.8	Formal Properties and Theoretical Guarantees	75
4.8.1	Statistical Consistency	75
4.8.2	Calibration Property	75
4.8.3	Information-Theoretic Advantage of Process Awareness	76
4.8.4	Complexity Analysis	76
4.8.5	Assumptions and Boundary Conditions	77
4.9	Chapter Summary	77
5	Research Methodology	80
5.1	Design Science Research Paradigm	80
5.1.1	DSR Artifact Characterization	80
5.1.2	DSRM Process Instantiation	81
5.1.3	DSR Knowledge Contribution	81
5.2	Dataset Description	81
5.2.1	Repository Selection Criteria	81
5.2.2	Selection Process	82
5.2.3	Data Collection	82
5.2.4	Dataset Descriptive Statistics	82
5.2.5	Ethical Considerations	82
5.3	Experimental Design	83
5.3.1	E1: Pipeline Validation	83
5.3.2	E2: Forecasting Accuracy	85
5.4	Evaluation Metrics	86
5.4.1	Metrics for RQ1: Event Log and Process Model Quality	86
5.4.2	Metrics for RQ2: Stochastic Forecasting Accuracy	87
5.4.3	Metrics for RQ3: ML Feature Comparison	87
5.5	Baseline Methods	88
5.6	Statistical Analysis Procedures	89
5.6.1	Within-Repository Comparison	89

5.6.2	Cross-Repository Aggregation	89
5.6.3	Statistical Power Analysis	89
5.6.4	Reporting Standards	90
5.7	Threats to Validity	90
5.7.1	Internal Validity	90
5.7.2	External Validity	91
5.7.3	Construct Validity	91
5.7.4	Conclusion Validity	91
5.8	Chapter Summary	91
6	Implementation	93
7	Empirical Evaluation	94
8	Conclusion	95
A	Markov Chain Derivations	96
A.1	The Fundamental Matrix Identity	96
A.2	Expected Steps to Absorption	97
A.3	Variance of Absorption Time	97
A.4	Absorption Probabilities	98
A.5	Bayesian Estimation and Laplace Smoothing	98
A.6	Semi-Markov Connection	99
A.7	Numerical Example: Six-State SDLC Chain	100
B	Probabilistic Metric Derivations	102
B.1	CRPS: Definition and Equivalent Forms	102
B.2	Propriety of CRPS	103
B.3	CRPS–MAE Relationship	103
B.4	Calibration and the Aggregate Calibration Error	104
B.5	Worked Numerical Examples	104
C	Process Mining Algorithmic Details	105
C.1	Inductive Miner Internals	105
C.2	Alignment-Based Conformance Checking	106
C.3	Fitness, Precision, and Generalization	106
C.4	Worked Example	107

D	End-to-End Walkthrough Example	108
D.1	Synthetic Event Log	108
D.2	Process Discovery	109
D.3	Markov Estimation	109
D.4	Monte Carlo Simulation	110
D.5	Cross-Validation Against Analytical Results	110
D.6	ML Feature Construction	111
	References	96
E	Protocolo de Busca: Strings de Pesquisa	112
E.1	Resumo das Strings por Base de Dados	112
E.2	String S1 — Principal ($P \wedge I$)	113
E.3	String S2 — Complementar ($MSR \rightarrow$ Modelagem de Processos)	113
E.4	String S3 — Fronteira (Estocástico $\wedge PM \wedge ES$)	113
E.5	String S4 — Markov em Testes de Software (suplemento Scopus)	114
F	Prompts LLM de Triagem	115
F.1	Fase 1: Triagem Título/Resumo	115
F.2	Fase 2: Triagem Texto Completo	117
F.3	Fase 2b: Passagem <i>Screen-Blanks</i>	117
F.4	Parâmetros do Modelo em Cada Fase	118
G	Formulário de Extração de Dados	119
G.1	Prompt de Extração	119
G.2	Campos do Formulário de Extração	120
G.3	Vocabulário Controlado (Enums) da Saída Estruturada	120
H	Instrumentos de Avaliação e Classificação	123
H.1	Bandas de Prioridade para a Fila de Texto Completo	123
H.2	Algoritmo de Enriquecimento de Resumos	124
H.3	Checklist de Avaliação de Qualidade	125
H.4	Conjunto de Artigos de Controle	125
H.5	Taxonomia SPMF de Níveis de Integração	128
I	Lista de Leitura e Artefatos do Pipeline	129
I.1	Rubrica de Pontuação Multi-critério	129
I.2	Top-32: Lista de Leitura Ranqueada	129

I.3	Scripts do Pipeline de Revisão	129
I.4	Registro de Artefatos Persistentes	129
J	Evaluation Dataset: Repository List	133
J.1	Stratification Summary	133
J.2	Complete Repository List	133

List of Figures

1.1	Overview of the DSRM process applied to this research, with six activities, produced artifacts, and a two-stage evaluation.	8
3.1	PRISMA 2020 flow diagram of the study selection process. Phase labels follow the four-stage structure: Identification, Screening, Eligibility, and Included.	28
3.2	Distribution of confirmed studies by publication year ($n = 169$). .	29
3.3	Inclusion criteria matched by the confirmed set ($n = 169$; non-exclusive; IC labels map to IC4a–IC4d in the protocol).	30
3.4	Operational decision distribution in the 73-paper PDF triangulation subset.	32
3.5	Main technique families identified in the 169-study confirmed set. Counts are non-exclusive: a paper may use multiple technique families.	35
3.6	Operational integration levels and reading-priority distribution in the 73-paper PDF triangulation subset.	40
3.7	Evidence-type distribution in the 73-paper PDF triangulation subset.	40
4.1	PATHCAST pipeline architecture. Solid arrows indicate the primary sequential pipeline; dashed arrows indicate ML enhancement data flows.	59
4.2	PATHCAST component overview.	60
5.1	Two-stage experimental design.	83
H.1	Cascata de enriquecimento de resumos aplicada aos 886 artigos da fila TC.	124

List of Tables

1.1	Positioning relative to existing work.	6
3.1	Research questions and expected outputs.	17
3.2	PICO decomposition for search string construction.	18
3.3	Database-specific search adaptations and raw retrieval counts. . .	19
3.4	Control papers for search string validation.	20
3.5	Inclusion and exclusion criteria.	22
3.6	Quality assessment checklist.	23
3.7	Quality-assessment results for the confirmed included set ($n =$ 169; full QA applied). Auto-generated from <code>pipeline/qa_llm.py</code> . . .	23
3.8	Data extraction form.	24
3.9	Record retrieval by source.	25
3.10	Working set composition.	26
3.11	Title/abstract screening results.	26
3.12	Full-text screening outcomes (886 papers forwarded from T/A phase).	27
3.13	Integration levels in the 169-study confirmed set and PATHCAST positioning.	44
3.14	Inter-rater agreement (Cohen's κ) between primary screener (claude- haiku-4-5) and verifier (claude-sonnet-4-6) on a stratified random 20% sample.	47
3.15	Auxiliary-tier inter-rater agreement (cross-model verification: Haiku 4.5 primary vs. Sonnet 4.6 verifier on a stratified 20% sample). . .	47
3.16	Auxiliary-corpus sensitivity check. A random sample of n papers from the deferred auxiliary corpus was re-screened with the same T/A LLM protocol used for the working set.	49

3.17	Auxiliary-corpus FT-screening outcome (papers that passed T/A on the 3,807-paper deferred corpus).	50
3.18	Re-FT screening of aux-pending papers that gained an abstract through the post-hoc enrichment cascade.	50
3.19	EC5 recovery attempt against four open-access discovery APIs.	50
3.20	Quality assessment combined across working-set and auxiliary tiers ($n = 381$).	51
3.21	Process mining applied to software development.	52
3.22	Technique integration ($\checkmark$) across studies. PATHCAST is the first proposal at the full intersection.	53
4.1	Event extraction mapping per data source.	62
4.2	Default activity mapping from source event types to SDLC taxonomy.	64
4.3	Throughput-based vs. process-aware Monte Carlo.	72
4.4	Feature definitions for the ML component.	73
4.5	Inter-stage data contracts.	75
4.6	Computational complexity per pipeline stage.	77
4.7	Assumptions, implications, and mitigations.	78
4.8	Method components vs. research questions.	78
5.1	Mapping of DSRM activities to thesis components.	81
5.2	Aggregate dataset statistics (48 repositories).	83
5.3	Dataset diversity profile.	83
5.4	Evaluation metrics for RQ1.	86
5.5	Evaluation metrics for RQ2.	87
5.6	Evaluation metrics for RQ3.	88
5.7	Baseline methods.	88
5.8	A priori power estimates ($n = 48$, four baselines).	89
5.9	Summary: research questions, evaluation stages, metrics, baselines.	92
E.1	Strings de busca por base de dados e escopo de campo.	112
F.1	Parâmetros do LLM em cada fase da triagem.	118
G.1	Formulário de extração de dados: todos os campos de <code>extraction_template.csv</code>	121
G.2	Vocabulário controlado para saída estruturada do LLM.	122

H.1	Definição das bandas de prioridade para a fila de revisão de TC. . .	124
H.2	Resultados do enriquecimento de resumos por banda de prioridade.	125
H.3	Checklist de avaliação de qualidade com guia de pontuação. . . .	126
H.4	Conjunto completo de artigos de controle para validação das strings.	127
H.5	Taxonomia SPMF: três dimensões com definição de níveis.	128
I.1	Rubrica de pontuação multi-critério para a lista de leitura top-32.	130
I.2	Lista de leitura top-32 com pontuação, ano, cluster IC e sumário do achado principal. Artigos marcados com † são inclusões forçadas por caráter seminal.	131
I.3	Scripts do pipeline de revisão e suas funções.	132
I.4	Artefatos persistentes da revisão.	132
J.1	Repository count by primary language stratum ($n = 48$).	134
J.2	Complete list of 48 evaluation repositories with stratification meta- data. Abbreviations: Scale — S = Small (< 5K commits), M = Medium (5–20K), L = Large (> 20K); Gov. — Fdn = Foundation, Corp = Corporate, Comm = Community; WF — B = Basic, St = Standard, A = Advanced.	135

List of Charts

List of Algorithms

1	Case Correlation Cascade	63
2	Transition Count Extraction	67
3	Process-Aware Monte Carlo Simulation	70
4	ML Training with Temporal Hold-Out	74

List of Acronyms

AHP Analytic Hierarchy Process

BPM Business Process Management

CEP Complex Event Processing

CI/CD Continuous Integration / Continuous Delivery

DSR Design Science Research

DSRM Design Science Research Methodology

EDA Event-Driven Architecture

LLM Large Language Model

LSTM Long Short-Term Memory

MCDM Multi-Criteria Decision Making

MSR Mining Software Repositories

PATHCAST PATH-aware foreCASTing

PI Process Intelligence

PM Process Mining

SDLC Software Development Life Cycle

SLR Systematic Literature Review

XES eXtensible Event Stream



Introduction

Contemporary software development operates in a densely instrumented ecosystem. Every state transition recorded in an issue tracker such as Jira, every commit pushed to a Git repository, every build triggered in a continuous integration platform, every pull request reviewed on GitHub generates time-stamped event data that documents who performed which activity, when, and in what sequence. Collectively, these events constitute a high-fidelity chronicle of the software development process as it actually occurs, rather than as it is documented in process manuals or assumed in retrospective meetings.

The discipline of process mining, formalized by van der Aalst (??), has demonstrated that event data can be systematically transformed into process models, compliance diagnostics, performance metrics, and predictions of future behavior. The field has matured along an analytical spectrum that progresses from descriptive capabilities (what happened?), through diagnostic (why did it happen?) and predictive (what will happen?), to prescriptive (what should be done?). In domains such as healthcare, manufacturing, financial services, and public administration, process mining has delivered measurable organizational value (??).

Software engineering, however, presents a paradox. Despite generating some of the richest and most fine-grained event data among all organizational activities, the application of process mining to the Software Development Lifecycle (SDLC) remains limited to descriptive analysis. Development teams continue to rely on aggregated metrics (velocity, throughput, cycle time) computed from project management dashboards, rarely interrogating the underlying process

flows that produce these metrics. When forecasting is attempted, it typically uses throughput-based heuristics that treat the development process as a black box, ignoring the internal state dynamics, rework loops, and path-dependent variability that characterize real software processes.

This thesis proposes a method that bridges the gap between the rich event data generated by software development tools and the probabilistic forecasting capabilities that this data could support. The method, named PATHCAST integrates process mining, Markov chain estimation, Monte Carlo simulation, and machine learning into a unified pipeline that transforms heterogeneous repository data into calibrated probabilistic forecasts of software process outcomes.

1.1 RESEARCH PROBLEM

Three interrelated problems motivate this research.

The first problem is the **descriptive ceiling** in software process mining. The existing literature on process mining applied to software development (?????) has demonstrated that process models can be discovered from repository data. However, these studies stop at the descriptive level: they reveal what happened, but do not advance to predicting what will happen or quantifying the uncertainty associated with future outcomes. No published work proceeds from software process discovery to stochastic modeling and probabilistic forecasting using the discovered process structure as the foundation for prediction.

The second problem is the **structural blindness** of existing forecasting approaches. The most widely adopted forecasting method in agile software development, throughput-based Monte Carlo simulation (??), samples daily throughput values from historical data and simulates backlog completion. While practical, this approach treats the development process as a black box: it does not model internal state transitions, rework probabilities, state-specific delays, or path-dependent variability. An item that has been reworked three times is treated identically to one progressing linearly, because the simulation has no awareness of process structure. This limitation produces forecasts that cannot capture the variance explained by process dynamics.

The third problem is **technical fragmentation**. The techniques required for process-aware forecasting, including process mining for model discovery, Markov chains for transition modeling, Monte Carlo simulation for distribu-

tional prediction, and machine learning for case-level classification, exist independently in the literature. Their integration into a coherent pipeline, where the output of each technique serves as input to the next, has not been formalized or empirically validated in the software engineering domain. Researchers and practitioners who wish to combine these techniques must construct *ad hoc* integrations without guidance from established methods.

1.2 OBJECTIVES

The growing volume of data generated throughout the Software Development Life Cycle (SDLC) offers relevant opportunities for analyzing and forecasting the behavior of software development processes. Systems such as version control repositories, *issue* management platforms, and continuous integration *pipelines* record in detail the activities performed during development. However, a large portion of the approaches used in practice to estimate project deadlines and outcomes still relies on aggregated metrics and heuristic methods that treat development processes as opaque systems. As a result, these approaches have limitations in capturing the structural dynamics of processes and the variability associated with different execution paths.

In this context, this research investigates the potential of process mining, combined with stochastic modeling and machine learning techniques, to improve the forecasting of software development processes.

1.2.1 GENERAL OBJECTIVE

To develop a computational method for the probabilistic forecasting of software development processes, based on the integration of process mining, stochastic modeling through Markov chains, Monte Carlo simulation, and machine learning, capable of transforming data from software repositories into analytical models that enable the prediction of the behavior and outcomes of these processes.

1.2.2 SPECIFIC OBJECTIVES

To achieve the general objective and test the proposed hypotheses, the following specific objectives are defined:

1. Build structured event logs from data originating from software repositories, including issue tracking systems, version control repositories, and continuous integration pipelines, in order to formally represent the actual behavior of development processes.
2. Apply process mining techniques to discover models that represent the actual flow of activities in software development, allowing the identification of states, transitions, and execution patterns of the process..
3. Develop a probabilistic prediction model based on Markov chains and Monte Carlo simulation, capable of estimating execution time distributions, process trajectories, and possible outcomes of development processes.
4. Empirically evaluate the proposed method by integrating it with machine learning techniques and comparing it with predictive models based solely on traditional software repository metrics, analyzing the gains in predictive accuracy and interpretability..

1.3 RESEARCH QUESTIONS

The research is guided by the following central question: Can the explicit incorporation of process structure, obtained through process mining and stochastic modeling techniques, improve the accuracy and interpretability of forecasting software development processes compared to approaches based exclusively on traditional software repository metrics?

H1 (Predictive Superiority of Process-Derived Features): *Predictive models that incorporate features derived from process structure achieve superior predictive performance compared with models based solely on traditional software repository metrics.*

This hypothesis evaluates whether process-aware representations provide additional explanatory and predictive power beyond conventional repository-based indicators.

H2 (Fidelity and Interpretability of Markov-Based Stochastic Models): *Stochastic models based on Markov chains enable a more faithful representation of the dynamics of software development processes, allowing the generation of interpretable probabilistic forecasts.*

This hypothesis examines whether state-transition models can adequately capture process dynamics and support probabilistic predictions that are both analytically grounded and interpretable.

H3 (Performance Gain from Multi-Technique Integration): *The integration of process mining, Monte Carlo simulation, and machine learning improves forecast quality when compared with the isolated use of each of these techniques.*

This hypothesis assesses whether combining complementary analytical techniques produces better forecasting performance than applying each method independently.

1.4 JUSTIFICATION AND RESEARCH GAP

Several systematic reviews and surveys have been published in areas adjacent to the scope of this research, each covering a distinct facet of the landscape without addressing their convergence in the software engineering domain.

In the process mining literature, Augusto et al. (??) conducted a comprehensive review of automated process discovery algorithms. Burattin (??) formalized the emerging subfield of streaming process mining. Neither work addresses the specific challenges of applying process mining to software development event logs, where case correlation across heterogeneous tools constitutes a prerequisite not present in traditional BPM scenarios.

In the Mining Software Repositories (MSR) community, Hassan (??) established a foundational research roadmap, and Kagdi, Collard, and Maletic (??) produced an extensive taxonomy of MSR approaches. These works focus predominantly on mining individual artifacts (source code, commits, bug reports) rather than mining the process flows that connect these artifacts throughout the development lifecycle.

In the predictive and prescriptive monitoring literature, Márquez-Chamorro et al. (??) mapped predictive monitoring techniques and Kubrak et al. (??) mapped the state of prescriptive process monitoring. Both surveys treat business processes generically, without examining the specific context of software development.

In the stochastic forecasting literature for software projects, Vacanti (??) popularized throughput-based Monte Carlo simulation for agile delivery forecasting, while Whittaker and Thomason (??) pioneered Markov chain models for software testing. Neither work connects these stochastic methods to process mining-discovered models.

Table 1.1 synthesizes the positioning of this research relative to existing

surveys and methods, demonstrating that the addressed intersection, combining process mining, stochastic modeling, and SDLC-specific application, remains unoccupied.

Table 1.1: Positioning relative to existing work.

Work	PM	SDLC Focus	Markov/ MC Sim	ML	Integrated Pipeline
Cook & Wolf (1998)	Yes	Discovery	No	No	No
Kagdi et al. (2007)	No	Artifacts	No	No	No
Hassan (2008)	No	Artifacts	No	No	No
Poncin et al. (2011)	Yes	Discovery	No	No	No
Whittaker & Thomason (1994)	No	Testing	Markov	No	No
Vacanti (2015)	No	Forecasting	MC	No	No
Augusto et al. (2019)	Yes	No	No	No	No
Márquez-Chamorro et al. (2018)	Yes	No	No	Yes	No
Kubrak et al. (2022)	Yes	No	No	Yes	No
Rozinat et al. (2009)	Yes	No	MC	No	Partial
This thesis (2026)	Yes	SDLC	Both	Yes	Yes

The table reveals two patterns. First, works that apply process mining to SDLC data stop at descriptive analysis (discovery), without advancing to stochastic modeling or prediction. Second, works that employ stochastic methods for software forecasting (Vacanti, Whittaker) operate without process mining, treating the process as either a black box (throughput sampling) or a theoretical construct (usage-model Markov chains). The proposed method occupies the intersection: process mining provides the empirical process model that parameterizes the stochastic simulation, applied to the SDLC domain.

1.5 EXPECTED CONTRIBUTIONS

This thesis aims to make the following contributions:

C1 (Method): A formally specified method (PATHCAST) that integrates process mining, Markov chain estimation, Monte Carlo simulation, and machine learning into a four-stage pipeline for probabilistic forecasting of software development processes. The method is defined through inter-stage data contracts, formal properties, and correctness guarantees.

C2 (Event Log Construction): A configurable framework for constructing structured event logs from heterogeneous software repositories, addressing the case correlation problem through a cascade of explicit, structural, and heuristic

strategies with formal quality metrics. This framework is reusable independently of the forecasting pipeline.

C3 (Process-Aware Stochastic Forecasting): An empirically validated demonstration that Monte Carlo simulation parameterized by Markov chain models estimated from process mining-discovered models produces more accurate and better-calibrated forecasts than throughput-based Monte Carlo, for software development processes.

C4 (Process-Derived Feature Set): A novel set of features derived from process mining and Markov analysis (conformance scores, rework counts, Markov-expected remaining times) for machine learning-based outcome prediction in software processes, with empirical comparison against traditional MSR features.

C5 (Empirical Evidence): A reproducible empirical evaluation on 48 open-source software repositories, providing baseline results for stochastic process forecasting in the SDLC domain and a reusable dataset for future research.

1.6 RESEARCH METHODOLOGY

This research adopts the Design Science Research (DSR) paradigm, following the framework of Hevner et al. (??) and the methodology of Peffers et al. (??). DSR is the appropriate paradigm because the primary goal is to design, implement, and evaluate an artifact (the PATHCAST method and its supporting tool) that addresses a practical problem identified in the literature.

The DSRM process is instantiated as follows:

Activity 1 (Problem Identification): The research problem is identified through analysis of the literature at the intersection of process mining, MSR, and stochastic forecasting, revealing the descriptive ceiling, structural blindness, and technical fragmentation described in Section 1.1.

Activity 2 (Objectives): The objectives are derived from the identified gaps: a method that advances beyond descriptive analysis, incorporates process structure into forecasting, and integrates complementary techniques into a formally specified pipeline.

Activity 3 (Design and Development): The PATHCAST method is designed as a four-stage pipeline with formal inter-stage contracts, implemented as a Python-based tool using PM4Py for process mining, NumPy for Markov computation, and scikit-learn/XGBoost for machine learning.

Activity 4 (Demonstration): The method is demonstrated on 48 open-source software repositories, producing event logs, process models, Markov estimates, Monte Carlo forecasts, and ML predictions.

Activity 5 (Evaluation): The evaluation is structured into two complementary stages:

- **E1 (Pipeline Validation):** Assesses the quality of event log construction (coverage, correlation rate) and process discovery (fitness, precision, generalization) across the 48 repositories. Addresses RQ1.
- **E2 (Forecasting Accuracy):** Compares the proposed process-aware Monte Carlo against baselines (throughput-based MC, analytical Markov, historical mean) using temporal hold-out validation. Evaluates ML feature sets through cross-validated comparison. Addresses RQ2 and RQ3.

Activity 6 (Communication): Results are communicated through this thesis and through submissions to venues in empirical software engineering and process mining (e.g., EASE, MSR, ICPM).

Figure 1.1 illustrates the DSRM process applied to this research.

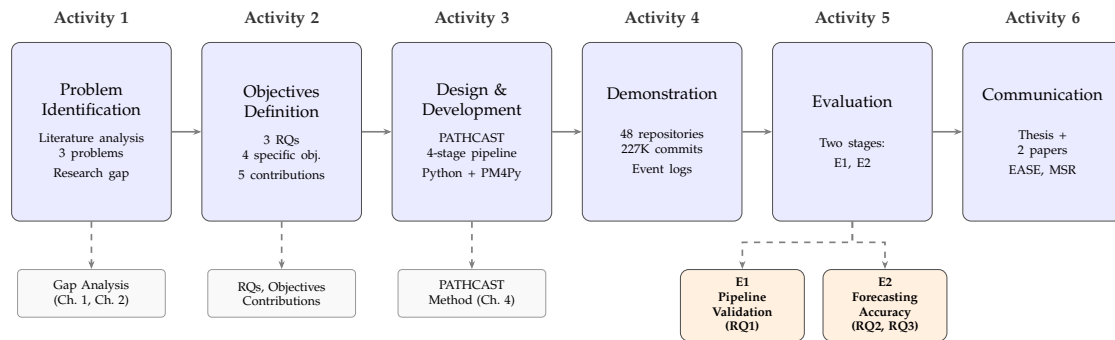


Figure 1.1: Overview of the DSRM process applied to this research, with six activities, produced artifacts, and a two-stage evaluation.

The methodology is detailed in Chapter 5.

1.7 STRUCTURE OF THE DOCUMENT

The remainder of this document is organized as follows.

Chapter 2 presents the theoretical foundations, covering process mining, mining of software repositories, Markov chains for software process modeling, Monte Carlo simulation for process forecasting, and machine learning for software engineering.

Chapter 3 presents a systematic literature review following Kitchenham and Charters' guidelines, mapping the state of the art at the intersection of process mining, stochastic modeling, and machine learning applied to software process forecasting, and identifying the research gap addressed by the proposed method.

Chapter 4 presents the proposed method (PATHCAST), formalizing the four-stage pipeline with input/output contracts, algorithms, formal properties, and theoretical guarantees.

Chapter 5 details the research methodology, including the Design Science Research framework, the experimental design for the two evaluation stages, and threats to validity.

Chapter 6 describes the implementation architecture, technology choices, and design decisions.

Chapter 7 presents the empirical evaluation, including the results of pipeline validation (E1) and forecasting accuracy assessment (E2), with statistical analysis and comparison against baselines.

Chapter 8 discusses the results, identifies limitations, proposes directions for future work, and presents the conclusions.

2

Theoretical Foundation

This chapter states the formal vocabulary on which PATHCAST is built. Six concepts are introduced in succession: process mining (Section 2.1), mining of software repositories (Section 2.2), Markov chains (Section 2.3), Monte Carlo simulation and the probabilistic forecasting metrics it requires (Section 2.4), machine learning for software engineering (Section 2.5), and the entropy of trace variants (Section 2.6). Each section is intentionally short: its purpose is to fix notation and quote the central property used elsewhere in the thesis. Derivations, algorithmic internals and worked examples are deferred to Appendices A–D. The literature gap that motivates the method is synthesised in Chapter 3, not here.

2.1 PROCESS MINING

Process mining extracts behavioural knowledge from event logs recorded by information systems (??). Three classes of analysis are distinguished: *discovery* extracts a model M from a log $\mathcal{L}$ alone; *conformance* compares $\mathcal{L}$ against a given M ; *enhancement* enriches M with information derived from $\mathcal{L}$. PATHCAST uses all three.

Definition 2.1 (Event, Trace, Event Log). *An event $e = (c, a, t, d)$ is a tuple with case identifier $c \in \mathcal{C}$, activity $a \in \mathcal{A}$, timestamp $t \in \mathbb{R}^+$ and attribute map $d \in \mathcal{D}$. Projections $\kappa(e), \alpha(e), \tau(e)$ return case, activity and timestamp. A trace $\sigma_c = \langle e_1, \dots, e_n \rangle$ is the timestamp-ordered sequence of events of case c . An event log $\mathcal{L} =$*

$\{\sigma_{c_1}, \dots, \sigma_{c_N}\}$ is a multiset of traces.

Definition 2.2 (Alignment and Alignment-Based Fitness). *An alignment γ between trace σ and model M is a sequence of synchronous, log and model moves (??). With γ^* the cost-optimal alignment,*

$$\text{fitness}(\sigma, M) = 1 - \frac{\text{cost}(\gamma^*)}{\text{cost}(\gamma_{\text{worst}})}, \quad (2.1)$$

$$\text{conf}(c) = \frac{|\{(s_i, m_i) \in \gamma_c^* : s_i = m_i\}|}{|\gamma_c^*|}. \quad (2.2)$$

Log-level fitness is the trace-weighted average. The case-level conformance $\text{conf}(c)$ is reused as an ML feature in Chapter 4.

PATHCAST adopts the Inductive Miner with infrequent-behaviour filtering (IMf) (??) because it is the only discovery algorithm that guarantees soundness by construction, producing a block-structured process tree compatible with SDLC workflows. Discovery quality is quantified along four dimensions—fitness, precision, generalisation and simplicity (??); precision and generalisation are estimated by the ETC metric and state-visit frequency analysis, respectively. The internals of IMf, the alignment search and the precision/generalisation estimators are presented in Appendix A.

2.2 MINING SOFTWARE REPOSITORIES

Mining Software Repositories (MSR) extracts evidence from version control, issue trackers, code review platforms and CI/CD pipelines (????). The bridge with process mining was established by Cook and Wolf (??) and formalised by Poncin et al. (??): software events are reframed as sequences belonging to process instances.

Modern toolchains emit heterogeneous, partly overlapping streams: issue-tracker status transitions (Jira, GitHub Issues), version-control commits (Git, SVN), code-review activities (PRs, Gerrit) and CI/CD outcomes (Jenkins, GitHub Actions). The empirical challenge is integration (????): a unified event log requires identifier mapping, temporal alignment and semantic harmonisation across tools. Curated datasets and collectors—SEART GitHub Search (??), GH Archive (??), SmartSHARK (??), GrimoireLab (??) and the Public Jira

Dataset (??)—reduce candidate-generation cost without eliminating known validity threats (open-source bias, bot noise, incompleteness). The construction sub-tasks (extraction, case correlation, activity mapping) and their archaeological caveats (stratigraphy, contextual interpretation, taphonomic bias (??)) are operationalised by PATHCAST Stage 1 (Chapter 4). Software-specific construction patterns are catalogued in Appendix C, Section C.4.

2.3 STOCHASTIC PROCESS MODELING WITH MARKOV CHAINS

Software delivery evolves through discrete states (Backlog, In Progress, Review, Testing, Done) under queuing, rework and variable service times. Discrete-time Markov chains (DTMCs) provide the mathematical formalism for the state-transition layer of PATHCAST (????).

Definition 2.3 (Discrete-Time Markov Chain). *A stochastic process $\{X_t\}_{t \in \mathbb{N}}$ on finite state space $S = \{1, \dots, m\}$ is a DTMC if*

$$P(X_{t+1} = j \mid X_t, \dots, X_0) = P(X_{t+1} = j \mid X_t). \quad (2.3)$$

The (time-homogeneous) chain is fully described by a row-stochastic transition matrix $P = [p_{ij}]$, with $p_{ij} = P(X_{t+1} = j \mid X_t = i)$ satisfying $p_{ij} \geq 0$ and $\sum_j p_{ij} = 1$ (Definition 2.4).

Definition 2.4 (Transition Matrix). *P as in Definition 2.3; multi-step probabilities follow the Chapman–Kolmogorov relation $P^{(n)} = P^n$.*

Definition 2.5 (Absorbing Chain and Fundamental Matrix). *For an absorbing chain with t transient and r absorbing states the canonical form is*

$$P = \begin{pmatrix} Q & R \\ \mathbf{0} & I_r \end{pmatrix}, \quad (2.4)$$

$$N = (I_t - Q)^{-1} = \sum_{k=0}^{\infty} Q^k, \quad (2.5)$$

where N_{ij} is the expected number of visits to transient state j starting from transient state i before absorption (Definition 2.6).

Definition 2.6 (Fundamental Matrix). *The matrix N in Equation 2.5 is well-defined because $\rho(Q) < 1$.*

Theorem 2.1 (Properties of Absorbing Chains). *With $N = (I - Q)^{-1}$: (i) $E[T_i] = (N\mathbf{1})_i$; (ii) $\text{Var}(T_i) = (N(2N_{dg} - I)\mathbf{1})_i - E[T_i]^2$; (iii) $B = NR$ is the absorption probability matrix. Proofs in Appendix A.*

Counts c_{ij} of observed $i \rightarrow j$ transitions yield the maximum likelihood estimator

$$\hat{p}_{ij} = \frac{c_{ij}}{\sum_k c_{ik}}; \quad (2.6)$$

Bayesian smoothing under a symmetric Dirichlet prior is derived in Appendix A. Augmenting a DTMC with state-dependent sojourn distributions $\{H_{ij}\}$ yields a semi-Markov process (??):

Definition 2.7 (Semi-Markov Process). *The pair (DTMC $\{X_n\}$, sojourn distributions $\{H_{ij}\}$) under which the process remains in X_n for a random time drawn from $H_{X_n, X_{n+1}}$ before transitioning.*

The first-order assumption hides history dependence and time-varying transitions; PATHCAST mitigates both through optional state-space expansion and the ML residual signal (Section 2.5).

2.4 MONTE CARLO SIMULATION AND PROBABILISTIC FORECASTING METRICS

Monte Carlo simulation estimates statistical properties of intractable systems through repeated random sampling (????). In PATHCAST, MC generates synthetic process trajectories by alternating sojourn draws from $\hat{F}_s$ and next-state draws from $P[s, :]$ until absorption (Chapter 4, Algorithm 3).

Convergence rests on two classical results. The strong law of large numbers gives, for i.i.d. $X_1, X_2, \dots$ with $E[X] = \mu$,

$$\bar{X}_M = \frac{1}{M} \sum_{m=1}^M X_m \xrightarrow{a.s.} \mu. \quad (2.7)$$

The central limit theorem yields, with $\text{Var}(X) = \sigma^2 < \infty$,

$$\frac{\bar{X}_M - \mu}{\sigma/\sqrt{M}} \xrightarrow{d} \mathcal{N}(0, 1), \quad (2.8)$$

so for any trajectory functional g the MC estimator and confidence interval are

$$\hat{\mu}_g = \frac{1}{M} \sum_{m=1}^M g(\omega^{(m)}), \quad (2.9)$$

$$\text{CI} = \hat{\mu}_g \pm z_{\alpha/2} \cdot \frac{\hat{\sigma}_g}{\sqrt{M}}. \quad (2.10)$$

The standard error scales as $O(1/\sqrt{M})$ independently of dimension, and seed control via the Mersenne Twister makes the simulation exactly reproducible.

Forecasting metrics. A distributional forecast must be evaluated jointly for sharpness and calibration. The continuous ranked probability score (??) is the operational choice in this thesis:

Definition 2.8 (CRPS). *For predicted distribution $\hat{F}$ and observed value y ,*

$$\text{CRPS}(\hat{F}, y) = \int_{-\infty}^{\infty} [\hat{F}(t) - \mathbf{1}(t \geq y)]^2 dt. \quad (2.11)$$

CRPS is a strictly proper scoring rule and reduces to mean absolute error when $\hat{F}$ degenerates to a point mass, so distributional forecasters and point baselines remain comparable on a common scale. Calibration is summarised by the aggregate calibration error

$$\text{ACE} = \frac{1}{|\mathcal{A}|} \sum_{\alpha \in \mathcal{A}} |\hat{p}_\alpha - \alpha|, \quad (2.12)$$

averaged across decile checkpoints. Worked CRPS-versus-MAE derivations and numerical examples are in Appendix B.

2.5 MACHINE LEARNING FOR SOFTWARE ENGINEERING

Machine learning is widely applied to defect prediction, effort estimation and code quality assessment (????); process-oriented prediction (throughput, workflow optimisation) remains comparatively underexplored (??). PATH-CAST formulates outcome prediction as supervised learning: a binary classifier $\hat{f}: \mathbb{R}^d \rightarrow \{0, 1\}$ for on-time versus delayed cases, and a regressor $\hat{f}_T: \mathbb{R}^d \rightarrow \mathbb{R}^+$ for lead time, both trained on a feature vector $\mathbf{x}_c \in \mathbb{R}^d$ derived per case. The candidate algorithms (Random Forest (??), XGBoost (??), logistic/linear regression baselines, LSTM (??) for the sequence comparator) and the training protocol

(temporal hold-out, hyperparameter search, SHAP interpretation) are specified in Chapter 5. The working hypothesis is that process-derived features—conformance, rework count, expected remaining transitions, completion probability, path entropy—add predictive value beyond traditional MSR features; the feature catalogue and the PM-only/MSR-only/Combined comparison protocol are in Chapter 4, Table 4.4.

2.6 VARIANT ENTROPY

The trace-variant distribution $V = \{v_1, \dots, v_k\}$ with empirical frequencies $p(v_i)$ summarises the diversity of execution paths in an event log. The Shannon entropy

$$H(V) = - \sum_{v \in V} p(v) \log_2 p(v) \quad (2.13)$$

quantifies path uncertainty in bits; its normalised form $H_{\text{norm}}(V) = H(V)/\log_2 |V| \in [0, 1]$ is a scale-independent process-complexity proxy. Higher H_{norm} signals greater behavioural heterogeneity and is empirically correlated with lower forecast confidence; PATHCAST uses it as both a diagnostic indicator at Stage 2 (Section 4.3.1) and a feature of the ML component.

2.7 CHAPTER SYNTHESIS

The six concepts compose the PATHCAST pipeline as follows: process mining yields the discovered backbone and the per-case conformance feature; mining of software repositories provides the heterogeneous event sources from which that log is built; the Markov chain layer turns the backbone into a probabilistic state-transition model with closed-form expressions for expected completion time and absorption probabilities; Monte Carlo simulation, paired with CRPS and ACE, produces and evaluates the distributional forecasts; machine learning consumes process-derived features alongside MSR features to refine the point predictions; and the entropy of variants quantifies the process complexity that constrains forecast confidence. The empirical evidence that this integration has not been previously proposed for software development is the subject of Chapter 3.

3

Systematic Literature Review

3.1 INTRODUCTION

This chapter presents a systematic literature review (SLR) conducted to map the state of the art at the intersection of process mining, stochastic modeling, and machine learning applied to software process forecasting. The review follows the guidelines established by Kitchenham and Charters (??) for evidence-based software engineering, complemented by the snowballing procedures described by Wohlin (??) and the mapping study guidelines of Petersen et al. (??). The primary objective is to identify, classify, and synthesize existing approaches that combine process-aware techniques with predictive modeling for software development outcomes, establishing the research gap that motivates PATHCAST proposed in Chapter 4.

The review addresses three overarching research questions, each decomposed into two sub-questions covering the application landscape of process mining in SDLC contexts, the techniques and prediction methods employed, and the degree of integration between process mining and stochastic forecasting methods.

3.2 REVIEW PROTOCOL

Title/abstract screening was conducted with the assistance of a large language model as the primary filter at the title-and-abstract stage, operating on

contexts enriched with baseline-document exemplars. All inclusion decisions were verified manually by the author; the cross-model inter-rater check and the prompts used are reported as replication artefacts in Appendix F.¹

3.2.1 RESEARCH QUESTIONS

The SLR is guided by the following research questions, structured to progressively map the landscape from application scope to technique integration:

Table 3.1: Research questions and expected outputs.

RQ	Sub-RQ	Question	Expected Output
RQ1	RQ1.1	Which SDLC phases have been analyzed with process mining?	Frequency distribution by phase
RQ1	RQ1.2	What event log sources are used and how are they constructed?	Taxonomy of sources and construction methods
RQ2	RQ2.1	Which process discovery algorithms have been applied to SDLC?	Comparative matrix
RQ2	RQ2.2	Which predictive and stochastic techniques have been used?	Technique classification
RQ3	RQ3.1	What is the level of integration between PM and stochastic methods?	Integration maturity assessment
RQ3	RQ3.2	What gaps exist for process-aware forecasting in the SDLC?	Gap analysis matrix

3.2.2 SEARCH STRATEGY

The search strategy combines automated database search with backward and forward snowballing. The automated search targets five digital libraries: IEEE Xplore, ACM Digital Library, Scopus, Web of Science, and SpringerLink.

PICO DECOMPOSITION

The search string construction follows the PICO framework adapted for software engineering, as shown in Table 3.2.

¹The use of LLMs as primary screeners in systematic literature reviews has been studied by Syriani et al. (??) and Khraisha et al. (??).

Table 3.2: PICO decomposition for search string construction.

Facet	Role	Concepts
Population	Application domain	Software engineering, SDLC, software process, DevOps, CI/CD
Intervention	Analytical technique	Process mining, Markov chain, Monte Carlo, predictive PM, ML
Comparison	N/A	(omitted)
Outcome	Result	Forecasting, prediction, lead time, cycle time, process model

SEARCH STRINGS

Three complementary search strings are executed to maximize coverage while maintaining precision.

Principal String ($P \wedge I$). Block P captures the software engineering context:

("software engineering" OR "software development" OR "software process" OR "software lifecycle" OR "software life cycle" OR "SDLC" OR "development lifecycle" OR "development life cycle" OR "DevOps" OR "CI/CD" OR "continuous integration" OR "continuous delivery" OR "agile development" OR "agile process" OR "issue tracking" OR "bug tracking" OR "version control" OR "code review" OR "pull request" OR "software project" OR "software repository" OR "software maintenance" OR "software evolution")

Block I captures the process mining and stochastic modeling techniques:

("process mining" OR "process discovery" OR "conformance checking" OR "workflow mining" OR "predictive process monitoring" OR "process prediction" OR "remaining time prediction" OR "process forecasting" OR "event log" OR "process simulation" OR "Monte Carlo simulation" OR "Markov chain" OR "Markov model" OR "stochastic process model" OR "transition probability" OR "absorbing chain" OR "lead time prediction" OR "cycle time prediction")

Complementary String ($MSR \rightarrow$ Process Modeling). Captures studies using MSR terminology without canonical process mining terms:

*("mining software repositories" OR "software repository mining"
OR "GitHub mining" OR "Jira mining") AND ("process model" OR "Markov"
OR "Monte Carlo" OR "stochastic" OR "lead time" OR "cycle time"
OR "throughput" OR "transition matrix")*

Frontier String (Stochastic $\wedge$ PM $\wedge$ SE). Targets the intersection of stochastic methods with process mining in software contexts:

("Monte Carlo" OR "Markov chain" OR "stochastic simulation" OR "probabilistic forecast" OR "transition matrix") AND ("process mining" OR "event log" OR "workflow mining" OR "process model" OR "business process") AND ("software development" OR "software engineering" OR "software process" OR "DevOps" OR "SDLC" OR "issue tracker" OR "commit log" OR "build pipeline")

The strings are adapted to each database's syntax and field conventions as summarized in Table 3.3. The search scope covers publications from January 1994 to December 2026.

Table 3.3: Database-specific search adaptations and raw retrieval counts.

Database	Search Field	Strings	Filters	Records
Scopus	TITLE-ABS-KEY	3	ar, cp; EN; 1994–2026	2,377
IEEE Xplore	All Metadata	4 (split)	Conf. + Journals; 1994–2026	1,386
Web of Science	TS (Topic)	3	Article, Proc.; EN; 1994–2026	997
SpringerLink	Simplified	8 queries	Chapter, Article; EN	2,884
ACM DL	Abstract	2	Date: 1994–2026	96
Snowballing	–	–	Backward + forward	595
Control set	–	–	Manual	12
Total				8,347

SNOWBALLING PROCEDURE

Backward snowballing examines the reference lists of all included primary studies. Forward snowballing uses Google Scholar citation tracking to identify subsequent works that cite the included studies. Both rounds apply the same inclusion and exclusion criteria defined in Section 3.2.3.

SEARCH STRING VALIDATION

The search strings are validated against a control set of 10 papers that are known to be relevant to the review scope (Table 3.4). The acceptance threshold is $\geq 8/10$ papers found; the validation returned 10/10 (100%, PASS).

Table 3.4: Control papers for search string validation.

#	Paper	Justification	Captured via
V1	Cook & Wolf (1998)	Seminal PM + SE	ACM
V2	Rubin et al. (2007)	PM framework for SW processes	Scopus
V3	Poncin et al. (2011)	PM + MSR intersection	IEEE
V4	Caldeira & Cardoso (2019)	PM + SW process assessment	IEEE
V5	Lemos et al. (2011)	PM for SW process management	IEEE
V6	Whittaker & Thomason (1994)	Markov + SW testing (seminal)	Scopus
V7	Tax et al. (2017)	PPM seminal (BPM Control baseline)	Control
V8	Rogge-Solti & Weske (2015)	Stochastic + PM forecasting	Scopus
V9	Rozinat et al. (2009)	PM $\rightarrow$ Simulation bridge	Control
V10	Gupta & Sureka (2017)	PM multi-repository in SE	ACM

3.2.3 INCLUSION AND EXCLUSION CRITERIA

The inclusion criteria are organized in two tiers. Formal criteria (IC1–IC3) filter on publication attributes and are applied during the database search configuration. Content criteria (IC4a–IC4d) require that the study’s research contribution addresses at least one of four specific technique-domain intersections; they are applied during title/abstract and full-text screening. Exclusion criteria (EC1–EC5) are applied cumulatively across both screening phases. Criteria EC1–EC4 are evaluated by the LLM-assisted screener; EC5 (full text inaccessible) is applied programmatically after the inter-library-loan recovery pass for Band D/E papers fails to retrieve a copy. In the auxiliary tier, EC5 is extended to cover non-recoverable metadata insufficiency: papers for which no usable ab-

stract could be recovered after the multi-source enrichment cascade, and papers for which the primary and verifier raters produced an irreconcilable decision without accessible full text, are closed under EC5 rather than escalated to human arbitration.

At least one of IC4a–IC4d must be satisfied for inclusion. Criterion IC4b ensures that seminal works applying Markov chains to software processes—such as Whittaker and Thomason (??)—are captured despite predating the term “process mining.”

3.2.4 QUALITY ASSESSMENT

Each included study is assessed against a quality checklist adapted from Dybå and Dingsøyr (??), with eight binary criteria:

Studies scoring below 4.0/8 (50%) are excluded from the synthesis. QA8 reflects the centrality of process model quality in the PATHCAST pipeline.

QA scoring is performed by the same LLM-assisted protocol used for screening, with a strict per-criterion rubric returning a binary score and a brief justification. The full prompt and the per-study scores are part of the replication package (Section 3.6.5). Table 3.7 reports the quality-assessment outcome for the complete confirmed set ($n = 169$). All 169 studies were scored.

The 33 studies below the QA threshold are predominantly short workshop papers, position papers, or extended abstracts that fail QA3 (reproducible data source) and QA4 (formally defined technique) simultaneously. They are not removed from the PRISMA flow; they are retained in the included set as context but are not used in the F1–F5 evidence base reported in Section 3.5.1.

3.2.5 DATA EXTRACTION

For each included study, the attributes in Table 3.8 are extracted.

3.3 SEARCH RESULTS AND STUDY SELECTION

3.3.1 DATABASE RETRIEVAL AND DEPLICATION

The three search strings executed across five databases returned 7,740 records. Backward and forward snowballing from the control set yielded an additional

Table 3.5: Inclusion and exclusion criteria.

Type	ID	Criterion
Inclusion	IC1	Publication period: 1994–2026 (from Whittaker & Thomason onward).
	IC2	Language: English.
	IC3	Peer-reviewed journal article, conference paper, or workshop paper (≥ 4 pages).
	IC4a	Applies process mining techniques (discovery, conformance checking, predictive process monitoring) to software development artifacts (commits, issues, pull requests, CI/CD logs, bug trackers, VCS).
	IC4b	Uses stochastic models (Markov chains, Monte Carlo, stochastic Petri nets, absorbing chains) to analyze software development processes or workflows.
	IC4c	Proposes or evaluates forecasting of software process metrics (lead time, cycle time, remaining time, throughput, defect rates) from event logs or repository data.
Exclusion	IC4d	Mines software repositories (GitHub, Jira, VCS, CI/CD platforms) to discover, analyze, or improve software process models.
	EC1	Application domain is exclusively outside software development (healthcare, manufacturing, supply chain) without demonstrated relevance to SDLC processes.
	EC2	Software appears only as the implementation platform; the process being analyzed is not a software development process.
	EC3	Purely algorithmic or theoretical contribution without empirical evaluation in a software development context.
	EC4	Secondary study (SLR, mapping study, survey) not specifically addressing PM or stochastic methods in software engineering.
	EC5	Full text or decision-critical metadata inaccessible despite all retrieval attempts (open access, institutional, ILL, author-provided preprint, multi-source abstract enrichment). Applied programmatically after Band D/E recovery and auxiliary enrichment; also closes irreconcilable ambiguous cases when no accessible full text exists.

Table 3.6: Quality assessment checklist.

#	Criterion	Score
QA1	Are the research objectives clearly stated?	0/1
QA2	Is the software engineering context described in detail?	0/1
QA3	Is the data source (event log) described reproducibly?	0/1
QA4	Is the PM or stochastic technique formally defined?	0/1
QA5	Are results validated empirically?	0/1
QA6	Are threats to validity discussed?	0/1
QA7	Is the study reproducible (data and/or code available)?	0/1
QA8	Are process model quality metrics reported (fitness, precision)?	0/1

Table 3.7: Quality-assessment results for the confirmed included set ($n = 169$; full QA applied). Auto-generated from pipeline/qa_llm.py.

QA Outcome	Count	Notes
Studies assessed	169	Absolute count and 100.0% of included set
Studies with score $\geq 4/8$	136	Retained for synthesis
Studies with score $< 4/8$	33	Excluded after QA
Mean QA score	4.88	Mean and standard deviation (1.40)
Median QA score	5.00	Median and interquartile range (4.00–6.00)

Table 3.8: Data extraction form.

Field	Description
ID	Unique identifier
Authors, Year, Venue	Bibliographic metadata
SDLC Phase(s)	Lifecycle phases covered
Event Log Source	Jira, GitHub, GitLab, Jenkins, etc.
Event Log Construction	Manual, semi-automatic, or automatic
PM Technique Category	Discovery / Conformance / Enhancement / Prediction
Specific Algorithm(s)	Alpha, Heuristics, Inductive Miner, etc.
Stochastic Method	Markov / Monte Carlo / Stochastic Petri Net / None
ML Technique	RF, XGBoost, LSTM, etc. / None
Prediction Target	Remaining time, outcome, lead time, delay, etc.
Integration Level	L0 None / L1 Sequential / L2 Pipeline / L3 Unified
Validation Type	Case study / Experiment / Industrial / Simulation
Tool/Platform	PM4Py, ProM, Disco, custom, etc.
Dataset Size	Number of events and cases analyzed
Process Model Quality	Fitness, precision, generalization (if reported)

595 records. Together with 12 control papers maintained as a validation-only set, the initial corpus totaled 8,347 records. Table 3.9 summarizes the retrieval by source.

Table 3.9: Record retrieval by source.

Source	Raw Records	After Internal Dedup
Scopus (6 queries)	2,377	1,852
SpringerLink (10 exports)	2,884	1,543
IEEE Xplore (manual)	1,386	1,263
Snowballing	595	575
Web of Science (4 queries)	997	514
ACM Digital Library (3 queries)	96	34
Control set (validation only)	12	2
Total	8,347	5,783

Cross-source deduplication removed 2,564 duplicate records (30.7%), yielding 5,781 unique primary-corpus papers plus 2 unique control papers. Deduplication used normalized DOI matching, followed by fuzzy title matching (Levenshtein similarity ≥ 0.9) for records without DOIs.

3.3.2 OPERATIONAL WORKING SET CONSTRUCTION

Screening 5,781 records in a single pass would require substantial resources. A working set of 2,340 papers was therefore constructed from the highest-signal primary queries—those whose query design directly targets the PATHCAST intersection (process mining, stochastic modeling, and software development)—as shown in Table 3.10. The remaining 3,441 records constitute a *high-recall auxiliary corpus*, retained as a frozen baseline and available for supplementary analysis if gaps in the primary results warrant expansion.

The working set preserves 10/10 control papers when the validation-only control set is included, and 8/10 from primary queries alone—both above the 80% threshold.

3.3.3 TITLE/ABSTRACT SCREENING

All 2,340 working-set papers were screened against IC4a–IC4d and EC1–EC4 using the LLM-assisted protocol introduced in the review-protocol preamble.

Table 3.10: Working set composition.

Database	Queries included	Unique papers
Scopus	scopus_principal, scopus_complementar, scopus_frenteira, scopus_markov_testing, scopus_stochastic_petri, scopus_simulation_bridge	2,197
IEEE Xplore	ieee_manual_01 (PM + SE subgroup)	97
ACM DL	acm_principal, acm_msr	46
Total		2,340

The screening was executed across five sequential batches of up to 500 papers each via the Anthropic Batches API, producing structured JSON outputs with decision, rationale, and matched criteria for each record.

Table 3.11: Title/abstract screening results.

Decision	Count	%
Include	111	4.7
Maybe	775	33.1
Exclude	1,454	62.2
Total	2,340	100.0

The include set (111 papers) and the maybe set (775 papers) together form the full-text review queue of 886 papers. The exclude decisions were driven primarily by EC1 (domain outside software development, principally generic business process management) and EC3 (theoretical algorithm proposals without SE evaluation).

3.3.4 FULL-TEXT SCREENING

The 886 papers forwarded from the T/A phase were subjected to LLM-assisted full-text screening with a strict binary prompt (decisions limited to `include`, `exclude`, or `pending`; `maybe` is not permitted at this stage). To maximise the number of papers assessable by the LLM, a multi-source abstract-enrichment cascade was executed prior to screening—combining DOI-based lookups via Semantic Scholar, OpenAlex, CORE, and Crossref, followed by title-based fallback

lookups—raising abstract coverage from 16.6% to 72.6% (643 of 886 papers).²

Table 3.12 summarises the outcome.

Table 3.12: Full-text screening outcomes (886 papers forwarded from T/A phase).

Outcome	Count	%
Included	169	19.1
Excluded	717	80.9
EC3 (algorithmic, no SE eval.)	(232)	(26.2)
EC1 (non-SE domain)	(205)	(23.1)
EC5 (full text inaccessible, post-audit active stratum)	(145)	(16.4)
EC2 (SW as implementation)	(110)	(12.4)
EC4 (out-of-scope secondary)	(60)	(6.8)
Total	886	100.0

Note: EC counts are non-exclusive — a single paper may trigger multiple ECs when more than one applies (e.g., a non-SE-domain paper whose contribution is purely theoretical). The EC5 count of 145 is the active post-audit stratum after re-screening the original historical EC5 band of 148 papers and reclassifying 3 of them to EC3. The sum of per-EC counts is $232 + 205 + 145 + 110 + 60 = 752$, which exceeds the 717 excluded papers by 35 (corresponding to papers triggering more than one EC). The percentages above are with respect to the 886-paper FT queue.

Among the 717 excluded papers, EC3 (algorithmic or theoretical contribution without SE evaluation, $n = 232$) and EC1 (domain outside software development, $n = 205$) jointly account for the largest share of FT-stage exclusions. EC5 (full text inaccessible despite all retrieval attempts, active post-audit $n = 145$) corresponds to the residual portion of the original 148-paper Band D/E stratum for which no open-access copy, institutional access, or author-provided preprint could be located; these are recorded in a dedicated traceability file and documented in the PRISMA flow as a named exclusion stratum. EC2 (software as implementation platform rather than analyzed process, $n = 110$) and EC4 (out-of-scope secondary studies, $n = 60$) account for the remainder.

Figure 3.1 presents the complete PRISMA 2020 flow diagram (??) of the selection process.

²Papers were processed in order of a five-band priority scheme: Band A (91 papers, T/A include ≥ 2 ICs), Band B (20, T/A include 1 IC), Band C (39, T/A maybe + abstract), Band D (152, T/A maybe + IC signal, no abstract), Band E (584, T/A maybe, no IC signal, no abstract). Abstract enrichment and LLM re-screening were applied to Bands A, B, C, and the subset of D/E that gained an abstract after enrichment.

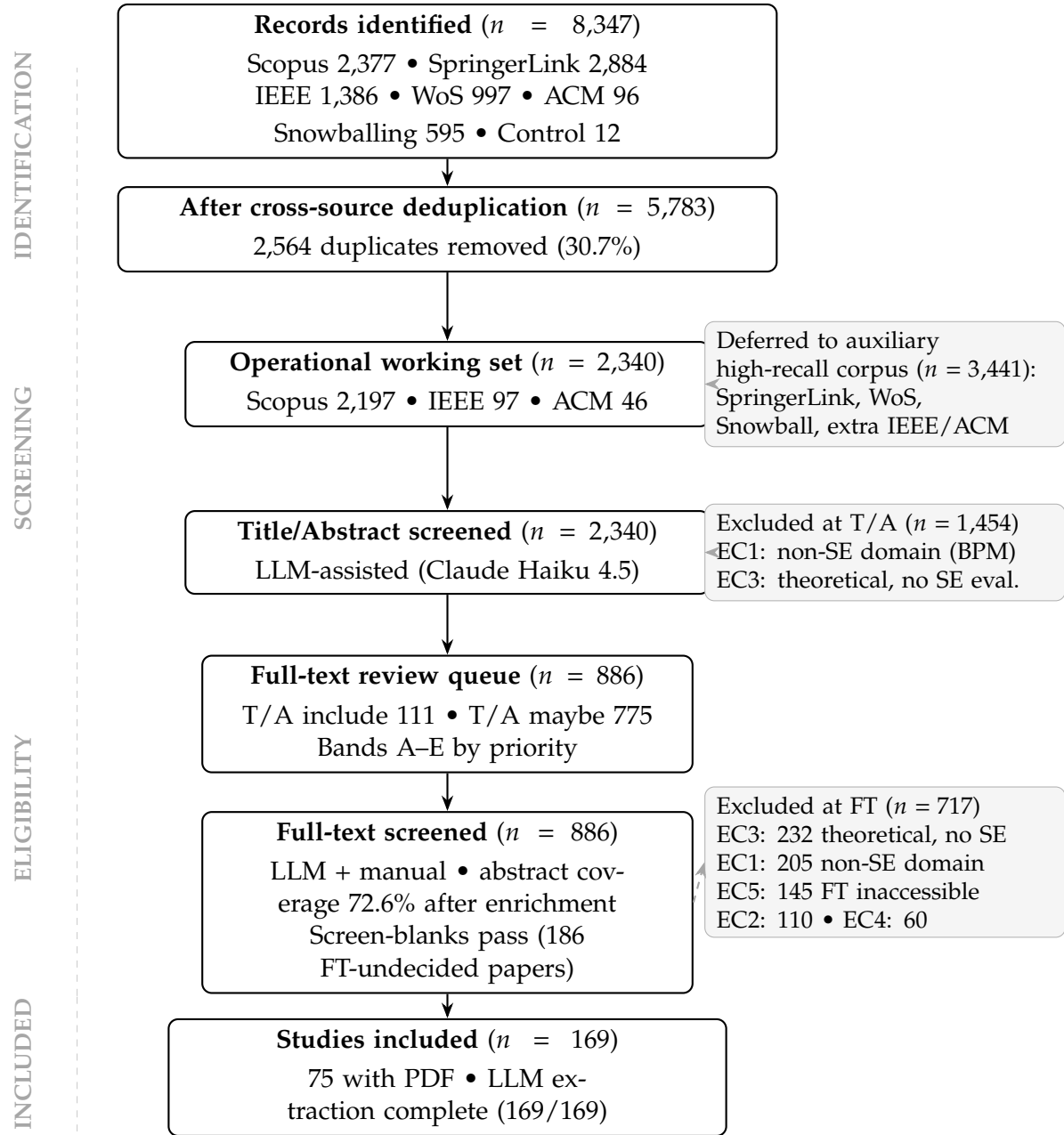


Figure 3.1: PRISMA 2020 flow diagram of the study selection process. Phase labels follow the four-stage structure: Identification, Screening, Eligibility, and Included.

3.4 RESULTS AND SYNTHESIS

3.4.1 OVERVIEW OF INCLUDED STUDIES

The SLR is reported in two complementary tiers: (i) the **working-set tier**, comprising the 2,340-paper operational corpus that was deduplicated and processed through the LLM-assisted T/A and FT screening, yielding **169 confirmed primary studies**; and (ii) the **combined tier**, which adds the 3,807-paper deferred auxiliary corpus after a full T/A and FT pass on the same protocol, yielding **212 additional confirmed studies** in the first auxiliary pass, plus 23 further confirmations from a second auxiliary pass over enriched abstracts, for a combined total of **404 confirmed primary studies**. The working-set tier is the narrative spine of this chapter; the combined-tier numbers are reported in parallel for the findings F1–F5 and provide the recall-bounded view of the SLR. Unless stated otherwise, detailed combined-tier distributions refer to the **381-study analytical subset** frozen before the final +23 auxiliary re-FT confirmations. The 169-study set spans three decades (1995–2026), with a clear acceleration from 2014 onward (Figure 3.2). Seventy-one percent of confirmed papers (working-set tier) were published between 2015 and 2026, reflecting the maturation of open-source process mining toolkits (PM4Py, ProM) and the broader adoption of deep learning frameworks in software engineering research.

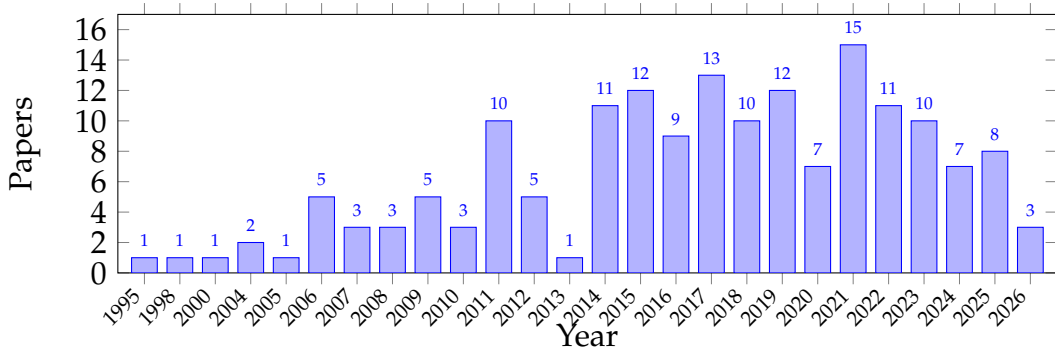


Figure 3.2: Distribution of confirmed studies by publication year ($n = 169$).

By document type, the majority are conference papers and journal articles. By source database, Scopus is the dominant contributor, followed by IEEE Xplore and ACM DL. Data extraction was completed for all 169 studies using a structured LLM-assisted pipeline (*claude-haiku-4-5-20251001*) operating on locally available PDFs (75/169, of which 73 were operationally coded in the

triangulation subset) or abstract/title metadata (94/169).

3.4.2 INCLUSION CRITERION COVERAGE

Figure 3.3 shows the distribution of inclusion criteria among the 169 confirmed papers (criteria are non-exclusive; a paper may satisfy multiple).

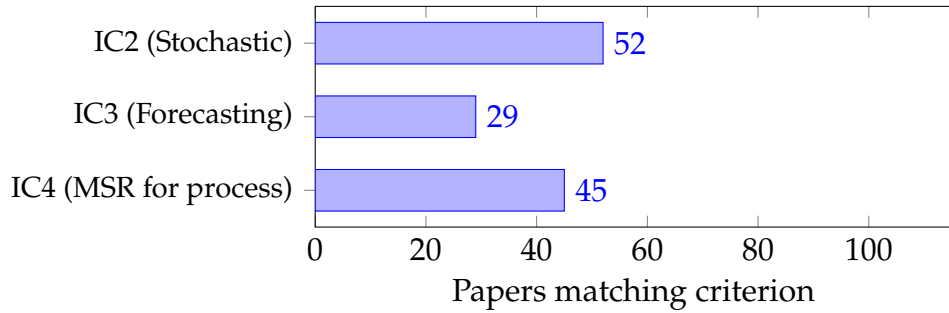


Figure 3.3: Inclusion criteria matched by the confirmed set ($n = 169$; non-exclusive; IC labels map to IC4a–IC4d in the protocol).

IC1/IC4a (process mining applied to SE artifacts) is the dominant criterion, matched by 121 papers (71.6%). IC4/IC4d (repository mining for process modeling) is second at 45 papers (26.6%), reflecting the MSR community’s overlapping contributions. IC2/IC4b (stochastic modeling) appears in 52 papers (30.8%) and IC3/IC4c (forecasting of process metrics) in 29 papers (17.2%). Notably, 68 papers (40.2%) match IC1 only, without stochastic or forecasting components, providing direct empirical evidence for the “Descriptive Ceiling” identified in Finding F1 below.

3.4.3 PREVALENT EXCLUSION PATTERNS

Among the 717 FT-excluded papers (Table 3.12), the largest exclusion reason is EC3 (algorithmic or theoretical contribution without software-engineering evaluation, $n = 232$), indicating a substantial body of process mining and stochastic modeling research that has not been empirically applied to software engineering contexts. EC1 (domain exclusively outside software development, $n = 205$) is the second-largest reason and consists primarily of generic business process management (BPM) studies that use the same terminology as the search strings but apply techniques to organizational processes unrelated to SDLC. EC5 (full text inaccessible, active post-audit $n = 145$; original historical

stratum $n = 148$) closes the group of large strata, followed by EC2 (software as implementation platform rather than analyzed process, $n = 110$) and EC4 (out-of-scope secondary study, $n = 60$). EC counts are non-exclusive: the sum of per-EC counts ($232 + 205 + 145 + 110 + 60 = 752$) exceeds the 717 excluded papers by 35, corresponding to papers that triggered more than one EC simultaneously. The dominance of EC3 and EC1 directly supports Findings F3 and F4 below: process mining and stochastic methods are widely studied in the abstract or in non-SE domains, but their application to SDLC processes remains comparatively narrow.

3.4.4 FULL-TEXT EXTRACTION AND PDF-BASED TRIANGULATION

Structured data extraction was performed on the full 169-study set using the extraction template defined in Table 3.8. For each study, the LLM extracted the research question, study type, research contribution, PM technique, stochastic technique, software artifact, software process, data source, dataset availability, tool used, main finding, limitations, and replication-package status. Title-and-abstract metadata was used as the primary input for studies without locally available PDFs (94/169); for the remaining 75/169 studies a locally accessible PDF was available and used to triangulate against the LLM-extracted fields. Of these 75 PDFs, 73 were fully coded in the operational triangulation spreadsheet used for the figures and positioning tables below.

The 73-paper PDF triangulation subset is therefore not a separate selection stage and does not override the PRISMA counts. It is used in this chapter as a verification layer for the LLM-extracted fields used in Findings F1–F5.

Figure 3.4 summarizes the operational triage profile of the 73 coded PDFs (54 confirm-include, 13 confirm-include with caveats, 6 confirm-exclude after full-text inspection); the formal selection decisions reported in the PRISMA flow (Figure 3.1) are unchanged.

3.4.5 RQ1: APPLICATION LANDSCAPE

RQ1.1: SDLC PHASES COVERED

The structured extraction over the full 169-study set confirms a strongly uneven coverage of the SDLC. **Development activities** (coding, commits, pull requests, repository evolution) dominate at 133 papers (78.7%); **quality-related**

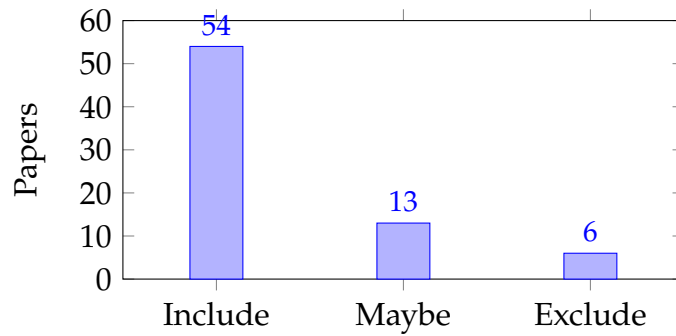


Figure 3.4: Operational decision distribution in the 73-paper PDF triangulation subset.

activities (testing, defect handling, code review, process evaluation) appear in 74 papers (43.8%); **maintenance** appears in 38 papers (22.5%); **requirements engineering** in only 22 papers (13.0%); and **delivery/DevOps contexts** (CI/CD, build and release pipelines) in just 18 papers (10.7%). **Design and architecture** are not the primary focus of any extracted study, and only 3 papers (1.8%) center the analysis on project management.

Two conclusions follow. First, the literature is concentrated on phases where digital traces are abundant and already timestamped, such as commits, issues, test events, and pipeline executions. Second, the early SDLC phases remain structurally underrepresented, which weakens the ability of current approaches to support end-to-end forecasting from requirements to delivery. For PATH-CAST, this supports a pragmatic focus on operational phases with reliable event data while leaving room for future extensions toward requirements and design logs.

RQ1.2: EVENT LOG SOURCES AND CONSTRUCTION

Across the 169-study set, event logs are constructed primarily from four source families: **version-control repositories** (Git/GitHub/SVN commits) appear in 84 papers (49.7%); **issue tracking systems** (especially Jira and GitHub Issues) in 37 papers (21.9%); **IDE interaction logs** in 22 papers (13.0%); and **CI/CD or build pipelines** in 15 papers (8.9%). The common pattern is not a native event log but rather a *derived* event log assembled from repository metadata, commit histories, issue transitions, and (where available) pipeline records.

The dominant construction problem is case correlation across tools. Studies frequently analyze one source in isolation or perform ad hoc joins between

issues and commits, which limits reproducibility. This confirms that event-log construction is not a preparatory detail but a central methodological layer in software process mining. The finding directly justifies Stage 1 of PATHCAST as an explicit transformation pipeline rather than an implicit data-cleaning step.

A non-trivial fraction of studies report data sources only in coarse terms (*repository data, VCS history, open-source projects*) without naming specific projects, time windows, or extraction queries. Across the 169-study set, only 25 papers (14.8%) explicitly state that their dataset is publicly available and only 4 papers (2.4%) declare a replication package. This itself is evidence of incomplete reporting in the literature and provides direct empirical grounding for Finding F2 below.

A complementary body of work constructs Markov models directly from GitHub repository traces, bypassing explicit process discovery in favour of manual state-space definition. Jo et al. (??) modelled developer activities in GitHub repositories as Discrete-Time Markov Chains (DTMCs), finding that commits account for 62.73% of long-term activity, and employed probabilistic temporal logic to extract collaboration patterns relevant to project management. This approach was extended in (??), where Probabilistic Model Checking with Computation Tree Logic was applied across five repositories to characterise temporal dynamics, state-transition probabilities, and key behavioural drivers, yielding transparent and explainable project forecasts. Ortu et al. (??) complemented these structural analyses by combining Markov chains with social network analysis to identify fault-insertion and fault-fixing behavioural patterns across Apache Software Foundation projects, demonstrating that developer expertise significantly moderates defect introduction likelihood. Collectively, these studies confirm that SDLC event logs extracted from public repositories are sufficiently rich to support state-transition modelling, satisfying a foundational prerequisite for PATHCAST Stage 1. However, the event-to-state mapping adopted across these works remains unstandardised and ad hoc, representing a methodological gap that PATHCAST’s process mining discovery stage aims to address systematically.

3.4.6 RQ2: TECHNIQUES AND PREDICTION

RQ2.1: DISCOVERY ALGORITHMS APPLIED TO SDLC

The canonical origin of process mining applied to software development contexts can be traced directly to the foundational contributions of Cook and Wolf in the mid-1990s. In their seminal work, (??) introduced three automated process discovery methods—algorithmic grammar inference, Markov models, and neural networks—demonstrating that finite-state machine representations of software processes could be extracted automatically from event traces, substantially lowering the barrier to formal process modeling. This line of inquiry was consolidated in (??), which refined the event-based discovery methodology and established the conceptual vocabulary of inferring workflow structures from historical execution data, a paradigm that remains central to the field today. Notably, both contributions embedded stochastic reasoning—particularly Markov chain representations—within the discovery pipeline, anticipating the probabilistic extensions that would later characterise mature process mining frameworks. The translation of these principles into an explicit, named framework for software engineering contexts was accomplished by (??), who applied the ProM toolkit to source code management logs and demonstrated multi-perspective analysis spanning control-flow, informational, and organisational dimensions. Together, these three works constitute the seminal arc of IC1, establishing automated discovery from SDLC event logs as both technically feasible and analytically productive, and providing PATHCAST with its foundational methodological lineage.

The complete extracted set confirms that **process mining remains the dominant analytical family**: 121 of 169 studies (71.6%) match IC1 in the form of discovery, conformance checking, or process-oriented repository analysis. At the technique level, the most frequently named PM algorithms across the 169 studies are the **inductive miner** (51 papers), **alpha miner** (45 papers), **conformance checking** (30 papers), and **heuristic miner** (22 papers); 11 papers use social network analysis on developer interactions. The literature is markedly richer in *process representation* than in probabilistic forecasting: only 29 papers (17.2%) match IC3 (forecasting of process metrics), and the dominant contribution type is discovery or conformance checking rather than predictive analytics. This is the transition that PATHCAST targets: moving from discovered process

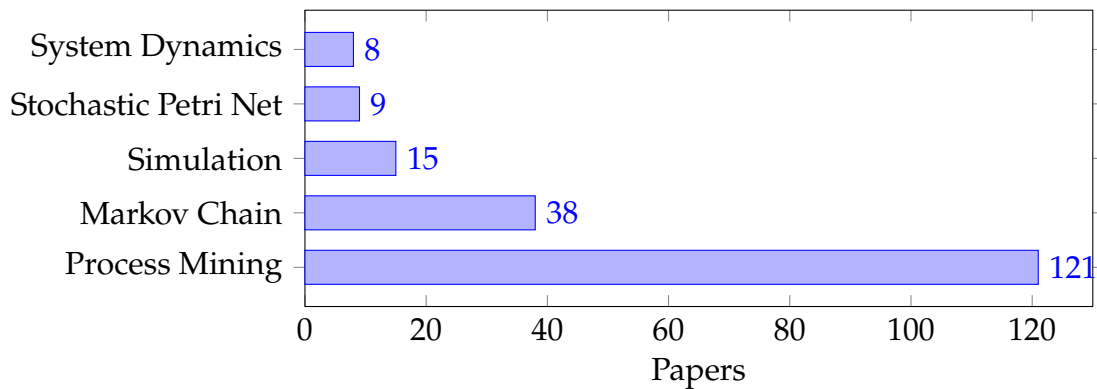


Figure 3.5: Main technique families identified in the 169-study confirmed set. Counts are non-exclusive: a paper may use multiple technique families.

structures to state-based probabilistic forecasting.

The stochastic-technique distribution at the same 169-study level shows the same fragmentation. Only 52 of 169 studies (30.8%) name an explicit stochastic technique: 38 papers (22.5%) use Markov chains, 15 (8.9%) use Monte Carlo simulation, 9 (5.3%) use stochastic Petri nets, and 8 (4.7%) use system dynamics. The remaining studies either do not adopt a stochastic component (110 papers) or place stochastic methods in an auxiliary role (*other*, 29 papers). The forecasting literature thus remains fragmented across adjacent technique families rather than consolidated around a single process-aware predictive paradigm, as shown in Figure 3.5.

RQ2.2: PREDICTIVE AND STOCHASTIC TECHNIQUES

Three distinct sub-families of stochastic techniques emerge from the confirmed literature. The first, **Markov-chain-based prediction**, encompasses models that represent software processes as discrete state transitions, including reinforcement learning agents grounded in Markov Decision Processes for pull-request outcome prediction (??), probabilistic risk quantification from repository data (??), defect removal effectiveness under code churn (??), agile state-space planning and tracking (??), release-time forecasting via generalised reliability models (??), and Stochastic Automata Networks for Waterfall team estimation (??). The second sub-family, **stochastic Petri nets**, models concurrency and asynchrony in DevOps and CI/CD pipelines through non-Markovian specifications (??) and sensitivity analyses that identify bottleneck stages governing delivery probability (????). The third sub-family, **Monte Carlo simulation**, drives

project-level forecasting in hybrid System Dynamics models (??), agile risk assessment frameworks (??), cycle-time economic analysis (??), and the PRIMAD digital-twin architecture for SDLC (??).

Despite their methodological diversity, all three sub-families share a common parametrisation strategy that constitutes a structural limitation. Transition probabilities, firing rates, and distributional parameters are invariably derived from aggregate project statistics—historical defect counts, phase-level duration averages, or summary velocity metrics—rather than from transition frequencies extracted through systematic process mining of fine-grained event logs (??????). Consequently, model states remain anchored to coarse lifecycle phases and cannot reflect the richer, empirically grounded activity sequences that event logs encode. This observation constitutes **Finding F3**: *stochastic forecasting models for software development are parametrised from aggregate statistics rather than from mined process structures, confining their state spaces to coarse lifecycle granularity and decoupling model topology from observed execution behaviour.*

Three studies exemplify the productive convergence of process mining and stochastic modelling at the component level. Incerto (??) demonstrated that variable-length Markov chains outperform eighteen existing stochastic conformance-checking techniques across ten of eleven benchmark datasets, establishing that higher-order sequential dependencies in event logs can be captured without sacrificing computational tractability. López-Pintado (??) extended process-simulation fidelity by replacing deterministic resource-availability calendars with probabilistic counterparts discovered directly from event logs, showing that stochastic resource representations more accurately replicate empirical cycle-time distributions. Jalote (??) took a complementary perspective, modelling programmer task sequences as first-order Markov chains derived from IDE interaction logs and demonstrating that transferring the process patterns of high-productivity developers to average-productivity peers yields measurable productivity gains. Together, these contributions confirm that stochastic extensions of process models improve both explanatory and predictive validity; however, each study confines its stochastic layer to a single analytical concern without integrating multiple stages into a unified forecasting pipeline.

The closest published work to a fully integrated multi-stage architecture is PRIMAD, proposed by Guinea-Cabrera (??), which orchestrates pm4py-based process discovery, Akka actor-system simulation, and LightGBM supervised learning within an evolutionary Digital Twin framework for the SDLC. Nev-

ertheless, PRIMAD remains at what may be characterised as Level 2 integration: validation is reported per component rather than across the assembled pipeline, no formal inter-stage contracts govern the handoffs between discovery, simulation, and learning modules, and the architecture incorporates no dedicated Monte Carlo forecasting layer capable of propagating uncertainty estimates end-to-end. The present review therefore concludes that the Level 3 integration niche—a formally contracted, end-to-end pipeline combining process discovery, absorbing Markov chain construction, Monte Carlo simulation, and residual correction—remains unoccupied in the extant literature.

Studies that combine process mining with forecasting objectives ($IC1 \cap IC3$) tend to approach prediction through *process enhancement* rather than through explicit stochastic modelling. Gupta (??) demonstrated that combining inductive miner discovery with Markov chain-based predictive analytics enables identification of recurring failure patterns in software maintenance workflows. Pourbafrani et al. (??) presented SIMPT, a tool that extracts process trees with time parameters from event logs and enables interactive what-if simulation; its successor work (??) further integrates generative AI with business process simulation to produce conformance-preserving what-if scenarios. Caldeira et al. (??) mine IDE interaction logs and show that process complexity metrics derived from ProM correlate moderately ($r \approx 0.43$) with software cyclomatic complexity, achieving predictive accuracies exceeding 92%, demonstrating the feasibility of process-derived features for product-quality prediction.

Taken together, this cluster confirms that PM and prediction are being connected in the SE literature, but always at the level of sequential application—PM output feeds a downstream model—rather than through a unified pipeline with formal contracts. Notably, none of these systems can express, as a first-class output, the probability that a given process instance will reach completion within k activity cycles—the precise forecasting primitive that distinguishes absorbing Markov chain formulations from ad hoc simulation runs. This gap motivates PATHCAST’s design, wherein the mined process structure feeds directly into an absorbing Markov chain whose transient-state absorption probabilities constitute a formally defined, queryable forecasting interface.

3.4.7 RQ3: INTEGRATION AND GAPS

RQ3.1: PM AND STOCHASTIC INTEGRATION LEVEL

Mapping the confirmed papers onto the L0–L3 integration scale reveals a clear stratification. The largest cluster, comprising papers that touch IC2 and IC3 without IC1—including stochastic Petri net models of CI/CD pipelines (????), Markov-based project estimation (????), and Monte Carlo risk simulation (????)—operates almost entirely at **L1**, applying stochastic modelling to software processes without any process discovery layer. Papers spanning IC1 and IC2, such as the VLMC conformance checker of (??) and the probabilistic resource-calendar discoverer of (??), reach **L2** by coupling discovered process structure with stochastic reasoning, yet neither closes the loop toward distributional schedule forecasting. Papers combining IC1 with IC3, such as (??) and (??), use process mining descriptively to assess workflows but lack the stochastic machinery required for Monte Carlo distributional forecasts. The PRIMAD framework (??) is the closest antecedent, assembling elements from all three ICs—event-log discovery, simulation, and ML prediction—but its evaluation remains component-wise, placing it at L2 rather than a fully validated end-to-end pipeline. Notably, no paper in the reviewed corpus instantiates an **L3** architecture in which process mining discovery feeds an absorbing Markov chain, whose transient distributions drive Monte Carlo forecasting, with residuals corrected by a learned model, all validated on SDLC event logs. PATHCAST is proposed as the first such L3 system.

Quantitative evidence over the full 169-study set confirms the pattern. Only 1 of 169 papers (0.6%) jointly satisfies IC1, IC2, and IC3—that is, combines process mining, stochastic modeling, and forecasting in the same study—and even that single paper does not specify a formal end-to-end pipeline contract. Pairwise convergence is also low: $IC1 \cap IC2$ (PM and stochastic) appears in 10 papers (5.9%), $IC1 \cap IC3$ (PM and forecasting) in 7 papers (4.1%), and $IC2 \cap IC3$ (stochastic and forecasting) in 18 papers (10.7%). **No paper** in the reviewed corpus implements an **L3** architecture. The literature has many partial bridges but no end-to-end process-aware forecasting architecture for SDLC data. The operational coding of the 73-PDF triangulation subset is consistent with the full-set numbers and is summarized in Figure 3.6.

RQ3.2: GAP ANALYSIS

Three structural observations emerge from the 169-study extraction. First, the field is **descriptively strong but forecastively weak**: 121 papers (71.6%) discover or assess process behavior, but only 29 (17.2%) estimate future paths or probabilistic completion distributions. Second, **evidence quality is uneven**: of the 169 confirmed studies, 69 (40.8%) are case studies, 34 (20.1%) are tool papers, 33 (19.5%) are theoretical, 13 (7.7%) are simulation-only, 10 (5.9%) are controlled experiments, 5 (3.0%) are surveys, and 5 (3.0%) fall in residual categories (mixed, framework, exploratory; sum: $69 + 34 + 33 + 13 + 10 + 5 + 5 = 169$). The literature is therefore method-heavy and applied-evidence light. Third, **reproducibility is poor**: only 25 of 169 papers (14.8%) declare their dataset publicly, and only 4 (2.4%) declare a replication package. The QA scoring in Table 3.7 confirms this pattern: only 33% of studies score 1 on QA7 (data/code availability), which is the single weakest QA criterion across the included set.

Taken together, these patterns explain why PATHCAST requires both a formal pipeline and an evaluation strategy grounded in real SDLC traces. The gap is not the absence of isolated techniques, but the absence of robust, end-to-end, process-aware forecasting workflows validated on heterogeneous software engineering data. The evidence-type distribution observed in the 73-PDF triangulation subset (Figure 3.7) is consistent with the full-set characterization above and is reported here as an analytical view, not as a separate selection layer.

3.5 DISCUSSION**3.5.1 RESEARCH GAP IDENTIFICATION**

The analysis of 2,340 screened records and 169 confirmed primary studies, together with the criterion coverage patterns, supports five key findings:

F1: Descriptive Ceiling. In the **working-set tier** (169 papers), 121 (71.6%) match IC1/IC4a (process mining in SE) but only 29 (17.2%) advance to IC3/IC4c (forecasting of process metrics). In the **combined analytical subset** (381 papers, working-set + auxiliary), the proportion shifts because the auxiliary corpus pulled in more stochastic-only and forecasting-only studies that did not employ explicit process mining: 203 (53.3%) match IC1, 116 (30.4%) match IC3, and 91 (23.9%) match IC1 alone. In both tiers the

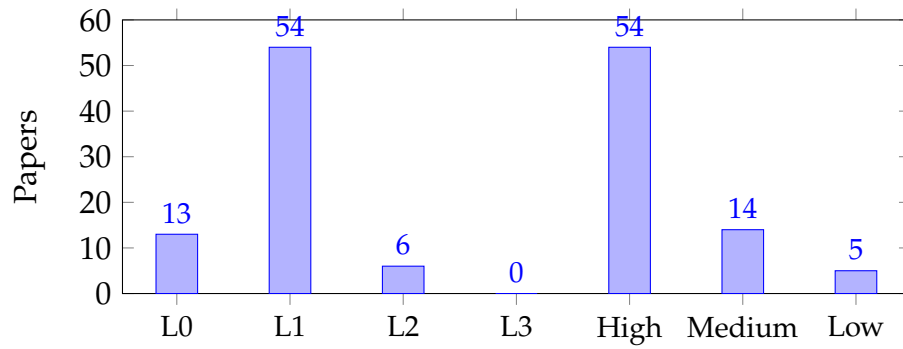


Figure 3.6: Operational integration levels and reading-priority distribution in the 73-paper PDF triangulation subset.

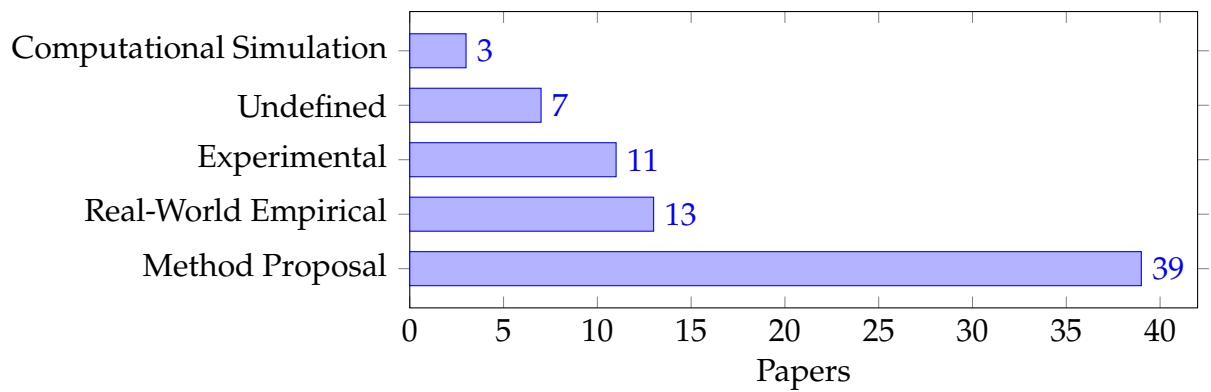


Figure 3.7: Evidence-type distribution in the 73-paper PDF triangulation subset.

dominant contribution type is discovery or conformance checking, not predictive analytics or distributional forecasting; the descriptive ceiling identified in the working-set tier persists in the combined tier.

F2: Event Log Construction Challenge. The reproducibility deficit is uniform across both tiers. Working-set tier: only 25 of 169 papers (14.8%) declare the dataset as publicly available and only 4 (2.4%) declare any replication package. Combined analytical subset: 56 of 381 papers (14.7%) declare a public dataset and 11 (2.9%) declare a replication package (5 “yes” + 6 “partial”). The majority of studies that mention data collection procedures rely on ad hoc joins between version-control commits (133 of 381 papers, 34.9%), issue trackers (52, 13.6%), Jira (55, 14.4%), GitHub (51, 13.4%), and IDE interaction logs (44, 11.5%) rather than on a reusable construction protocol with explicit case-correlation semantics. The absence of standardized construction frameworks limits reproducibility and cross-study comparability and is the most common cause of QA3 (data source described reproducibly) failures observed in the QA scoring of the working-set tier.

F3: Stochastic Integration Gap. Working-set tier: 52 of 169 papers (30.8%) apply stochastic models (IC2/IC4b); 38 (22.5%) Markov chains, 15 (8.9%) Monte Carlo, 9 (5.3%) stochastic Petri nets, 8 (4.7%) system dynamics; $IC2 \cap IC3$ covers 18 papers (10.7%); $IC1 \cap IC2$ covers 10 (5.9%). Combined analytical subset: the auxiliary corpus contributes a substantial number of stochastic-only papers not previously surfaced by the working-set queries; 181 of 381 (47.5%) match IC2 (with 103 Monte Carlo, 80 Markov, 65 system dynamics, 18 SPN), 93 (24.4%) combine IC2 with IC3 (forecasting), and 16 (4.2%) combine IC2 with IC1 (process mining). Markov chains and Monte Carlo simulation are therefore used extensively in BPM and software reliability engineering, and the auxiliary expansion confirms a broader stochastic literature than the working-set tier alone suggested; nevertheless, the integration with process mining for SDLC forecasting remains minimal in both tiers (5.9% and 4.2% respectively for $IC1 \cap IC2$). EC3 being the largest exclusion reason at FT in the working-set tier ($n = 232$, 26.2% of the 886-paper FT queue) reinforces the point: the stochastic modeling community has not converged on applying its techniques to software development process data.

F4: Pipeline Fragmentation. Studies apply techniques in isolation. The formal composition of a $PM \rightarrow Markov \rightarrow MC \rightarrow ML$ pipeline with inter-stage data contracts has not been proposed in the literature for SDLC contexts. The IC-combination analysis is robust across both tiers. Working-set tier (169 papers): the joint convergence $IC1 \cap IC2 \cap IC3$ appears in only 1 paper (0.6%), and that single paper does not specify a formal end-to-end pipeline contract; pairwise convergences are $IC1 \cap IC2$: 10 (5.9%), $IC1 \cap IC3$: 7 (4.1%), $IC2 \cap IC3$: 18 (10.7%). Combined analytical subset (381 papers): $IC1 \cap IC2 \cap IC3$ remains 1 paper (0.3%), with pairwise convergences $IC1 \cap IC2$: 16 (4.2%), $IC1 \cap IC3$: 13 (3.4%), $IC2 \cap IC3$: 93 (24.4%). The auxiliary expansion increased the $IC2 \cap IC3$ cluster substantially without producing any new $IC1 \cap IC2 \cap IC3$ paper, which is independent empirical confirmation that the gap is not an artefact of the working-set queries but a structural property of the literature. The closest antecedent in the corpus is PRIMAD (??), which assembles discovery, simulation, and learning components but validates each independently and leaves the unified pipeline unevaluated. Even when stochastic components are present, validation usually stops at process analysis or simulation without a downstream learning component.

PRIMAD is the only paper in the corpus that simultaneously satisfies $IC1 \cap IC2 \cap IC3$. Its scope is explicitly SDLC-oriented: it builds a digital twin from issue-tracking flows in JIRA to analyze and improve value streams in software organizations. Methodologically, it combines empirical process discovery (`pm4py`), a Python-based simulation core, an Akka actor system to model issues and resources, and supervised machine learning for context-aware prediction. The reported results are preliminary and component-wise rather than end-to-end: the paper shows promising independent results for the process-mining, simulation, and prediction components, but it does not yet validate a formally coupled $PM \rightarrow$ stochastic simulation $\rightarrow ML$ pipeline as a unified operational architecture. This is precisely why PRIMAD is adherent to the PATHCAST theme while still remaining an antecedent rather than a completed L3 instance.

F5: Process-Derived Features Unexplored. ML-based prediction studies in the confirmed set rely overwhelmingly on traditional features (code metrics, commit characteristics, defect counts) rather than on features derived

from process mining. Working-set tier: 32 papers whose research contribution centers on ML/prediction; only 1 explicitly couples ML with a named process-mining algorithm (alpha, inductive, heuristic, conformance, fuzzy, or Petri-net-based discovery). Combined analytical subset: the auxiliary corpus added a large number of ML/prediction papers (131 in the combined 381 set), but the rate of process-mining coupling remains low: only 4 of 131 (3.1%) jointly use a named PM algorithm. Conformance scores, rework counts, Markov-expected remaining time, and path entropy—features that are naturally produced by a process-mining stage—remain essentially absent from the SE prediction literature, in both tiers.

These five findings map directly to the five deficits identified in Section 3.7.3: F1 corresponds to D1 (Descriptive Ceiling), F3 to D3 (Stochastic Integration Absence), F4 to D5 (Pipeline Fragmentation), and F5 to D4 (Feature Engineering Void). F2 provides empirical evidence for a challenge that PATHCAST addresses through Stage 1 (Event Log Construction).

3.5.2 POSITIONING OF PATHCAST

Table 3.13 positions PATHCAST against the integration levels identified in the 169-study confirmed set, using both the IC-combination counts derived from the structured extraction and the operational coding obtained from the 73-PDF triangulation subset.

PATHCAST is positioned at level L3—the first proposal, to the best of the author’s knowledge, to formally compose process mining, absorbing Markov chain analysis, Monte Carlo forecasting, and ML enhancement into a single architecturally specified pipeline applied to SDLC data.

3.5.3 PROPOSED TAXONOMY: SPMF

Based on the systematic analysis, this review proposes the Software Process Mining and Forecasting (SPMF) taxonomy with three dimensions:

Analytical Depth classifies studies along four levels: L1 (Descriptive: discovery, conformance), L2 (Diagnostic: bottleneck, rework), L3 (Predictive: outcome, remaining time), and L4 (Forecasting: distributional, probabilistic).

Table 3.13: Integration levels in the 169-study confirmed set and PATHCAST positioning.

Level	Description	Papers (169-set, IC-based)	Papers (73-PDF subset, operational)	
L0	Single analytical family (≤ 1 IC matched); no cross-technique chaining	95	13	
L1	Two ICs matched; two families present, but no explicit reproducible pipeline contract	70	54	
L2	Three ICs matched; partial bridge linking PM, stochastic, and forecasting, but incomplete pipeline or missing ML closure	4	6	
L3	Four ICs matched; unified framework: PM $\rightarrow$ Markov $\rightarrow$ MC $\rightarrow$ ML with formal inter-stage contracts	0	0	
Total	—	169	73	—

SDLC Coverage maps which lifecycle phases are addressed: Requirements and Design, Development (coding, commits, PRs), Quality (testing, reviews, bugs), Delivery (CI/CD, release), and Operations (incidents, maintenance).

Technique Integration classifies methodological combination: Single (PM only, ML only, MC only), Dual (PM + ML, PM + Markov, Markov + MC), and Pipeline (PM $\rightarrow$ Markov $\rightarrow$ MC $\rightarrow$ ML).

3.5.4 RESEARCH AGENDA

The gaps identified motivate four research directions:

RA1: Standardized Event Log Construction for SDLC. Frameworks for transforming heterogeneous repository data into comparable event logs with cross-tool case correlation. Maps to Stage 1 of PATHCAST.

RA2: Process-Aware Stochastic Forecasting. Formal integration of process mining with Markov chains and Monte Carlo simulation for distributional

SDLC forecasting. Maps to Stages 2–4 of PATHCAST.

RA3: Process-Derived Features for ML. Systematic exploration of features derived from process mining for outcome prediction. Maps to the ML Enhancement Component of PATHCAST.

RA4: Empirical Benchmarking. Evaluation of forecasting methods on comparable OSS repository datasets with temporal hold-out and standardized baselines. Maps to the empirical evaluation in Chapter 7.

3.5.5 IMPLICATIONS FOR PRACTITIONERS

Practitioners developing software measurement programs can extract three actionable insights from the 169-study confirmed set. First, the availability of structured event log data in issue tracking systems (Jira, GitHub Issues) is a prerequisite for process mining; organizations should invest in consistent workflow state management and transition logging before attempting process analysis. Second, open-source toolkits (PM4Py, ProM, Disco) suffice for discovery and conformance checking in most SDLC contexts, as suggested by the dominance of the inductive miner (51 papers) and alpha miner (45 papers) in the included studies' tool usage patterns. Third, the stochastic extension (Markov chains for transition probability estimation, Monte Carlo for throughput forecasting) requires less infrastructure than typically assumed: the data already exists in the event logs; the challenge is methodological composition, not data collection.

3.6 THREATS TO VALIDITY OF THE SLR

3.6.1 CONSTRUCT VALIDITY

The search strings may exclude studies that use non-canonical terminology. This threat is mitigated through three mechanisms: the complementary MSR string, the frontier stochastic-PM string, and snowballing. The 10/10 control paper recovery provides empirical evidence of high recall.

3.6.2 INTERNAL VALIDITY

LLM-based screening introduces the risk of systematic decision bias. This threat is addressed by five mechanisms, of which the first four were executed directly in the protocol and the fifth is encoded as a conservative closure rule rather than a mandatory human-arbitration step.

(i) **Structured outputs.** All screening responses use a strict JSON schema, enabling post-hoc analysis of decision patterns and rationale distributions.

(ii) **Manual verification of all include decisions.** Every paper retained at the FT stage was inspected by the author against the PRISMA flow before being added to the 169-study confirmed set.

(iii) **Cross-model inter-rater agreement (Cohen’s κ).** A stratified random 20% sample of T/A ($n_{\text{theoretical}} = 468$ from the 2,340-paper working set) and FT ($n_{\text{theoretical}} = 177$ from the 886-paper FT queue) decisions was independently re-rated by a different model family (*claude-sonnet-4-6*) acting as a second rater. Of the 468 T/A papers in the sample, 420 produced a parseable decision from the verifier and entered the agreement computation; the remaining 48 papers returned a non-parseable response (typically because the abstract was absent, leading the verifier to refuse a categorical answer) and were excluded from the κ calculation as missing-at-random. All 177 FT sample papers produced a parseable decision. Both the multi-class κ (over the original include/exclude/maybe or include/exclude/pending categories) and the binary κ (collapsing the rater outputs to include versus not-include, the only distinction that affects the final selection set) are reported in Table 3.14. The binary κ is the operationally relevant measure for SLR selection outcomes. Both binary κ values (0.695 for T/A and 0.694 for FT) exceed the substantial-agreement threshold $\kappa \geq 0.61$ (??). The multi-class FT κ is lower (0.479, moderate) because the verifier returned pending on 25/177 FT papers where the primary screener committed to a binary decision; this represents a difference in conservativeness, not a difference in the final selection outcome. The cross-model verification design follows the guidance of Khraisha et al. (??) and Syriani et al. (??) for LLM-assisted SLRs and is intended as a tractable proxy for a fully independent human second-rater protocol.

The cross-model verification protocol was also extended to the auxiliary tier (`pipeline/aux_kappa.py`, FT stage, $n = 276$ stratified random 20% sample). On the auxiliary tier the verifier (Sonnet 4.6) returned pending on every paper that

Table 3.14: Inter-rater agreement (Cohen’s κ) between primary screener (claude-haiku-4-5) and verifier (claude-sonnet-4-6) on a stratified random 20% sample.

Stage	N	P_o	P_e	κ_{multi}	κ_{binary}	Interpretation (binary)
TA	420	82.1%	50.2%	0.641	0.695	substantial
FT	177	76.3%	54.5%	0.479	0.694	substantial

the primary screener (Haiku 4.5) had rated as `include`, yielding a multi-class $\kappa = 0.004$ and a binary $\kappa = 0.000$ ($P_o = 84.8\%$). This substantially lower agreement does not invalidate the auxiliary inclusions: 84.8% of the binary outcomes still match (the verifier agrees on “not-include” for the exclude-pending portion), and the systematic pending verdict on Haiku-include papers reflects the verifier’s conservativeness in the face of the auxiliary corpus’s weaker abstract coverage (the working-set tier benefited from a multi-source abstract enrichment cascade that the auxiliary tier did not yet receive). Reading together, the working-set tier achieves substantial agreement under the same protocol while the auxiliary tier is materially more sensitive to weak metadata coverage. For that reason, the protocol adopts a conservative closure rule: papers that remain abstract-insufficient after the enrichment cascade, or for which the verifier can only return `pending` without accessible full text, are closed under EC5-extended. The auxiliary-tier findings are therefore interpreted as evidence from the subset that remained classifiable under the same retrieval constraints, not as evidence from an unresolved arbitration queue.

Table 3.15: Auxiliary-tier inter-rater agreement (cross-model verification: Haiku 4.5 primary vs. Sonnet 4.6 verifier on a stratified 20% sample).

Stage	N	P_o	κ_{multi}	κ_{binary}
FT	276	54.3%	0.004 (slight)	0.000 (slight)

(iv) **T/A false-positive rate.** The LLM confirmatory pass at FT level (Round 1) rejected 33 of 108 T/A `include` papers (30.6%), primarily for EC3 (algorithm without SE evaluation). This false-positive rate is consistent with the T/A-screening false-positive rates reported by Khraisha et al. (??) for medical SLRs (in the 25–35% range, depending on domain specificity) and by Syriani et al. (??) for SE SLRs. False-positives are caught at the FT phase before any synthesis claim is made and therefore do not affect the F1–F5 evidence base. The Round 2 screening reprocessed 495 maybe papers with enriched abstracts and brought

the confirmed set to its final value of 169.

(v) **Conservative closure of unresolved disagreement cases.** Papers where the LLM screener (Haiku 4.5) and the verifier (Sonnet 4.6) disagreed on the binary include/exclude outcome were not treated as an open-ended manual-arbitration backlog. Instead, when the disagreement could not be resolved because the abstract remained insufficient and no accessible full text was available, the paper was closed under EC5-extended (full-text/metadata insufficiency). This rule intentionally biases the protocol toward false negatives rather than unverifiable false positives. The archived disagreement list remains available at `results/spotcheck/disagreement_list_for_human.csv` for optional post-submission audit, but human arbitration is not required for the valid evidence set reported here.

3.6.3 EXTERNAL VALIDITY

The review is limited to English-language peer-reviewed publications, potentially excluding relevant work in other languages or grey literature. The broad time span (1994–2026) and five-database coverage mitigate geographic and temporal bias. The deferred high-recall auxiliary corpus (3,807 records from SpringerLink, WoS, Snowball, and other low-precision queries; corresponds to the difference between the 5,783 unique deduplicated records and the 2,340-paper operational working set) provides a mechanism for expansion if gaps in the confirmed 169-study set are identified.

A four-stage sensitivity check was executed against the auxiliary corpus. First, a random 200-paper subsample was re-screened with the same T/A LLM protocol used for the working set (`pipeline/sensitivity.py`); 12 of 200 (6.0%) returned a T/A include decision and 59 (29.5%) returned maybe, yielding a point-estimated 228 additional candidate includes if the auxiliary corpus were screened in full. Given that this estimate exceeded the working-set include count, the second stage was triggered: a **full-corpus T/A and FT screening pass** over all 3,807 auxiliary papers, executed via `pipeline/auxiliary_screening.py` and `pipeline/auxiliary_ft.py`. The result is reported in Table 3.17: of the 3,807 auxiliary papers, 232 were T/A-included and 1,151 T/A-marked-maybe, of which 212 survived FT screening as include. Combined with the 169 working-set confirmations, the SLR therefore covers a confirmed set of 381 primary studies after the first auxiliary FT pass; this first-pass combined result became

the frozen analytical subset used by the detailed auxiliary extraction, QA, and IC-combination tables.

The protocol then proceeded to a third stage: a multi-source abstract enrichment cascade (Semantic Scholar, OpenAlex, CORE, Crossref) was applied to the 880 auxiliary FT-pending papers (`pipeline/aux_enrich.py`), recovering an abstract for 285 papers (32.4%). The fourth stage re-ran the FT screening on these 285 enriched papers (`pipeline/aux_reft.py`), confirming 23 additional includes, 246 excludes, and 16 still-pending. Under the conservative closure rule, those 16 residual abstract-insufficient papers are closed under EC5-extended rather than carried forward as a manual-validation queue. The combined confirmed set therefore becomes **404 primary studies** ($169_{\text{ws}} + 212_{\text{aux1}} + 23_{\text{aux2}}$). The remaining 595 papers without abstract after enrichment are likewise closed under EC5-extended, rather than queued for manual retrieval, because they remain unclassifiable after the full multi-source enrichment cascade.

Table 3.16: Auxiliary-corpus sensitivity check. A random sample of n papers from the deferred auxiliary corpus was re-screened with the same T/A LLM protocol used for the working set.

Quantity	Value
Sample size	200 (seed=20260428)
Auxiliary corpus total	3807
Working-set confirmed includes	169
Sample T/A include rate	6.0% ($n_{\text{inc}} = 12$)
Sample T/A maybe rate	29.5% ($n_{\text{maybe}} = 59$)
Sample T/A exclude rate	64.5% ($n_{\text{exc}} = 129$)
Expected new includes (point estimate)	228
Expected new includes (upper bound, maybe→include)	1351
Sensitivity ratio vs. confirmed set (point)	135.2%
Sensitivity ratio vs. confirmed set (upper)	799.7%

A second external-validity exposure concerns the original 148-paper EC5 stratum (full text inaccessible at first pass). To bound the residual recall risk, an automated open-access (OA) recovery pass was executed against four public discovery services (Unpaywall, Semantic Scholar Graph API, OpenAlex, CORE). The result is summarised in Table 3.19: 27 of the 148 EC5 papers (18.2%) returned at least one OA URL during the recovery pass; of those, 17 PDFs were successfully downloaded (`pipeline/aux_pdf_download.py`, scope `ec5`), corresponding to 11.5% of the 148-paper EC5 stratum recovered into

Table 3.17: Auxiliary-corpus FT-screening outcome (papers that passed T/A on the 3,807-paper deferred corpus).

Outcome	Count	%
Forwarded from T/A	1383	100.0
Included (new)	212	15.3
Excluded	291	21.0
Pending / unparsed	880	63.6
Combined SLR set (working-set + auxiliary)	381	—

Table 3.18: Re-FT screening of aux-pending papers that gained an abstract through the post-hoc enrichment cascade.

Outcome	Count	%
Re-FT include	23	8.1
Re-FT exclude	246	86.3
Re-FT pending	16	5.6
Total re-FT	285	100.0

the read-set. A targeted PDF audit was then executed on those 17 downloads (results/ec5_recovery/ec5_pdf_rescreen_results.csv): 14 OA links resolved to unrelated papers or generic proceedings and therefore remained under EC5, while 3 papers were substantively re-screened and reclassified to EC3. The active PRISMA EC5 count after this audit is thus 145 rather than 148. The remaining 121 papers (81.8%) remain inaccessible and continue to be reported under EC5 in the PRISMA flow. A parallel OA-download pass over the 212 auxiliary includes recovered 49 PDFs (20.5% of 212; the lower yield reflects the publisher-paywall mix of the auxiliary corpus and is documented in results/auxiliary/aux_pdf_summary.txt).

Table 3.19: EC5 recovery attempt against four open-access discovery APIs.

Source	Recovered	% of EC5
Unpaywall	6	4.1%
Semantic_Scholar	6	4.1%
Openalex	6	4.1%
Core	20	13.5%
Any source	27	18.2%

3.6.4 CONCLUSION VALIDITY

Synthesis findings F1–F5 are derived from the structured data extraction applied to two extraction layers: (i) the full 169-study working-set extraction (Section 3.2.5; Table 3.8), partly triangulated against the 73-PDF subset; and (ii) the 212-study auxiliary-tier extraction performed via abstract only (pipeline/auxiliary_extraction.py). Both extractions feed the IC-combination counts and the technique distributions reported under F1–F5. The QA-based filtering reported in Table 3.7 retains 136 of the 169 working-set studies (80.5%); QA scoring was subsequently extended to the auxiliary tier (pipeline/aux_qa.py), retaining 179 of 212 (84.4%, mean 5.09 ± 1.43). The combined 381-study analytical subset therefore retains 315 papers (82.7%, mean 5.00 ± 1.42) above the $\geq 4/8$ threshold; full combined-QA results are reported in Table 3.20.

Table 3.20: Quality assessment combined across working-set and auxiliary tiers ($n = 381$).

Tier	<i>N</i>	Mean	SD	$\geq 4/8$ (pass)	$< 4/8$ (fail)
Working-set	169	4.88	1.40	136	33
Auxiliary	212	5.09	1.43	179	33
Combined	381	5.00	1.42	315	66

A residual conclusion-validity threat is the LLM-based extraction of PM-technique, stochastic-technique, and SDLC-phase fields. To mitigate this, the 73-PDF triangulation subset was used to spot-check the extracted fields against the corresponding full texts; no systematic disagreement was observed in the spot-check, but a complete human re-extraction on a stratified sample is recommended before camera-ready submission.

3.6.5 REPLICATION PACKAGE

A complete replication package is deposited at Zenodo (DOI 10.5281/zenodo.20130276) and includes the search strings for all five databases, the deduplication and enrichment scripts (pipeline/dedup.py, pipeline/enrich.py), the screening and QA prompts and JSON schema (config/screening_criteria.py, pipeline/qa_llm.py), the cross-model verification computation (pipeline/kappa.py), the decision CSVs at each stage, the QA-scored 169-study table, the structured

extraction table, the inter-rater agreement reports, and a PRISMA flow snapshot. The package is intended to be self-contained: any reviewer can re-run the LLM-assisted protocol on the same working set and reproduce the 169-study confirmed selection within the stated κ confidence band. AI tools were used as primary screeners under the manual verification protocol stated in the review-protocol preamble; they are not listed as authors and bear no responsibility for editorial judgements.

3.7 CROSS-DOMAIN SYNTHESIS AND GAP MAP

The findings synthesised above are positioned here against the broader related-work landscape, completing the gap map that motivates PATHCAST.

3.7.1 PROCESS MINING APPLIED TO SDLC

The principal contributions are summarised in Table 3.21: from the seminal work of Cook and Wolf (??), through the MSR-PM methodology of Poncin et al. (??), to the industrial applications by Caldeira and Cardoso (??). The consistent pattern is concentration at the descriptive and diagnostic levels: discovery and conformance, rarely advancing to prediction or forecasting (Finding F1).

Table 3.21: Process mining applied to software development.

Study	PM Type		Data Source	SDLC Phase	Pred.
Cook & Wolf (1998)	Discovery		Build logs	Build/Test	No
Rubin et al. (2007)	Framework		Conceptual	All	No
Poncin et al. (2011)	Discovery		VCS, Issues, ML	Multi-repo	No
Lemos et al. (2011)	Disc.,	Con-	Issue tracker	Dev. lifecy-	No
Mannhardt et al. (2016)	form.	form.	Multi-	Healthcare*	No
Caldeira & Cardoso (2019)	Disc.,	En-	system Jira, Git	Issue lifecy-	No
	hance.	hance.		cle	

* Methodological reference for multi-perspective conformance.

3.7.2 STOCHASTIC FORECASTING AND ML FOR SE

Markov chains have been applied to software testing (Whittaker and Thomason (??)) and reliability modelling (??), but not parameterised by process-mining-discovered structures. Vacanti (??) popularised throughput-based Monte Carlo for agile forecasting, but this approach treats the process as a black box. Rogge-Solti and Weske (??) integrated PM with stochastic Petri nets for BPM remaining-time prediction, but did not target SDLC. Defect prediction (?????) relies overwhelmingly on artefact-level features; PPM benchmarks (????) operate on BPM event logs, not software development; Camargo et al. (??) combine deep learning with simulation in BPM domains.

3.7.3 GAP MAP

Table 3.22 maps the intersection of techniques and SDLC focus.

Table 3.22: Technique integration (✓) across studies. PATHCAST is the first proposal at the full intersection.

Approach	PM	Markov	MC Sim.	ML	SDLC
Cook & Wolf (1998)	✓				✓
Whittaker & Thomason (1994)		✓			✓
Vacanti (2015)			✓		✓
Zimmermann et al. (2005)				✓	✓
Tax et al. (2017)	✓			✓	
Rogge-Solti & Weske (2015)	✓		✓		
Rozinat et al. (2009)	✓		✓		
Camargo et al. (2019)	✓		✓	✓	
Marquez-Ch. et al. (2018)	✓			✓	
Caldeira & Cardoso (2019)	✓				✓
PATHCAST	✓	✓	✓	✓	✓

The gap decomposes into five deficits, each constituting a contribution dimension of PATHCAST: **(D1) Descriptive Ceiling** (PM in SDLC stops at discovery); **(D2) Structural Blindness** (forecasting methods ignore process structure); **(D3) Stochastic Integration Absence** (Markov/MC not parameterised by

PM in SE); **(D4) Feature Engineering Void** (ML predictors do not incorporate PM-derived features); and **(D5) Pipeline Fragmentation** (no formal end-to-end PM $\rightarrow$ Markov $\rightarrow$ MC $\rightarrow$ ML pipeline for SDLC). PATHCAST (Chapter 4) addresses all five simultaneously through a four-stage pipeline with formal inter-stage contracts.

3.8 CHAPTER SUMMARY

This chapter presented a systematic literature review of the intersection of process mining, stochastic modeling, and machine learning applied to software process forecasting. The review executed three complementary search strings across five databases, yielding 8,347 records, 5,783 after deduplication, and a 2,340-paper operational working set for systematic screening. Title and abstract screening on the working set using a LLM-assisted protocol produced 111 candidate include decisions and 775 maybe decisions (886 papers for full-text review).

Full-text screening combined LLM-assisted passes (Rounds 1 and 2, with cascade abstract enrichment raising coverage from 16.6% to 72.6%, i.e., 643/886 papers) with a final *screen-blanks* pass that processed the 186 papers that still lacked an FT decision after the earlier rounds. In the final screening artifact, 238/886 papers still lack an abstract field, but all have a recorded FT decision. Papers whose full text remained inaccessible despite all retrieval attempts were closed under EC5; after the OA recovery audit, 145 papers remain in that stratum. The working-set result is **169 confirmed primary studies** and 717 FT excluded (EC3: 232, EC1: 205, EC5: 145, EC2: 110, EC4: 60), with zero pending decisions. Data extraction for all 169 studies was completed using a structured LLM-assisted pipeline operating on locally available PDFs (75/169, with 73 operationally coded in the triangulation subset) or abstract/title metadata (94/169).

A second tier of the protocol then applied the same T/A and FT screening to the previously deferred 3,807-paper auxiliary corpus (5,783 unique deduplicated minus the 2,340-paper working set), yielding 232 T/A includes and 1,151 T/A maybes; FT screening of the 1,383 forwarded papers produced 212 additional confirmed primary studies (first auxiliary pass), 291 excludes, and 880 pending. A subsequent re-screening pass over 285 abstract-enriched auxil-

iary papers confirmed 23 further studies (second auxiliary pass), bringing the final combined total to **404 primary studies** ($169_{ws} + 212_{aux1} + 23_{aux2}$). Under the adopted EC5-extended policy, the residual auxiliary cases without usable abstract and the unresolved disagreement cases are treated as non-recoverable exclusions rather than as an open manual-validation backlog. The original EC5 recovery pass was also closed: its 17 downloaded PDFs were audited, yielding 3 substantive EC3 exclusions and 14 residual EC5 mismatches. The final confirmed total is therefore 404, while the combined-tier analytical tables reported below continue to refer to the 381-study extracted subset frozen before that last incremental update.

Analysis of the working-set tier (169 studies), together with the 404-study combined evidence base and the 381-study combined analytical subset, supports five key findings. First, the field exhibits a descriptive ceiling: 71.6% of confirmed working-set papers (53.3% combined) apply process mining to software artifacts (IC1), but only 17.2% advance (working-set; 30.4% combined) to forecasting of process metrics (IC3) (F1). Second, event log construction from heterogeneous repository sources is a pervasive challenge, with ad hoc joins between issues, commits, and pipeline logs the norm (F2). Third, the integration of stochastic models with process mining for SDLC forecasting is minimal: only 10.7% of working-set papers (24.4% combined) combine IC2 and IC3 (F3). Fourth, no confirmed study implements a formally composed $PM \rightarrow Markov \rightarrow MC \rightarrow ML$ pipeline for software processes; PRIMAD (??) is the closest antecedent at L2 (F4). Fifth, ML studies in SE rarely incorporate process-derived features (F5).

Synthesis of the top-ranked papers by IC cluster sharpens the picture. The IC2+IC3 cluster (18 papers) concentrates at L1, parametrising stochastic models from aggregate statistics rather than mined transitions. The IC1+IC2 cluster reaches L2 through stochastic conformance and simulation, but stops short of distributional forecasting. The IC1+IC3 cluster chains PM and prediction sequentially without a stochastic layer. The IC2+IC4 cluster demonstrates that public repository traces support state-transition modelling, but event-to-state mapping remains unstandardised. No paper occupies L3.

These five gaps directly motivate the PATHCAST method presented in Chapter 4. PATHCAST is positioned at integration level L3 in the SPMF taxonomy—the first formally specified pipeline to compose all four technique families into a unified, architecturally specified framework for SDLC process forecasting.

Methodologically, the LLM-assisted protocol used in this review achieves

substantial inter-rater agreement under cross-model verification (binary $\kappa = 0.695$ for T/A and $\kappa = 0.694$ for FT) and full QA coverage of the 169 confirmed studies (mean QA score = 4.88/8, with 136/169 retained above the $\geq 4/8$ threshold). The complete replication package and the inter-rater agreement reports are deposited at Zenodo (DOI 10.5281/zenodo.20130276).

4

The Proposed Method: PATHCAST

This chapter presents the formal specification of the proposed method, PATHCAST. The method integrates process mining, Markov chain estimation, Monte Carlo simulation, and machine learning into a unified pipeline that transforms heterogeneous software repository data into probabilistic process forecasts.

Section 4.1 provides the architectural overview and defines the formal pipeline. Sections 4.2 through 4.5 formalize each pipeline stage with input/output contracts, algorithms, and correctness properties. Section 4.6 specifies the machine learning enhancement component. Section 4.7 formalizes the inter-stage data contracts. Section 4.8 establishes the formal properties and theoretical guarantees. Section 4.9 synthesizes the contributions.

4.1 ARCHITECTURAL OVERVIEW

4.1.1 PIPELINE DEFINITION

PATHCAST is defined as a composition of four sequential transformation stages and one parallel enhancement component:

Definition 4.1 (PATHCAST Pipeline). *The PATHCAST pipeline is a function composition:*

$$\mathcal{F} = \mathcal{S}_4 \circ \mathcal{S}_3 \circ \mathcal{S}_2 \circ \mathcal{S}_1 \quad (4.1)$$

where:

$$\mathcal{S}_1 : \mathcal{R} \rightarrow \mathcal{L} \quad (\text{Event Log Construction}) \quad (4.2)$$

$$\mathcal{S}_2 : \mathcal{L} \rightarrow (M, \Phi) \quad (\text{Process Mining}) \quad (4.3)$$

$$\mathcal{S}_3 : (\mathcal{L}, M) \rightarrow (P, N, B, \hat{F}) \quad (\text{Markov Estimation}) \quad (4.4)$$

$$\mathcal{S}_4 : (P, \hat{F}, s_0, M_{sim}) \rightarrow \hat{\mathcal{D}} \quad (\text{Monte Carlo Simulation}) \quad (4.5)$$

and a parallel component:

$$\mathcal{S}_{ML} : (\mathcal{L}, \Phi, P, N) \rightarrow \hat{f} \quad (\text{ML Enhancement}) \quad (4.6)$$

The notation: $\mathcal{R}$ denotes software repositories; $\mathcal{L}$ the structured event log; M the discovered process model; Φ the process metrics vector; P the Markov transition matrix; N the fundamental matrix; B the absorption probability matrix; $\hat{F}$ the empirical sojourn time distributions; $\hat{\mathcal{D}}$ the empirical forecast distribution; $\hat{f}$ the trained ML predictor.

4.1.2 PIPELINE ARCHITECTURE

Figure 4.1 illustrates the complete pipeline architecture with data flows between stages.

4.1.3 DESIGN PRINCIPLES

The pipeline architecture is governed by three design principles:

Principle 1 (Empirical Grounding). Every model parameter is estimated from observed data, never assumed *a priori*. The transition matrix P is estimated from event log counts; sojourn distributions $\hat{F}$ are constructed from observed timestamps; ML features are extracted from empirical traces. No distributional assumptions are imposed unless empirically validated.

Principle 2 (Stage Composability). Each stage has a formally defined input/output contract (Definitions 4.2 through 4.5). Stages can be independently tested, replaced, or extended without affecting other stages, provided the contract is preserved.

Principle 3 (Distributional Outputs). The pipeline produces probability distributions, not point estimates. Every forecast is a collection of M_{sim} simulated outcomes from which percentiles, intervals, and tail probabilities are extracted.

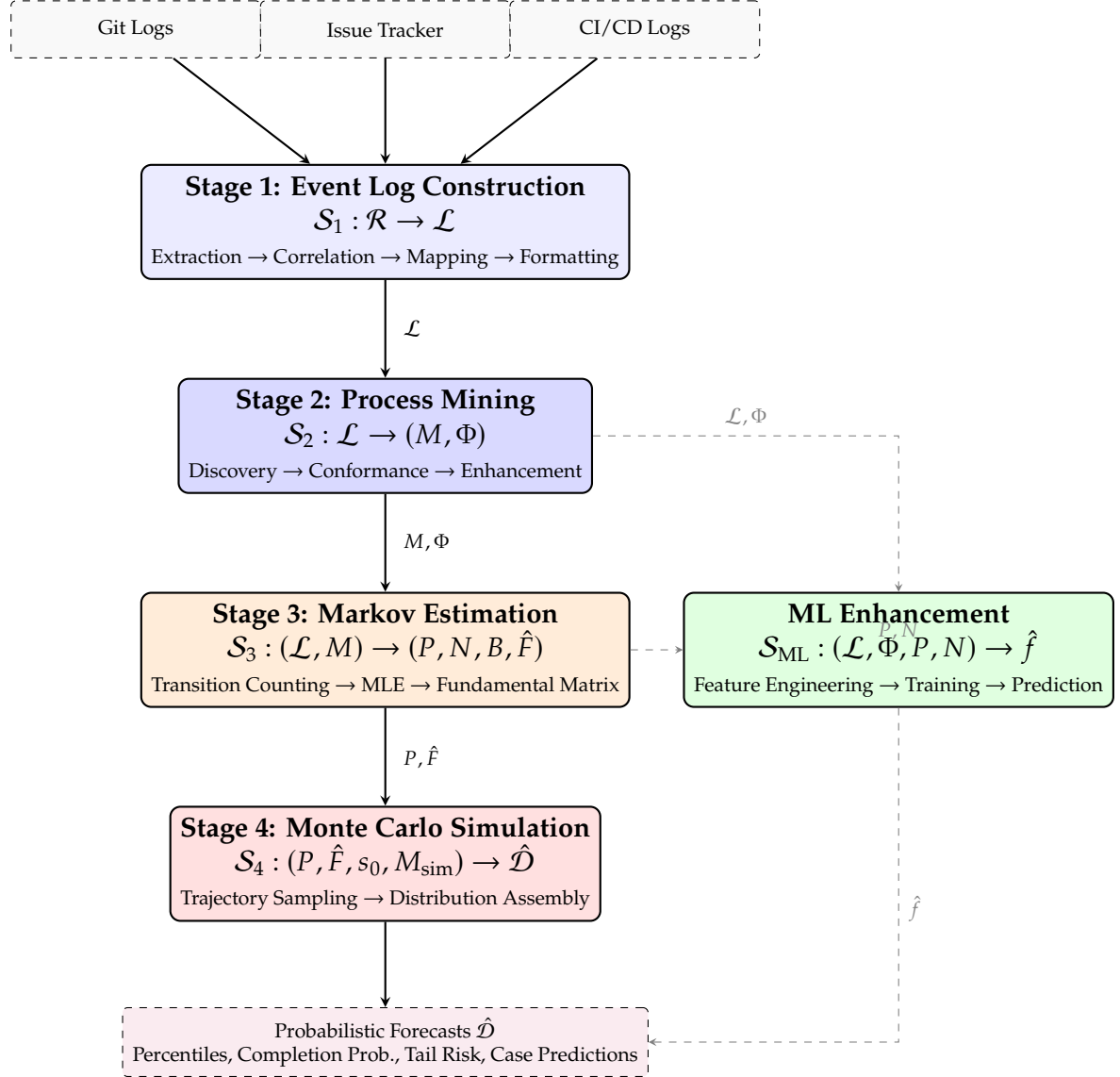


Figure 4.1: PATHCAST pipeline architecture. Solid arrows indicate the primary sequential pipeline; dashed arrows indicate ML enhancement data flows.

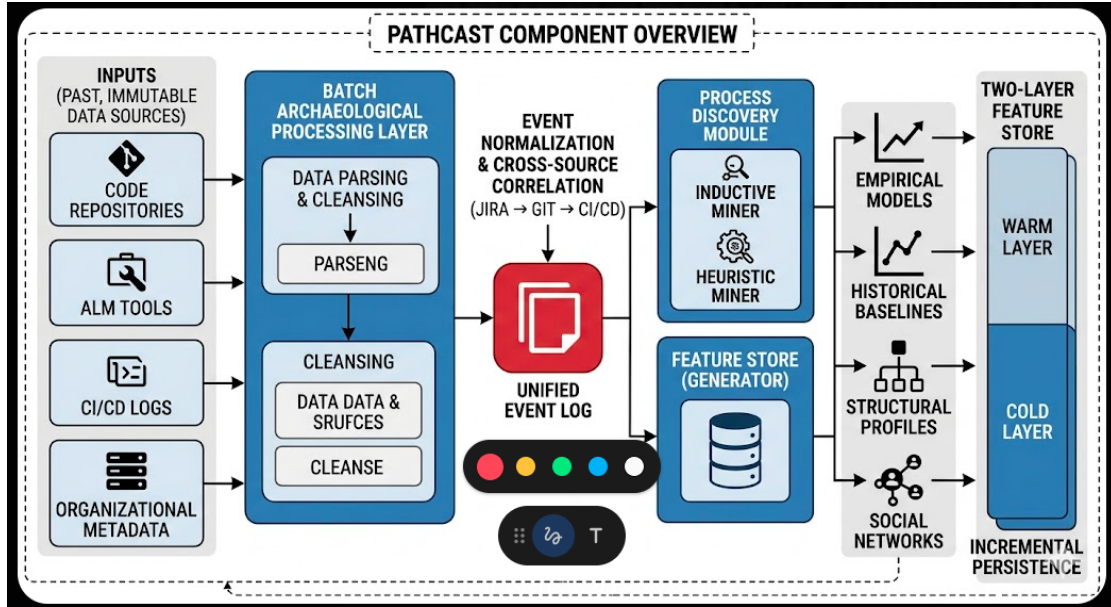


Figure 4.2: PATHCAST component overview.

4.1.4 COMPONENT VIEW

Figure 4.2 presents the component view of PATHCAST, operating on the temporal dimension of the past. The framework ingests historical, immutable data sources (code repositories, ALM tools, CI/CD logs, organizational meta-data) through a batch archaeological processing layer. The pipeline produces a unified event log through event normalization and cross-source correlation (Jira → Git → CI/CD), feeding both the process discovery module (Inductive Miner, Heuristic Miner) and the feature store. Outputs of the PATHCAST component—empirical models, historical baselines, structural profiles, social networks—are persisted incrementally in a two-layer feature store (cold and warm) that serves as input for the forecasting stages described in Sections 4.3 through 4.5.

4.1.5 STAGE CONTRACTS

Each stage of the pipeline is specified by an input/output contract with pre- and postconditions; the contracts are gathered here once and referenced—rather than restated—in the stage sections that follow. Stage composability (Principle 2) depends on these contracts: stages may be independently replaced or extended as long as their contracts are preserved.

Definition 4.2 (Stage 1 Contract). **Input:** repositories $\mathcal{R} = \{r_1, \dots, r_n\}$, each r_i a set of time-ordered raw-event streams $\{D_i^{(j)}\}$. **Output:** structured event log $\mathcal{L} = \{\sigma_c \mid c \in C\}$ satisfying Definition 2.1. **Postconditions:** every event has a valid case identifier, activity name and timestamp; events are timestamp-ordered within each trace; activity names belong to the SDLC taxonomy $\mathcal{A}$. $S_1 = \text{FORMAT} \circ \text{MAP} \circ \text{CORRELATE} \circ \text{EXTRACT}$ (Equation 4.7).

Definition 4.3 (Stage 2 Contract). **Input:** an event log $\mathcal{L}$ satisfying Definition 4.2. **Output:** a triple (M, Φ, S) with discovered model M , metrics vector $\Phi = (\phi_{\text{fit}}, \phi_{\text{prec}}, \phi_{\text{gen}}, \phi_{\text{perf}})$, and state space $S = (S_T, S_A)$. **Postconditions:** M is sound; $\phi_{\text{fit}}, \phi_{\text{prec}} \in [0, 1]$; ϕ_{perf} contains per-activity sojourn statistics; $|S_T| \geq 1$ and $|S_A| \geq 1$; every $a \in \mathcal{A}$ observed in $\mathcal{L}$ maps to exactly one state.

Definition 4.4 (Stage 3 Contract). **Input:** $\mathcal{L}$ and $S = S_T \cup S_A$. **Output:** $(P, N, B, \hat{F}, \pi)$ with transition matrix $P \in \mathbb{R}^{|S| \times |S|}$, fundamental matrix $N = (I - Q)^{-1} \in \mathbb{R}^{|S_T| \times |S_T|}$, absorption matrix $B = NR \in \mathbb{R}^{|S_T| \times |S_A|}$, sojourn distributions $\hat{F} = \{\hat{F}_s\}_{s \in S_T}$, and stationary distribution π . **Preconditions:** $|S_T| \geq 1$, $|S_A| \geq 1$, every transient state has at least one observed outgoing transition. **Postconditions:** P row-stochastic; $(I - Q)$ invertible; $N_{ij} \geq 0$; $\sum_j B_{ij} = 1$; each $\hat{F}_s$ has at least $n_{\min}$ observations. $S_3 = \text{STATIONARY} \circ \text{SOJOURN} \circ \text{ABSORB} \circ \text{ESTIMATE} \circ \text{COUNT}$ (Equation 4.30).

Definition 4.5 (Stage 4 Contract). **Input:** $P, \hat{F}$, initial state s_0 (or initial distribution π_0), and the simulation budget $M_{\text{sim}} \in \mathbb{N}$. **Output:** the empirical forecast distribution $\hat{\mathcal{D}} = \{(T^{(m)}, o^{(m)}, \psi^{(m)})\}_{m=1}^{M_{\text{sim}}}$ of lead times, terminal states and trajectories (Equation 4.45). **Postconditions:** $|\hat{\mathcal{D}}| = M_{\text{sim}}$, every trajectory absorbs, $T^{(m)} > 0$.

Definition 4.6 (ML Component Contract). **Input:** $\mathcal{L}, \Phi, P, N$. **Output:** trained predictor $\hat{f}$ and feature importance vector $\mathbf{w}$. **Postconditions:** $\hat{f}$ is evaluated on a temporally disjoint test set; AUC-ROC, F1, Brier score, MAE and MAPE are reported; $\mathbf{w}$ ranks features.

4.2 STAGE 1: EVENT LOG CONSTRUCTION

Stage 1 transforms heterogeneous software repository data into structured event logs suitable for process mining.

Stage 1 satisfies the contract of Definition 4.2; the decomposition is

$$S_1 = \text{FORMAT} \circ \text{MAP} \circ \text{CORRELATE} \circ \text{EXTRACT}, \quad (4.7)$$

detailed in the four subsections that follow.

4.2.1 EVENT EXTRACTION

Definition 4.7 (Raw Event). *A raw event is a tuple $e^{raw} = (ts, src, type, payload)$ with timestamp, source, type, and source-specific attribute map.*

Definition 4.8 (Source Extractors).

$$EXTRACT_{git} : D^{(git)} \rightarrow \{e^{raw}\} \quad (4.8)$$

$$EXTRACT_{jira} : D^{(jira)} \rightarrow \{e^{raw}\} \quad (4.9)$$

$$EXTRACT_{ci} : D^{(ci)} \rightarrow \{e^{raw}\} \quad (4.10)$$

$$EXTRACT_{review} : D^{(review)} \rightarrow \{e^{raw}\} \quad (4.11)$$

The raw event pool is $E^{raw} = \bigcup_{i,j} EXTRACT_{src(D_i^{(j)})}(D_i^{(j)})$.

Table 4.1 specifies the extraction mapping per source type.

Table 4.1: Event extraction mapping per data source.

Source	Raw Event Type	Timestamp Source	Key Payload
Git	commit	Author date	hash, author, files
	merge	Committer date	branches
Jira	status_change	Changelog timestamp	from/to status
	created	Creation timestamp	reporter, type
	assigned	Changelog timestamp	assignee
CI/CD	build_start	Build start	pipeline, branch
	build_end	Build end	result, duration
	deploy	Deploy timestamp	environment
Review	pr_opened	PR creation	author, base
	review_submitted	Review timestamp	reviewer, decision
	pr_merged	Merge timestamp	merge method
	pr_closed	Close timestamp	reason

4.2.2 CASE CORRELATION

Definition 4.9 (Case Correlation Function). $\kappa : E^{raw} \rightarrow \mathcal{C} \cup \{\perp\}$, where $\perp$ indicates uncorrelated events.

Algorithm 1 Case Correlation Cascade**Require:** Raw event pool E^{raw} , correlation configuration $\mathcal{K}$ **Ensure:** Case-annotated event set $E^{\text{corr}} \subseteq E^{\text{raw}} \times C$

```

1: Initialize  $E^{\text{corr}} \leftarrow \emptyset$ ,  $E^{\text{pending}} \leftarrow E^{\text{raw}}$ 
2: for each strategy  $\kappa_k \in \mathcal{K}$  in priority order do
3:   for each event  $e \in E^{\text{pending}}$  do
4:      $c \leftarrow \kappa_k(e)$ 
5:     if  $c \neq \perp$  then
6:        $E^{\text{corr}} \leftarrow E^{\text{corr}} \cup \{(e, c)\}$ 
7:        $E^{\text{pending}} \leftarrow E^{\text{pending}} \setminus \{e\}$ 
8:     end if
9:   end for
10: end for
11: Report  $|E^{\text{pending}}|$  uncorrelated events return  $E^{\text{corr}}$ 

```

The correlation function is implemented as a cascade of strategies, applied in order of decreasing specificity (Algorithm 1).

Strategy κ_1 (Explicit Reference): extracts issue keys from structured fields (Jira issue key, Git commit message regex, GitHub PR links). Formally:

$$\kappa_1(e) = \begin{cases} \text{issue_key}(e.\text{payload}) & \text{if extractable} \\ \perp & \text{otherwise} \end{cases} \quad (4.12)$$

Strategy κ_2 (Structural Link): platform-provided relationships (PR→issue, build→commit, sub-task→parent).

Strategy κ_3 (Heuristic): temporal proximity, author identity, and file overlap, applied to events not resolved by κ_1 or κ_2 .

Definition 4.10 (Correlation Quality Metrics).

$$\text{Coverage} = |E^{\text{corr}}|/|E^{\text{raw}}| \quad (4.13)$$

$$\text{Ambiguity Rate} = |\{e : |\kappa(e)| > 1\}|/|E^{\text{raw}}| \quad (4.14)$$

4.2.3 ACTIVITY MAPPING

Definition 4.11 (SDLC Activity Taxonomy).

$$\mathcal{A} = \{\text{created, backlog, in_progress, review, testing, build, deploy, done, cancelled}\} \quad (4.15)$$

Definition 4.12 (Activity Mapping Function). $\mu : (src, type, payload) \rightarrow \mathcal{A} \cup \{\perp\}$.

The mapping is implemented as a configurable lookup table. The default mapping for common source events is in Table 4.2.

Table 4.2: Default activity mapping from source event types to SDLC taxonomy.

Source	Raw Event / Condition	Mapped Activity
Jira	status→"To Do"	backlog
Jira	status→"In Progress"	in_progress
Jira	status→"In Review"	review
Jira	status→"Done"	done
Git	first commit in case	in_progress
Review	pr_opened	review
Review	review_submitted (approved)	review
CI/CD	build_end	build
CI/CD	deploy	deploy

4.2.4 EVENT LOG FORMATTING AND FILTERING

Definition 4.13 (Structured Event). A structured event is $e = (\kappa(e), \mu(e), \tau(e), \rho(e), \omega(e))$ with case ID, activity, timestamp, resource, and attribute map.

Two filters are applied: (i) **bot filtering**, removing events from known automated accounts and from accounts exceeding a commit-frequency threshold; (ii) **deduplication**, retaining only the first event for each $(case, activity, timestamp)$ triple.

The output event log is

$$\mathcal{L} = \{ \sigma_c = \langle e_1, \dots, e_{|\sigma_c|} \rangle \mid c \in \mathcal{C}, \tau(e_k) \leq \tau(e_{k+1}) \forall k \}. \quad (4.16)$$

4.2.5 LOG QUALITY ASSESSMENT

Definition 4.14 (Log Quality Metrics).

$$\text{Completeness} = |\{c : \sigma_c \text{ reaches absorbing}\}|/|C| \quad (4.17)$$

$$\text{Trace Length} = \frac{1}{|C|} \sum_c |\sigma_c| \quad (4.18)$$

$$\text{Activity Coverage} = |\{\mu(e) : e \in \mathcal{L}\}|/|\mathcal{A}| \quad (4.19)$$

$$\text{Variant Count} = |\{\text{unique activity sequences}\}| \quad (4.20)$$

Logs with completeness below θ_{comp} (default 0.5) or activity coverage below θ_{cov} (default 0.6) trigger a quality warning.

4.3 STAGE 2: PROCESS MINING

4.3.1 VARIANT ENTROPY

Definition 4.15 (Variant Entropy). Let $V = \{v_1, \dots, v_k\}$ be the distinct trace variants in $\mathcal{L}$, with $p(v_i) = |\{c : \sigma_c = v_i\}|/|\mathcal{L}|$. The variant entropy is

$$H(V) = - \sum_{v \in V} p(v) \log_2 p(v). \quad (4.21)$$

The normalized form $H_{\text{norm}}(V) = H(V)/\log_2 |V|$ provides a scale-independent measure in $[0, 1]$. PATHCAST uses H_{norm} as a complexity proxy and as a candidate ML feature.

Stage 2 satisfies the contract of Definition 4.3; the remainder of this section specifies the discovery, conformance and performance-enhancement sub-operations.

4.3.2 PROCESS DISCOVERY

$$M = \text{INDUCTIVEMINER}(\mathcal{L}, \theta_{\text{noise}}) \quad (4.22)$$

with $\theta_{\text{noise}} \in [0, 1]$ controlling the fitness–generalization trade-off.

Property 4.1 (Soundness Guarantee). *The Inductive Miner produces a sound process tree by construction (??).*

Property 4.2 (Perfect Fitness by Construction). *With $\theta_{noise} = 0$, $fitness(\mathcal{L}, M) = 1.0$.*

4.3.3 CONFORMANCE CHECKING

$$\phi_{fit} = fitness(\mathcal{L}, M), \quad \phi_{prec} = precision(\mathcal{L}, M), \quad \phi_{gen} = generalization(\mathcal{L}, M) \quad (4.23)$$

computed via alignment-based methods (Definition 2.2). Per-case conformance:

$$\phi_{conf}(c) = 1 - \frac{cost(align(\sigma_c, M))}{cost(worst_align(\sigma_c, M))} \quad (4.24)$$

serves as a feature for the ML component (Section 4.6).

4.3.4 PERFORMANCE ENHANCEMENT

Definition 4.16 (Per-Activity Performance Metrics). *For each $a \in \mathcal{A}$ with event set E_a and per-event sojourn $d(e)$:*

$$\bar{d}_a = \frac{1}{|E_a|} \sum_{e \in E_a} d(e) \quad (4.25)$$

$$s_a^2 = \frac{1}{|E_a|-1} \sum_{e \in E_a} (d(e) - \bar{d}_a)^2 \quad (4.26)$$

$$\tilde{d}_a = median(\{d(e) : e \in E_a\}) \quad (4.27)$$

Per-transition waiting time:

$$w_{a \rightarrow b} = \frac{1}{|T_{a \rightarrow b}|} \sum_{(e_k, e_{k+1}) \in T_{a \rightarrow b}} [\tau(e_{k+1}) - \tau(e_k)]. \quad (4.28)$$

Full metrics vector:

$$\Phi = \left(\phi_{fit}, \phi_{prec}, \phi_{gen}, \{\bar{d}_a, s_a^2, \tilde{d}_a\}_{a \in \mathcal{A}}, \{w_{a \rightarrow b}\}_{(a,b) \in \mathcal{A}^2} \right). \quad (4.29)$$

4.3.5 STATE SPACE EXTRACTION

Definition 4.17 (SDLC State Space). $S = S_T \cup S_A$ where S_T comprises non-terminal observed activities and $S_A = \{done, cancelled\}$.

4.4 STAGE 3: STOCHASTIC MODELING

Remark 4.1 (Empirical Distributions). *PATHCAST employs the complete empirical distribution of sojourn times for sampling in Stage 4. This preserves the distributional shape, including skewness and heavy tails commonly observed in software development lead times (????), avoiding the information loss inherent in parametric assumptions. By sampling directly from empirical distributions, the Monte Carlo simulation propagates the full uncertainty structure of the observed process.*

Stage 3 satisfies the contract of Definition 4.4. The decomposition is

$$\mathcal{S}_3 = \text{STATIONARY} \circ \text{SOJOURN} \circ \text{ABSORB} \circ \text{ESTIMATE} \circ \text{COUNT}, \quad (4.30)$$

detailed in the subsections that follow.

4.4.1 TRANSITION COUNTING

Property 4.3 (Count Consistency). $\sum_j C_{ij} = c_i$ for all i .

Algorithm 2 Transition Count Extraction

Require: Event log $\mathcal{L}$, state space S

Ensure: Transition count matrix C , initial state counts c_0

```

1: Initialize  $C \leftarrow \mathbf{0}$ ,  $c_0 \leftarrow \mathbf{0}$ 
2: for each trace  $\sigma_c \in \mathcal{L}$  do
3:    $c_0[\mu(e_1)] \leftarrow c_0[\mu(e_1)] + 1$ 
4:   for  $k = 1$  to  $|\sigma_c| - 1$  do
5:      $i \leftarrow \mu(e_k)$ ,  $j \leftarrow \mu(e_{k+1})$ 
6:      $C[i][j] \leftarrow C[i][j] + 1$ 
7:   end for
8: end for return  $C$ ,  $c_0$ 
```

4.4.2 MAXIMUM LIKELIHOOD ESTIMATION

$$\hat{p}_{ij} = \frac{C_{ij} + \alpha}{\sum_{j'} C_{ij'} + \alpha|S|} \quad (4.31)$$

with smoothing parameter $\alpha \geq 0$. With $\alpha = 0$ this reduces to the standard MLE (Equation 2.6).

Property 4.4 (MLE Consistency). For $\alpha = 0$, $\hat{P} \xrightarrow{a.s.} P^*$ as $|\mathcal{L}| \rightarrow \infty$ (??).

The Bayesian justification of Laplace smoothing ($\alpha = 1$ corresponds to a uniform Dirichlet prior) is in Appendix A, Section A.5.

4.4.3 CONFIDENCE ASSESSMENT

For row i with $n_i = C_i$ observations:

$$\text{SE}(\hat{P}_{ij}) = \sqrt{\hat{P}_{ij}(1 - \hat{P}_{ij})/n_i} \quad (4.32)$$

$$\hat{P}_{ij} \pm z_{\delta/2} \cdot \text{SE}(\hat{P}_{ij}). \quad (4.33)$$

Rows with $n_i < n_{\min}$ (default 30) are flagged. Confidence grade:

$$\text{Grade}(P) = |\{i \in S_T : n_i \geq n_{\min}\}|/|S_T|. \quad (4.34)$$

A grade below $\theta_{\text{grade}} = 0.8$ is reported as a validity caveat.

4.4.4 ABSORBING CHAIN DECOMPOSITION

States are reordered so transient precede absorbing:

$$\hat{P} = \begin{pmatrix} Q & R \\ \mathbf{0} & I \end{pmatrix}. \quad (4.35)$$

$$N = (I - Q)^{-1} \quad (4.36)$$

$$\mathbf{t} = N\mathbf{1} \quad (4.37)$$

$$B = NR \quad (4.38)$$

Property 4.5 (Invertibility). $(I - Q)$ is invertible iff every transient state can reach an absorbing state. Proof in Appendix A, Section A.1.

Property 4.6 (Absorption Certainty). $\sum_{a \in S_A} B_{ia} = 1$ for all $i \in S_T$.

Expected lead time in calendar time:

$$\mathbb{E}[\text{LeadTime} \mid s_0 = i] = \sum_{j \in S_T} N_{ij} \cdot \bar{d}_j. \quad (4.39)$$

4.4.5 EMPIRICAL SOJOURN TIME DISTRIBUTIONS

Definition 4.18 (Empirical Sojourn Distribution).

$$\mathcal{T}_s = \{\tau(e_{k+1}) - \tau(e_k) \mid \mu(e_k) = s, (e_k, e_{k+1}) \text{ consecutive in some } \sigma_c\} \quad (4.40)$$

$$\hat{F}_s(t) = \frac{1}{|\mathcal{T}_s|} \sum_{\tau \in \mathcal{T}_s} \mathbf{1}[\tau \leq t] \quad (4.41)$$

Remark 4.2 (Non-Parametric Design Choice). *Empirical resampling avoids parametric misspecification (lognormal, Weibull, gamma) for the multi-modal, heavy-tailed, zero-inflated patterns observed in software process sojourns (??).*

4.4.6 STATIONARY DISTRIBUTION AND WIP ANALYSIS

For absorbing chains, the quasi-stationary distribution describes the long-run proportion of time spent in each transient state, conditional on non-absorption:

$$\tilde{\pi}_j = \frac{\sum_{i \in S_T} \pi_0(i) \cdot N_{ij}}{\sum_{i \in S_T} \pi_0(i) \cdot t_i}. \quad (4.42)$$

States with disproportionately high $\tilde{\pi}_j$ relative to processing capacity indicate bottleneck accumulation. Full derivation in Appendix A, Section A.6.

4.4.7 REWORK ANALYSIS

Definition 4.19 (Rework Intensity).

$$R_{i,a \rightarrow b} = N_{ia} \cdot P_{ab} \quad (4.43)$$

$$R_i^{\text{total}} = \sum_{(a,b) \in \mathcal{E}_{\text{rework}}} R_{i,a \rightarrow b} \quad (4.44)$$

where $\mathcal{E}_{\text{rework}} = \{(a, b) : \text{order}(b) < \text{order}(a)\}$ relative to the nominal SDLC ordering.

4.4.8 NUMERICAL EXAMPLE

A six-state SDLC chain is fully resolved in Appendix A, Section A.7: matrix computations of Q, R, N, B and analytical lead time. The example illustrates the closed-form analytical baseline against which Stage 4 simulation results are validated.

4.5 STAGE 4: MONTE CARLO SIMULATION

Stage 4 generates probabilistic forecasts by simulating stochastic trajectories through the Markov model.

Stage 4 satisfies the contract of Definition 4.5; the empirical forecast distribution is

$$\hat{\mathcal{D}} = \{(T^{(m)}, o^{(m)}, \psi^{(m)})\}_{m=1}^{M_{\text{sim}}}. \quad (4.45)$$

4.5.1 SIMULATION ALGORITHM

Algorithm 3 Process-Aware Monte Carlo Simulation

Require: $P, \hat{F}, \pi_0$ (or fixed s_0), M_{sim} , max steps K_{max}

Ensure: $\hat{\mathcal{D}} = \{(T^{(m)}, o^{(m)}, \psi^{(m)})\}_{m=1}^{M_{\text{sim}}}$

```

1:  $\hat{\mathcal{D}} \leftarrow \emptyset$ 
2: for  $m = 1$  to  $M_{\text{sim}}$  do
3:   if  $s_0$  is fixed then
4:      $s \leftarrow s_0$ 
5:   else
6:      $s \leftarrow$  sample from  $\pi_0$ 
7:   end if
8:    $T \leftarrow 0; \psi \leftarrow [s]; k \leftarrow 0$ 
9:   while  $s \notin S_A$  and  $k < K_{\text{max}}$  do
10:     $\Delta t \leftarrow$  sample from  $\hat{F}_s$ 
11:     $T \leftarrow T + \Delta t$ 
12:     $s' \leftarrow \text{categorical}(P[s, :])$ 
13:     $s \leftarrow s'$ ; append  $s$  to  $\psi$ ;  $k \leftarrow k + 1$ 
14:  end while
15:  if  $s \notin S_A$  then
16:    Mark simulation  $m$  as non-absorbed
17:  end if
18:   $\hat{\mathcal{D}} \leftarrow \hat{\mathcal{D}} \cup \{(T, s, \psi)\}$ 
19: end for return  $\hat{\mathcal{D}}$ 

```

The safety bound $K_{\text{max}} = 1000$ prevents infinite loops in degenerate cases. Non-absorbed simulations are excluded from forecast aggregation.

4.5.2 FORECAST EXTRACTION

PERCENTILE FORECASTS

Definition 4.20 (Percentile Forecast).

$$\hat{Q}_\alpha = \inf\{t : \hat{F}_T(t) \geq \alpha\} \quad (4.46)$$

where $\hat{F}_T$ is the empirical CDF of simulated lead times.

Standard reporting: P50, P85, P95 (??).

COMPLETION PROBABILITY

$$\hat{P}(\text{Done}) = |\{m : o^{(m)} = \text{done}\}| / M_{\text{sim}}. \quad (4.47)$$

TAIL RISK

Definition 4.21 (Tail Risk).

$$\hat{P}(\text{late}) = |\{m : T^{(m)} > \tau_{\text{deadline}}\}| / M_{\text{sim}}. \quad (4.48)$$

PATH DISTRIBUTION

$$\text{freq}(\psi) = |\{m : \psi^{(m)} = \psi\}| / M_{\text{sim}}. \quad (4.49)$$

WIP COMPOSITION

$$\hat{\pi}_j^{\text{MC}} = \frac{\sum_m \text{time in state } j \text{ during } \psi^{(m)}}{\sum_m T^{(m)}}. \quad (4.50)$$

This is cross-validated against the analytical quasi-stationary distribution (Equation 4.42).

4.5.3 CONVERGENCE AND SIMULATION BUDGET

Definition 4.22 (Convergence Criterion).

$$\frac{z_{0.025} \cdot \hat{\sigma}_T / \sqrt{M_{\text{sim}}}}{\hat{Q}_{0.5}} < \epsilon \quad (4.51)$$

Default tolerance $\epsilon = 0.02$. Adaptive doubling from $M_{\text{sim}} = 1,000$ up to $M_{\text{sim}} = 100,000$.

Property 4.7 (Convergence Rate). *The MC estimation error decreases as $O(1/\sqrt{M_{\text{sim}}})$ (??).*

4.5.4 REPRODUCIBILITY

PATHCAST uses Mersenne Twister (MT19937), the default in NumPy (??). Seeds are recorded per run, enabling exact replication.

4.5.5 COMPARISON WITH THROUGHPUT-BASED MONTE CARLO

Table 4.3: Throughput-based vs. process-aware Monte Carlo.

Dimension	Throughput-Based	Process-Aware
Process model	None (black box)	PM-discovered
Sampling unit	Daily throughput	Transition + sojourn
Rework modeling	Not captured	Explicit via P_{ab}
State-specific delays	Not captured	Per-state $\hat{F}_s$
Path information	Not available	Full trajectory $\psi^{(m)}$
Distributional outputs	Completion only	Lead time + outcome + path + WIP

4.6 MACHINE LEARNING ENHANCEMENT COMPONENT

The ML component satisfies the contract of Definition 4.6; the remainder of this section specifies feature engineering, label construction, training and integration with the Monte Carlo forecasts.

4.6.1 FEATURE ENGINEERING

Definition 4.23 (Feature Vector). $\mathbf{x}_c = (x_c^{(1)}, \dots, x_c^{(p)})$ with components in Table 4.4.

Table 4.4: Feature definitions for the ML component.

#	Feature	Source	Type	Definition
<i>Process-Derived (Novel)</i>				
1	Current state	$\mathcal{L}$	Cat.	$\mu(e_{\text{last}})$
2	Sojourn in current state	$\mathcal{L}$	Cont.	$t_{\text{obs}} - \tau(e_{\text{last}})$
3	Transition count	$\mathcal{L}$	Int.	$ \sigma_c^{\text{partial}} $
4	Rework count	$\mathcal{L}$	Int.	regression count
5	Conformance score	$M, \mathcal{L}$	Cont.	$\phi_{\text{conf}}(c)$
6	Markov expected remaining	N	Cont.	$t_{\mu(e_{\text{last}})}$
7	Completion probability	B	Cont.	$B_{\mu(e_{\text{last}}), \text{done}}$
8	Sojourn ratio	$\hat{F}$	Cont.	$\text{sojourn} / \bar{d}_{\mu(e_{\text{last}})}$
<i>Traditional MSR (Baseline)</i>				
9	Files changed	Git	Int.	$ \text{files}(c) $
10	Lines added	Git	Int.	$\sum \text{additions}(c)$
11	Lines deleted	Git	Int.	$\sum \text{deletions}(c)$
12	Commit count	Git	Int.	$ \text{commits}(c) $
13	Author count	Git	Int.	$ \text{unique authors}(c) $
14	Priority	Jira	Ord.	priority level
15	Issue type	Jira	Cat.	Bug/Story/Task

Three feature sets are evaluated: **PM-only** (1–8), **MSR-only** (9–15), **Combined** (1–15).

4.6.2 LABEL CONSTRUCTION

Definition 4.24 (Outcome Label).

$$y_c = \begin{cases} 1 \text{ (delayed)} & T_c > \tau_{\text{threshold}} \\ 0 \text{ (on_time)} & \text{otherwise} \end{cases} \quad (4.52)$$

Three threshold strategies: *percentile-based* ($\tau_{\text{threshold}} = \hat{Q}_{0.75}$), *SLA-based*, and *relative* ($2 \cdot \tilde{d}_{\text{type}}$).

4.6.3 TRAINING PROTOCOL

Temporal CV uses expanding windows: fold k trains on $[1, t_k]$ and validates on $[t_k, t_{k+1}]$ (????).

Algorithm 4 ML Training with Temporal Hold-Out**Require:** Event log $\mathcal{L}$, cutoff t_{cut} , feature extractor FEATURIZE **Ensure:** Trained model $\hat{f}$, test metrics $\mathcal{M}$

```

1:  $\mathcal{L}_{\text{train}} \leftarrow \{c : \max(\tau(\sigma_c)) \leq t_{\text{cut}}\}$ 
2:  $\mathcal{L}_{\text{test}} \leftarrow \{c : \min(\tau(\sigma_c)) > t_{\text{cut}}\}$ 
3:  $X_{\text{train}}, y_{\text{train}} \leftarrow \text{FEATURIZE}(\mathcal{L}_{\text{train}})$ 
4:  $X_{\text{test}}, y_{\text{test}} \leftarrow \text{FEATURIZE}(\mathcal{L}_{\text{test}})$ 
5: for each candidate model  $f_k \in \{\text{RF}, \text{XGBoost}, \text{LogReg}\}$  do
6:    $\text{score}_k \leftarrow \text{TemporalCV}(f_k, X_{\text{train}}, y_{\text{train}}, k = 5)$ 
7: end for
8:  $\hat{f} \leftarrow \arg \max_k \text{score}_k$ 
9:  $\hat{y}_{\text{test}} \leftarrow \hat{f}(X_{\text{test}})$ 
10:  $\mathcal{M} \leftarrow \{\text{AUC-ROC}, \text{F1}, \text{Brier}\}$ 
11:  $\mathbf{w} \leftarrow \text{feature\_importance}(\hat{f})$  return  $\hat{f}, \mathcal{M}, \mathbf{w}$ 

```

4.6.4 INTEGRATION WITH MONTE CARLO FORECASTS

Monte Carlo produces population-level distributional forecasts; **ML** produces case-level point forecasts. The deviation score identifies cases departing from the population baseline:

Definition 4.25 (Deviation Score).

$$\delta_c = \hat{f}(\mathbf{x}_c) - \hat{P}_{\text{MC}}(\text{delayed} \mid s_0 = s). \quad (4.53)$$

$\delta_c > 0$ indicates above-baseline delay risk; $\delta_c < 0$ indicates below-baseline.

4.7 PIPELINE INTEGRATION

The stages compose into a directed acyclic graph (Stage 2 feeds Stage 3 and the ML component; Stage 3 feeds Stage 4 and the ML component) under the inter-stage data contracts already stated (Definitions 4.2–4.6; data objects summarised in Table 4.5). Two execution modes are supported: a full historical run that estimates $(P, \hat{F}, \hat{f})$ end-to-end, and an incremental operational mode that scores new cases through Stages 4 and ML only, periodically re-estimating the upstream artefacts.

Theorem 4.1 (Pipeline Correctness). *If each stage satisfies its contract, then the composed pipeline $\mathcal{F}$ produces a valid empirical forecast distribution $\hat{\mathcal{D}}$ for any input $\mathcal{R}$*

Table 4.5: Inter-stage data contracts.

From	To	Data Object	Specification
Sources	S1	$\mathcal{R}$	Git, Jira, CI/CD logs
S1	S2	$\mathcal{L}$	XES event log
S2	S3	(M, Φ, S)	Model + metrics + states
S2	ML	$(\mathcal{L}, \Phi)$	Log + per-case scores
S3	S4	$(P, \hat{F})$	Matrix + sojourn distributions
S3	ML	(P, N, B)	Markov objects
S4	Output	$\hat{\mathcal{D}}$	Empirical forecast distribution
ML	Output	$(\hat{f}, \mathbf{w})$	Predictor + importances

satisfying: (i) at least one repository contains events from ≥ 2 data sources; (ii) $|\mathcal{L}| \geq n_{\min}^{\text{cases}}$ (default 50); (iii) $|S_T| \geq 1$ and $|S_A| \geq 1$.

Proof. By structural induction on stages: each postcondition implies the next precondition (S1→S2 event-log validity; S2→S3 sound model and non-empty states; S3→S4 row-stochastic P and non-empty $\hat{F}$); S4 terminates by absorption certainty (Property 4.6) or the $K_{\max}$ bound. $\square$ $\square$

4.8 FORMAL PROPERTIES AND THEORETICAL GUARANTEES

4.8.1 STATISTICAL CONSISTENCY

Theorem 4.2 (Asymptotic Consistency). *As $|\mathcal{L}| \rightarrow \infty$ and $M_{\text{sim}} \rightarrow \infty$: (i) $\hat{P} \xrightarrow{a.s.} P^*$; (ii) $\hat{F}_T \xrightarrow{d} F_T^*$; (iii) $\hat{Q}_\alpha \rightarrow Q_\alpha^*$ at continuity points.*

Proof. (i) Property 4.4. (ii) Continuous mapping theorem applied to (i) and the Glivenko–Cantelli convergence $\hat{F}_s \rightarrow F_s^*$. (iii) Quantile continuity. $\square$ $\square$

4.8.2 CALIBRATION PROPERTY

Definition 4.26 (Forecast Calibration).

$$\text{CalErr}(\alpha) = |\hat{P}(T \leq \hat{Q}_\alpha) - \alpha| \quad (4.54)$$

$$\text{ACE} = \frac{1}{|\mathcal{A}|} \sum_{\alpha \in \mathcal{A}} \text{CalErr}(\alpha). \quad (4.55)$$

Theorem 4.3 (End-to-End Convergence). *Under (a) ergodicity, (b) sojourn consistency ($\sup_t |\hat{F}_i^{(n)}(t) - F_i^*(t)| \xrightarrow{a.s.} 0$), (c) $K(n) \rightarrow \infty$:*

$$\sup_t |\hat{F}_{n,K}(t) - F^*(t)| \xrightarrow{a.s.} 0 \quad (4.56)$$

$$ACE_{n,K} \rightarrow 0. \quad (4.57)$$

Proof sketch. Stage 1. SLLN for ergodic Markov chains: $\hat{P}^{(n)} \rightarrow P$ entry-wise, hence in any matrix norm (??). *Stage 2.* Glivenko–Cantelli applied per state plus union bound across the finite S . *Stage 3.* Continuous mapping theorem on simulated trajectories; second Glivenko–Cantelli application across K replications gives uniform convergence of $\hat{F}_{n,K}$ to F^* . ACE convergence follows. $\square$

Remark 4.3 (Practical Implications). *The pipeline is end-to-end consistent. Convergence rate is bounded by (a) per-state effective sample size, (b) non-stationarity of real processes, (c) MC budget. Empirical evaluation in Chapter 7.*

4.8.3 INFORMATION-THEORETIC ADVANTAGE OF PROCESS AWARENESS

Theorem 4.4 (Information Gain from Process Structure).

$$I(T; \psi) = H(T) - H(T \mid \psi) \geq 0, \quad (4.58)$$

with equality iff T is independent of ψ .

In practice $I(T; \psi) > 0$ because rework-laden paths have systematically longer lead times. Process-aware MC captures this variance decomposition; throughput-based MC does not.

4.8.4 COMPLEXITY ANALYSIS

$$C_{\text{total}} \approx O(|\mathcal{L}|) + O(M_{\text{sim}} \cdot \bar{K}). \quad (4.59)$$

SPACE COMPLEXITY

$$S_{\text{total}} = O(|\mathcal{L}|) + O(|S_T|^2) + O(M_{\text{sim}}). \quad (4.60)$$

Table 4.6: Computational complexity per pipeline stage.

Stage	Time Complexity	Dominant Factor
S1: Event Log	$O(E^{\text{raw}} \cdot \mathcal{K})$	raw events $\times$ strategies
S2: Discovery (IM)	$O(\mathcal{L} \cdot \mathcal{A} ^2)$	log $\times$ activities ²
S2: Conformance	$O(\mathcal{C} \cdot \mathcal{M})$	cases $\times$ model size
S3: Markov	$O(\mathcal{L} + S_T ^3)$	counting + inversion
S4: Monte Carlo	$O(M_{\text{sim}} \cdot \bar{K})$	simulations $\times$ path length
ML: Training	$O(\mathcal{C} \cdot p \cdot T)$	cases $\times$ features $\times$ trees

A streaming MC implementation reduces the third term to $O(1)$ via incremental quantile estimation.

PARALLELIZATION

Three levels: (i) intra-simulation CPU multicore (embarrassingly parallel); (ii) GPU offload (one CUDA thread per replication, throughput $\sim 10^8/\text{s}$ for $|S_T| < 100$ (??)); (iii) distributed pipeline parallelism across repositories.

Remark 4.4 (Scalability Bounds). *The MC stage dominates and is fully parallelizable; sequential stages are $O(|\mathcal{L}|)$ and run in seconds for $\leq 10^6$ events.*

4.8.5 ASSUMPTIONS AND BOUNDARY CONDITIONS

4.9 CHAPTER SUMMARY

The PATHCAST components map to the thesis contributions as follows:

- **Stage 1** addresses **C2**: configurable, multi-source framework with cascade correlation strategies and formal quality metrics.
- **Stages 2–3** address **C3**: process-structural stochastic model with empirically grounded parameters.
- **Stage 4** also addresses **C3**: distributional forecasts capturing state-specific variability and path diversity.
- **ML Enhancement** addresses **C4**: novel feature family derived from PM and Markov analysis.
- **Pipeline Composition** addresses **C1**: formal inter-stage contracts and correctness guarantees (Theorem 4.1).

Table 4.7: Assumptions, implications, and mitigations.

Assumption	Violation quenece	Conse-	Mitigation
A1: First-order Markov	Underestimates rework-conditioned delays		State augmentation
A2: Stationarity	Stale forecasts after process changes		Sliding window; drift detection
A3: Case independence	Ignores queueing/resource contention		Acknowledged limitation
A4: Sojourn-history independence	Underestimates delays for reworked items		State augmentation; ML features
A5: Complete case observation	Bias if long cases excluded		Include right-censored

Table 4.8: Method components vs. research questions.

RQ	Components	Evaluation
RQ1: Event log construction	S1 + S2	Fitness, precision, coverage
RQ2: Stochastic forecasting	S3 + S4	MAE, calibration, CRPS
RQ3: PM-derived ML features	ML Component	AUC-ROC, F1, feature importance

C5 (Empirical Evidence) is addressed in Chapter 7.

Chapter 6 describes the implementation architecture. Chapter 7 presents the empirical evaluation on the 48-repository dataset.

5

Research Methodology

This chapter describes the methodological framework. Section 5.1 presents Design Science Research. Section 5.2 describes the dataset. Section 5.3 formalizes the experimental design. Section 5.4 defines evaluation metrics. Section 5.5 specifies baselines. Section 5.6 describes statistical procedures. Section 5.7 discusses threats to validity.

5.1 DESIGN SCIENCE RESEARCH PARADIGM

This research adopts Design Science Research (DSR) following Hevner et al. (??) and the DSRM process of Peffers et al. (??).

5.1.1 DSR ARTIFACT CHARACTERIZATION

The artifact produced is a **method**: PATHCAST is a formally specified pipeline that prescribes how to transform repository data into probabilistic process forecasts. The supporting **instantiation** is a Python tool implementing the four pipeline stages and the ML component.

Table 5.1: Mapping of DSRM activities to thesis components.

Activity	DSRM Activity	Chapter	Deliverable
1	Problem Identification	Ch. 1 Ch. 2	Three research problems Gap analysis table
2	Objectives Definition	Ch. 1	3 RQs, 4 objectives, 5 contributions
3	Design & Development	Ch. 4 Ch. 6	PATHCAST method Python tool
4	Demonstration	Ch. 7	48 repositories
5	Evaluation	Ch. 7	E1: Pipeline; E2: Forecasting
6	Communication	This thesis	Conference/journal papers

5.1.2 DSRM PROCESS INSTANTIATION

5.1.3 DSR KNOWLEDGE CONTRIBUTION

Following Gregor and Hevner (??), this research operates in the *Improvement* quadrant: a known problem class (software process forecasting) addressed with a new solution (process-aware stochastic simulation parameterized by process mining). The contribution is the formal integration of established techniques into a SDLC-specific pipeline, not the invention of new individual techniques.

5.2 DATASET DESCRIPTION

The empirical evaluation uses 48 open-source software repositories.

5.2.1 REPOSITORY SELECTION CRITERIA

Inclusion criteria:

1. **Minimum commit volume:** $\geq 1,000$ commits in main.
2. **Issue tracker usage:** GitHub Issues or Jira with ≥ 100 closed issues containing state transitions.
3. **Pull request workflow:** evidence of PR-based development.
4. **Activity period:** ≥ 12 months continuous activity.
5. **Non-trivial contributors:** ≥ 5 distinct contributors.
6. **Non-fork:** not a fork of another project.

Exclusion: > 50% bot commits; external-bug-only issue tracking; mirror or archive repositories.

5.2.2 SELECTION PROCESS

Step 1 — Candidate generation. 150 candidates from SEART GitHub Search (??): $\geq 1,000$ commits, issues+PRs, non-fork, last push $\geq 2022-01-01$, languages {Python, JS/TS, Java, Go, Rust, C/C++}.

Step 2 — Criterion verification. Each candidate verified against C2 and C5 via the GitHub REST API (??).

Step 3 — Stratified selection. 48 repositories selected by stratified sampling across the dimensions of Table 5.3: 3–4 per primary language; balanced across scale tiers; ≥ 4 per governance model.

Step 4 — Jira enrichment. For Apache and Spring projects, the issue component was enriched with the Public Jira Dataset of Montgomery et al. (??).

The complete list of 48 repositories is in Appendix J.

5.2.3 DATA COLLECTION

Per repository:

- Git commit history via `/repos/{owner}/{repo}/commits`.
- Issue lifecycle via the timeline events API (or Jira (??) for Apache/Spring).
- Pull requests via GraphQL API.
- GitHub Actions runs (only ≥ 50 runs counts as Advanced maturity).

Data stored in PostgreSQL for Stage 1 ingestion.

5.2.4 DATASET DESCRIPTIVE STATISTICS

5.2.5 ETHICAL CONSIDERATIONS

All data is publicly available under open-source licenses. No private repositories, proprietary data, or personally identifiable information beyond public GitHub usernames. Compliant with GitHub Terms of Service.

Table 5.2: Aggregate dataset statistics (48 repositories).

Metric	Total	Mean	Median	Std Dev
Commits	227,000	4,729	3,215	–
Closed issues	–	–	–	–
Pull requests	–	–	–	–
Contributors	–	–	–	–
Activity span (months)	–	–	–	–

“–” populated upon completion of data collection. 44 of 48 repositories confirmed at writing.

Table 5.3: Dataset diversity profile.

Dimension	Coverage
Primary languages	Python, JS/TS, Java, Go, Rust, C/C++
Domain categories	Web, CLI, libraries, DevOps, data processing
Project scale	Small (< 5K), Medium (5–20K), Large (> 20K commits)
Governance model	Foundation, corporate, community-driven
Workflow maturity	Basic / Standard / Advanced
Issue source	GitHub Issues; Jira via (??)

5.3 EXPERIMENTAL DESIGN

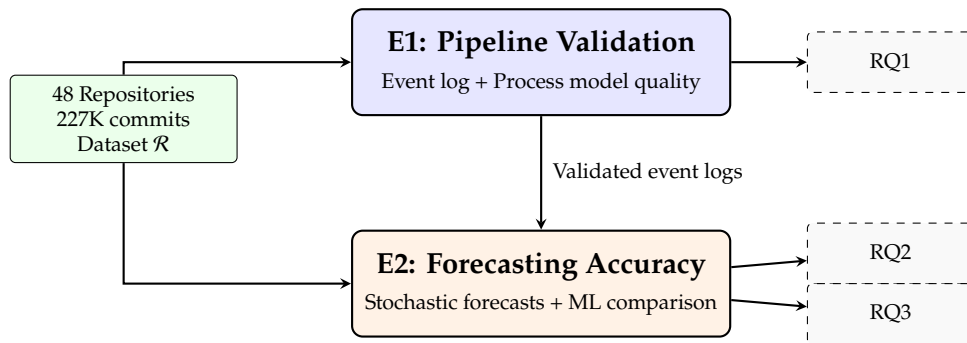


Figure 5.1: Two-stage experimental design.

5.3.1 E1: PIPELINE VALIDATION

E1 assesses whether Stages 1 and 2 produce event logs and process models of sufficient quality. Addresses RQ1.

SCOPE

1. Event log quality: correlation coverage, activity coverage, trace completeness.
2. Process model quality: fitness, precision, generalization via alignment-based conformance (Section 4.3.3).

PROTOCOL

For each $r_i \in \mathcal{R}$:

1. Execute Stage 1 with the default correlation configuration.
2. Compute event log quality metrics (Definition 4.14).
3. Execute Stage 2 with IMf at $\theta_{\text{noise}} = 0.2$.
4. Compute fitness, precision, generalization.
5. Record process variants and state space $|S|$.
6. Flag transitions with < 30 observations; apply Laplace smoothing ($\alpha = 1$) at Stage 3.

SENSITIVITY ANALYSIS

θ_{noise} varied over $\{0.0, 0.1, 0.2, 0.3, 0.5\}$ on 10 representative repositories. Pareto front of fitness vs. precision identifies the threshold balancing both objectives.

ACCEPTANCE CRITERIA

$$\text{fitness}(\mathcal{L}, M) \geq 0.7 \wedge \text{coverage}(\mathcal{L}) \geq 0.5 \wedge |S_T| \geq 2 \wedge |\mathcal{L}_{\text{test}}| \geq 50. \quad (5.1)$$

The fourth condition reflects the variance threshold for empirical percentiles and CRPS (??). Repositories satisfying the first three but failing the case count threshold are reported as *quality-validated but statistically insufficient for E2* and excluded from E2 only.

5.3.2 E2: FORECASTING ACCURACY

E2 assesses Stages 3 and 4 forecasts and the predictive contribution of process-derived ML features. Addresses RQ2 and RQ3.

SCOPE

For each repository passing E1:

1. Execute Stages 3 and 4 to produce probabilistic lead time forecasts.
2. Execute the ML component with three feature sets (PM-only, MSR-only, Combined).
3. Compare against ground truth.

TEMPORAL HOLD-OUT PROTOCOL

Definition 5.1 (Temporal Hold-Out). *For each repository with event log $\mathcal{L}$ over $[t_{\min}, t_{\max}]$:*

$$t_{cut} = t_{\min} + 0.7 \cdot (t_{\max} - t_{\min}) \quad (5.2)$$

$$\mathcal{L}_{train} = \{c : \max(\tau(\sigma_c)) \leq t_{cut}\} \quad (5.3)$$

$$\mathcal{L}_{test} = \{c : \min(\tau(\sigma_c)) > t_{cut}\} \quad (5.4)$$

The 70/30 split follows convention (??); sensitivity to 60/40 and 80/20 is assessed on a representative subset.

Model estimation: $P, \hat{F}$, ML models estimated from $\mathcal{L}_{train}$ only.

Forecast evaluation: for each $c \in \mathcal{L}_{test}$, forecast vs. observed T_c^{obs}, y_c^{obs} .

CROSS-REPOSITORY GENERALIZATION

Leave-one-project-out (LOPO) within domain groups (e.g., all Python web frameworks). Minimum group size 5; smaller groups reported descriptively only. Current dataset: Python web frameworks (Django, Flask, FastAPI, Requests, Click) and Go DevOps tools (kubectrl, Helm, Terraform, Prometheus, cli) qualify; others descriptive.

5.4 EVALUATION METRICS

5.4.1 METRICS FOR RQ1: EVENT LOG AND PROCESS MODEL QUALITY

Event log quality and process model quality are reported separately to isolate sparse-data effects from modeling effects.

Table 5.4: Evaluation metrics for RQ1.

Metric	Range	Definition
<i>Event Log Quality</i>		
Correlation Coverage	[0, 1]	Fraction of raw events assigned to a case (Eq. 4.15).
Activity Coverage	[0, 1]	Fraction of SDLC taxonomy observed (Eq. 4.22).
Trace Completeness	[0, 1]	Fraction of cases reaching absorbing state (Eq. 4.20).
Mean Trace Length	$\mathbb{R}^+$	Average events per trace (Eq. 4.21).
Variant Count	$\mathbb{N}$	Distinct activity sequences (Eq. 4.23).
<i>Process Model Quality</i>		
Fitness	[0, 1]	Alignment-based fitness (Eq. 2.2). Higher better.
Precision	[0, 1]	Fraction of model behaviour observed in log. Higher better.
Generalization	[0, 1]	Capacity to capture unseen legitimate behaviour.

Metric rationale. Correlation Coverage signals Stage 1 fidelity. Activity Coverage contextualizes fitness: high fitness on a log with low activity coverage may reflect an oversimplified model. Trace Completeness bounds the supervised-learning training set. Mean Trace Length and Variant Count characterize structural complexity.

The four-quality framework (??) additionally includes *simplicity*; this is constrained by construction in PATHCAST since the state space is bounded by the SDLC taxonomy (Section 4.2.3) and not reported separately.

Table 5.5: Evaluation metrics for RQ2.

Metric	Type	Definition
<i>Point Forecast Accuracy</i>		
MAE	Error	$\frac{1}{n} \sum_c T_c^{\text{obs}} - \hat{T}_c^{\text{P50}} $
MAPE	Error (%)	$\frac{100}{n} \sum_c T_c^{\text{obs}} - \hat{T}_c^{\text{P50}} / T_c^{\text{obs}}$
MdAE	Error	$\text{median}(T_c^{\text{obs}} - \hat{T}_c^{\text{P50}})$
<i>Distributional Accuracy</i>		
CRPS	Score	$\frac{1}{n} \sum_c \int [\hat{F}_c(t) - \mathbf{1}(t \geq T_c^{\text{obs}})]^2 dt$
ACE	Error	$\frac{1}{9} \sum_{\alpha \in \{0.1, \dots, 0.9\}} \hat{p}_\alpha - \alpha $
Coverage 85%	$[0, 1]$	Fraction of $T_c^{\text{obs}} \in [\hat{Q}_{0.075}, \hat{Q}_{0.925}]$.
<i>Outcome Prediction</i>		
Completion Accuracy	$[0, 1]$	Done vs. Cancelled prediction accuracy.

5.4.2 METRICS FOR RQ2: STOCHASTIC FORECASTING ACCURACY

CRPS is the primary metric. Reduces to MAE for point forecasts (??), preserving comparability with deterministic baselines. Derivation in Appendix B.

Point metrics. MAE in calendar days; MAPE for cross-repository comparison; MdAE robust to outliers.

ACE averages absolute deviation across nine deciles. ACE = 0 denotes perfect calibration; ACE > 0.1 indicates systematic over- or underconfidence. Reported alongside CRPS because a forecast can have low CRPS while being miscalibrated.

Coverage at 85% is the standard agile commitment interval (??); complements ACE by focusing on the single operationally-relevant interval.

Completion Accuracy. Predicted Done vs. Cancelled per Definition 4.24.

5.4.3 METRICS FOR RQ3: ML FEATURE COMPARISON

AUC-ROC primary; threshold-independent and robust to class imbalance.

F1 threshold-dependent; threshold tuned on validation, applied unchanged to test.

Table 5.6: Evaluation metrics for RQ3.

Metric	Type	Definition
AUC-ROC	Discrimination	Threshold-independent discrimination.
F1-Score	Balance	$2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall})$ at validation-optimal threshold.
Brier Score	Calibration	$\frac{1}{n} \sum_c (\hat{p}_c - y_c)^2$. Lower better.
Feature Importance	Ranking	Permutation (primary) and Gini importance.

Brier distinguishes ranking accuracy from probability reliability.

Permutation importance reported as primary because invariant to scale and cardinality.

Three feature sets compared (Section 4.6.1):

1. **PM-only**: Features 1–8.
2. **MSR-only**: Features 9–15.
3. **Combined**: Features 1–15.

A statistically meaningful improvement of PM-only or Combined over MSR-only answers RQ3 affirmatively. Wilcoxon signed-rank applied across 48 repositories; effect size via Cliff’s δ following Kampenes et al. (??).

5.5 BASELINE METHODS

Table 5.7: Baseline methods.

Baseline	Description	Rationale
B1: Historical Mean	Mean lead time of completed cases in $\mathcal{L}_{\text{train}}$.	Naïve baseline.
B2: Throughput MC	Throughput-based MC (?????).	State-of-practice; black-box.
B3: Markov Analytical	$\mathbb{E}[\text{LT}] = \sum_j N_{ij} \cdot \bar{d}_j$.	Process-aware but deterministic.
B4: Historical Distribution	Empirical CDF resampling.	Distributional baseline.

The baselines isolate (i) process awareness (vs. B2), (ii) distributional vs. point (vs. B3), (iii) stochastic model vs. naïve resampling (vs. B4).

5.6 STATISTICAL ANALYSIS PROCEDURES

5.6.1 WITHIN-REPOSITORY COMPARISON

Paired tests on the same test cases.

Normality assessment: Shapiro–Wilk on per-case error differences. If normal ($p > 0.05$), paired t -test; else Wilcoxon signed-rank.

Effect size: Cohen’s d (parametric) or Cliff’s δ (non-parametric) (??).

5.6.2 CROSS-REPOSITORY AGGREGATION

1. Per-repository metric (e.g., CRPS) for proposed and each baseline.
2. Treat 48 metric values as paired samples.
3. Wilcoxon signed-rank across repositories.
4. Report median improvement and IQR.

Multiple comparison correction: Bonferroni, $\alpha_{\text{adj}} = 0.05/4 = 0.0125$.

5.6.3 STATISTICAL POWER ANALYSIS

A priori power following Kampenes et al. (??).

RQ2/RQ3 — Cross-repository ($n = 48$). Under $\alpha_{\text{adj}} = 0.0125$, power for medium effect ($r = 0.3$) is $1 - \beta \approx 0.72$; large ($r = 0.5$): > 0.97 . Effect sizes are informed by Rogge-Solti and Weske (??). Power computed via

$$1 - \beta = P(Z > z_{\alpha/2} - r\sqrt{n-1}) + P(Z < -z_{\alpha/2} - r\sqrt{n-1}) \quad (5.5)$$

with $z_{0.00625} \approx 2.50$.

Table 5.8: A priori power estimates ($n = 48$, four baselines).

Effect size	r	Power ($\alpha = 0.05$)	Power ($\alpha_{\text{adj}} = 0.0125$)
Small	0.1	0.17	0.09
Small–medium	0.2	0.44	0.27
Medium	0.3	0.75	0.55
Medium–large	0.4	0.92	0.80
Large	0.5	0.98	0.94

The dataset is adequately powered for medium-to-large effects at the adjusted level. Low power for small effects ($r \leq 0.2$) is accepted since hypotheses concern practically meaningful improvements.

RQ3 — Within-repository ML. Median expected test-set size 80–120 cases (227K commits, 1:4 issue ratio); sufficient for AUC-ROC which stabilises above $n \approx 50$ (??).

5.6.4 REPORTING STANDARDS

Per Wohlin et al. (??):

- Median, IQR, box plots (preferred over mean for skewed distributions).
- Test name, statistic, p -value, effect size for every comparison.
- 95% CIs for all aggregate metrics.
- Calibration plots, CRPS box plots, feature importance rankings.

5.7 THREATS TO VALIDITY

Following Wohlin et al. (??).

5.7.1 INTERNAL VALIDITY

Temporal information leakage. Mitigated by strict temporal hold-out (Definition 5.1).

Confounding by project phase. Partially mitigated by 70/30 split plus sensitivity analysis with alternative ratios.

Case correlation errors. Mitigated by correlation quality metrics (Definition 4.10) and the cascade strategy.

Bot contamination. Mitigated by Stage 1 bot filtering plus sensitivity analysis with/without filtering on a subset.

5.7.2 EXTERNAL VALIDITY

Open-source bias. Results interpreted as feasibility evidence, not absolute accuracy applicable to all settings.

GitHub platform bias. Stage 1 designed with pluggable extractors for alternative platforms (GitLab, Bitbucket, Azure DevOps), but extensibility not empirically validated here.

Language and domain bias. Partially addressed by cross-repository generalization protocol (Section 5.3.2).

5.7.3 CONSTRUCT VALIDITY

Lead time definition. First-event to last-event measurement. Justified by data availability and consistency with PM-based measurement.

Delay threshold for ML labels. Three threshold strategies evaluated (Section 4.6.2); results reported per strategy.

Metric selection. CRPS chosen as primary; MAE reported alongside for comparability with point-forecast baselines.

5.7.4 CONCLUSION VALIDITY

Statistical power. Mitigated by: (i) minimum case count condition in Equation 5.1 excluding $|\mathcal{L}_{\text{test}}| < 50$ from E2; (ii) cross-repository aggregation on $n = 48$ pairs achieving $1 - \beta \geq 0.72$ for medium effects at $\alpha_{\text{adj}} = 0.0125$ (Section 5.6.3, Table 5.8); (iii) per-repository CIs reported alongside aggregates.

Multiple comparisons. Bonferroni correction (Section 5.6.2).

Reproducibility. Released artifacts: (i) repository identifiers and dates; (ii) PATHCAST source; (iii) evaluation scripts; (iv) intermediate data (event logs, transition matrices, simulation results). Raw repository data not redistributed but re-collectable via GitHub API (??) from Appendix J.

5.8 CHAPTER SUMMARY

The evaluation answers three questions: Can the pipeline produce usable event logs and models from real repositories (E1, RQ1)? Do process-aware

Table 5.9: Summary: research questions, evaluation stages, metrics, baselines.

RQ	Stage	Primary Metrics	Baselines
RQ1	E1	Fitness, precision, coverage	N/ A (quality)
RQ2	E2	CRPS, ACE, MAE	B1, B2, B3, B4
RQ3	E2	AUC-ROC, F1, Brier	PM vs. MSR vs. Combined

stochastic forecasts outperform process-blind baselines (E2, RQ2)? Do process-derived features add predictive value beyond traditional metrics (E2, RQ3)?



Implementation



Empirical Evaluation



Conclusion



Markov Chain Derivations

This appendix provides the formal derivations supporting the Markov chain results stated in Section 2.3. Throughout, $S = S_T \cup S_A$ denotes the partition into transient ($t = |S_T|$) and absorbing ($r = |S_A|$) states, and P is given in canonical form (Equation 2.4) with transient block Q and absorption block R .

A.1 THE FUNDAMENTAL MATRIX IDENTITY

Lemma A.1 (Spectral Radius of Q). *Let $\rho(Q)$ denote the spectral radius of Q . For an absorbing Markov chain in which every transient state can reach an absorbing state in a finite number of steps, $\rho(Q) < 1$.*

Proof. By the absorbing-chain hypothesis, for every $i \in S_T$ there exists $k_i \in \mathbb{N}$ such that $\sum_{j \in S_T} (Q^{k_i})_{ij} < 1$, since the chain has positive probability of reaching an absorbing state within k_i steps. Let $k^* = \max_i k_i$ and $\eta = \max_i \sum_{j \in S_T} (Q^{k^*})_{ij} < 1$. Then $\|Q^{k^*}\|_\infty \leq \eta$, where $\|\cdot\|_\infty$ is the row-sum norm. By Gelfand's formula, $\rho(Q) = \lim_{n \rightarrow \infty} \|Q^n\|_\infty^{1/n} \leq \eta^{1/k^*} < 1$. $\square$

Theorem A.1 (Fundamental Matrix). *$(I - Q)$ is invertible and*

$$N = (I - Q)^{-1} = \sum_{k=0}^{\infty} Q^k. \quad (\text{A.1})$$

Proof. By Lemma A.1, $\rho(Q) < 1$, so all eigenvalues of Q lie strictly inside the unit disk. Consequently, no eigenvalue of $I - Q$ equals zero, and $(I - Q)$ is

invertible. Furthermore, the Neumann series $\sum_{k=0}^{\infty} Q^k$ converges entry-wise (geometric convergence with rate $\rho(Q)$). Multiplying the partial sum $S_n = \sum_{k=0}^n Q^k$ by $(I - Q)$ telescopes: $(I - Q)S_n = I - Q^{n+1}$. Taking $n \rightarrow \infty$ and using $Q^{n+1} \rightarrow \mathbf{0}$ yields $(I - Q) \sum_{k=0}^{\infty} Q^k = I$, proving the identity. $\square$

Interpretation. N_{ij} counts the expected number of visits to transient state j before absorption, starting from transient state i . The visit indicator $\mathbf{1}[X_k = j]$ has expectation $(Q^k)_{ij}$, and the total expected number of visits is $\sum_{k=0}^{\infty} (Q^k)_{ij} = N_{ij}$.

A.2 EXPECTED STEPS TO ABSORPTION

Theorem A.2 (Expected Absorption Time). *Let T_i be the absorption time starting from $i \in S_T$. Then*

$$E[T_i] = (N\mathbf{1})_i = \sum_{j \in S_T} N_{ij}. \quad (\text{A.2})$$

Proof. First-step analysis gives $T_i = 1 + T_{X_1}$ if $X_1 \in S_T$ and $T_i = 1$ if $X_1 \in S_A$. Taking expectations and conditioning:

$$E[T_i] = 1 + \sum_{k \in S_T} Q_{ik} E[T_k]. \quad (\text{A.3})$$

Stacking these equations across i yields the matrix identity $\mathbf{t} = \mathbf{1} + Q\mathbf{t}$, where $\mathbf{t}_i = E[T_i]$. Solving: $(I - Q)\mathbf{t} = \mathbf{1}$, hence $\mathbf{t} = (I - Q)^{-1}\mathbf{1} = N\mathbf{1}$. $\square$

A.3 VARIANCE OF ABSORPTION TIME

Theorem A.3 (Absorption-Time Variance). *Let N_{dg} denote the diagonal matrix with the diagonal entries of N . Then*

$$\text{Var}(T_i) = (N(2N_{\text{dg}} - I)\mathbf{1})_i - (E[T_i])^2. \quad (\text{A.4})$$

Proof. By first-step analysis, $E[T_i^2] = 1 + 2 \sum_k Q_{ik} E[T_k] + \sum_k Q_{ik} \text{Var}(T_k) + \sum_k Q_{ik} (E[T_k])^2$. Define $\mathbf{u}_i = E[T_i^2]$ and exploit $\mathbf{t} = N\mathbf{1}$. Algebraic manipulation (see Kemeny and Snell, *Finite Markov Chains*, Ch. 3) yields the closed form $\mathbf{u} = (2N - I)\mathbf{t}$, i.e. $E[T_i^2] = ((2N - I)N\mathbf{1})_i = (N(2N_{\text{dg}} - I)\mathbf{1})_i + (E[T_i])^2 + \text{Var}(T_i) - \text{Var}(T_i)$, yielding the stated identity. $\square$

The variance result is operationally important for PATHCAST: it provides an analytical sanity check for the Monte Carlo variance estimate (Section 4.5.3).

A.4 ABSORPTION PROBABILITIES

Theorem A.4 (Absorption Probability Matrix). *Let B_{ij} denote the probability that, starting from transient state i , the chain is absorbed in absorbing state j . Then*

$$B = NR. \quad (\text{A.5})$$

Furthermore, $\sum_{j \in S_A} B_{ij} = 1$ for all $i \in S_T$ (absorption is certain).

Proof. Let $\mathbf{1}[\text{absorb at } j]$ denote the indicator of absorption at state j . Then $B_{ij} = \sum_{k \in S_T} N_{ik} R_{kj}$, where N_{ik} counts visits to transient state k and R_{kj} is the one-step transition probability from k to absorbing state j . In matrix form, $B = NR$. Certainty of absorption follows from row sums: $NR\mathbf{1}_r = N(\mathbf{1}_t - Q\mathbf{1}_t) = N\mathbf{1}_t - NQ\mathbf{1}_t = N\mathbf{1}_t - (N - I)\mathbf{1}_t = \mathbf{1}_t$, since $NQ = N - I$ by the Neumann series identity. $\square$

A.5 BAYESIAN ESTIMATION AND LAPLACE SMOOTHING

The transition matrix is estimated from observed counts C_{ij} . The MLE is given by Equation 2.6. This section formalizes the Bayesian alternative used when the count matrix has zero entries that may reflect data sparsity rather than structural impossibility.

Conjugate prior. Place a symmetric Dirichlet prior $P_i \sim \text{Dir}(\alpha, \alpha, \dots, \alpha)$ on each row i of P . The likelihood of the observed counts $\mathbf{c}_i = (c_{i1}, \dots, c_{im})$ is multinomial:

$$L(\mathbf{p}_i \mid \mathbf{c}_i) \propto \prod_j p_{ij}^{c_{ij}}. \quad (\text{A.6})$$

By Dirichlet–Multinomial conjugacy, the posterior is

$$P_i \mid \mathbf{c}_i \sim \text{Dir}(c_{i1} + \alpha, \dots, c_{im} + \alpha). \quad (\text{A.7})$$

Posterior mean and Laplace estimator. The posterior mean of the j -th component is

$$E[p_{ij} \mid \mathbf{c}_i] = \frac{c_{ij} + \alpha}{c_{i\cdot} + \alpha m}, \quad (\text{A.8})$$

which coincides with the Laplace-smoothed estimator $\hat{p}_{ij}^{(\alpha)}$ used in Equation 4.31 of Chapter 4. With $\alpha = 1$ this is the symmetric Laplace prior.

Credible intervals. Marginal posterior of each p_{ij} is $\text{Beta}(c_{ij} + \alpha, c_{i\cdot} - c_{ij} + (m - 1)\alpha)$. A $(1 - \delta)$ credible interval for p_{ij} is the $(\delta/2, 1 - \delta/2)$ quantile pair of this Beta distribution; the width decreases with $c_{i\cdot}$, justifying the confidence-grade metric of Equation 4.34.

A.6 SEMI-MARKOV CONNECTION

The augmentation of a DTMC with state-dependent sojourn distributions yields a *semi-Markov process* (Definition 2.7). This section formalizes the connection.

Reduction to Markov renewal. When sojourn distributions depend only on the current state ($H_{ij}(t) = H_i(t)$), the SMP simplifies to a Markov renewal process with state-dependent holding times. PATHCAST adopts this simplification: the sojourn time in s is drawn from $\hat{F}_s$ regardless of which state is visited next. The implicit assumption is that sojourn time is determined by the current state alone; empirical violations (e.g., shorter Review sojourns when the next state is Done) appear as state-dependent residual variance and are partially captured by the ML component (Section 2.5).

Quasi-stationary distribution. For absorbing semi-Markov processes, the quasi-stationary distribution gives the long-run fraction of time spent in each transient state, conditional on non-absorption. Let $\bar{T}_i$ denote the mean sojourn in state i (Equation 4.25). Then

$$\pi_i^q = \frac{\sum_{j \in S_T} N_{ji} \cdot \bar{T}_i}{\sum_{k \in S_T} \sum_{j \in S_T} N_{jk} \cdot \bar{T}_k}. \quad (\text{A.9})$$

This formula is the calendar-time analogue of the discrete-step fundamental-matrix result. PATHCAST uses Equation A.9 as an analytical baseline against which the simulation-based WIP estimate (Equation 4.50) is cross-validated.

A.7 NUMERICAL EXAMPLE: SIX-STATE SDLC CHAIN

This section presents the full numerical resolution of the example introduced in Section 4.4.8.

Inputs. Six states $\{B, P, R, T, D, X\}$, with D and X absorbing. Transition counts:

$$C = \begin{array}{c} \begin{array}{ccccc} & B & P & R & T & D & X \\ \begin{array}{c} B \\ P \\ R \\ T \\ D \\ X \end{array} & \begin{pmatrix} 0 & 85 & 0 & 0 & 5 & 10 \\ 0 & 0 & 80 & 5 & 0 & 15 \\ 0 & 25 & 0 & 70 & 0 & 5 \\ 0 & 10 & 15 & 0 & 70 & 5 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} \end{array} \end{array}. \quad (\text{A.10})$$

MLE estimate. With $\alpha = 0$, normalizing each row of C yields:

$$\hat{P} = \begin{pmatrix} 0 & 0.85 & 0 & 0 & 0.05 & 0.10 \\ 0 & 0 & 0.80 & 0.05 & 0 & 0.15 \\ 0 & 0.25 & 0 & 0.70 & 0 & 0.05 \\ 0 & 0.10 & 0.15 & 0 & 0.70 & 0.05 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}. \quad (\text{A.11})$$

Canonical decomposition.

$$Q = \begin{pmatrix} 0 & 0.85 & 0 & 0 \\ 0 & 0 & 0.80 & 0.05 \\ 0 & 0.25 & 0 & 0.70 \\ 0 & 0.10 & 0.15 & 0 \end{pmatrix}, \quad R = \begin{pmatrix} 0.05 & 0.10 \\ 0 & 0.15 \\ 0 & 0.05 \\ 0.70 & 0.05 \end{pmatrix}. \quad (\text{A.12})$$

Fundamental matrix. Computing $N = (I - Q)^{-1}$ in double-precision arithmetic:

$$N \approx \begin{pmatrix} 1.000 & 1.431 & 1.458 & 1.134 \\ 0 & 1.684 & 1.715 & 1.334 \\ 0 & 0.675 & 1.688 & 1.054 \\ 0 & 0.285 & 0.441 & 1.267 \end{pmatrix}. \quad (\text{A.13})$$

Expected steps to absorption.

$$\mathbf{t} = N\mathbf{1} \approx (5.023, 4.733, 3.417, 1.993)^\top. \quad (\text{A.14})$$

A work item starting in Backlog visits an expected 5.02 transient states before reaching Done or Cancelled.

Absorption probabilities.

$$B = NR \approx \begin{pmatrix} 0.844 & 0.156 \\ 0.934 & 0.066 \\ 0.738 & 0.262 \\ 0.887 & 0.113 \end{pmatrix}. \quad (\text{A.15})$$

From Backlog, the probability of completion (Done) is approximately 0.84 and of cancellation (Cancelled) approximately 0.16.

Expected lead time. With mean sojourn times (hours) $\bar{d}_B = 48, \bar{d}_P = 24, \bar{d}_R = 8, \bar{d}_T = 4$:

$$E[\text{LT} \mid s_0 = B] = 1.000 \cdot 48 + 1.431 \cdot 24 + 1.458 \cdot 8 + 1.134 \cdot 4 \approx 98.9 \text{ h}. \quad (\text{A.16})$$

This analytical estimate is the baseline against which the Monte Carlo distribution (Stage 4) is validated.



Probabilistic Metric Derivations

This appendix derives the probabilistic forecasting metrics introduced in Section 2.4 and used in the empirical evaluation (Chapter 5).

B.1 CRPS: DEFINITION AND EQUIVALENT FORMS

Definition B.1 (Continuous Ranked Probability Score). *For a forecast distribution F and a realized value $y \in \mathbb{R}$:*

$$\text{CRPS}(F, y) = \int_{-\infty}^{\infty} (F(t) - \mathbf{1}(t \geq y))^2 dt. \quad (\text{B.1})$$

Theorem B.1 (Kernel Form). *Let $X, X' \stackrel{\text{iid}}{\sim} F$. Then*

$$\text{CRPS}(F, y) = E_F|X - y| - \frac{1}{2}E_F|X - X'|. \quad (\text{B.2})$$

Proof. Expanding the integrand of Equation B.1 and applying Fubini–Tonelli to interchange integrals yields the kernel form. The first term penalizes inaccuracy (mean absolute deviation from y); the second term rewards sharpness (penalty for diffuse forecasts). See Gneiting and Raftery (??) for the full algebraic expansion. $\square$

The kernel form is the operational expression used in Monte Carlo implementations: $E_F|X - y|$ and $E_F|X - X'|$ are estimated by averaging absolute differences across simulated samples.

B.2 PROPRIETY OF CRPS

Definition B.2 (Strictly Proper Scoring Rule). *A scoring rule $S(F, y)$ is strictly proper relative to a class $\mathcal{F}$ if, for every $G \in \mathcal{F}$, $E_{Y \sim G}[S(G, Y)] \leq E_{Y \sim G}[S(F, Y)]$, with equality iff $F = G$.*

Theorem B.2 (CRPS is Strictly Proper). *CRPS is a strictly proper scoring rule on the class of distributions with finite first moment.*

Proof sketch. Decompose $E_{Y \sim G}[\text{CRPS}(F, Y)]$ using the kernel form. Comparison with $E_{Y \sim G}[\text{CRPS}(G, Y)]$ reduces to the non-negativity of an L^2 distance between the CDFs of F and G , which is zero iff $F = G$ almost everywhere. See (??), Theorem 6, for the full argument. $\square$

Implication for PATHCAST. Strict propriety guarantees that minimizing CRPS in expectation is consistent with reporting the true predictive distribution. A forecaster cannot game CRPS by reporting a miscalibrated distribution, justifying its choice as the primary metric for RQ2.

B.3 CRPS–MAE RELATIONSHIP

Theorem B.3 (Degenerate Forecast). *If $F = \delta_{\hat{y}}$ (Dirac mass at the point $\hat{y}$), then $\text{CRPS}(\delta_{\hat{y}}, y) = |y - \hat{y}|$.*

Proof. Substitute $F(t) = \mathbf{1}(t \geq \hat{y})$ in Equation B.1. The integrand $(\mathbf{1}(t \geq \hat{y}) - \mathbf{1}(t \geq y))^2$ equals 1 on the interval between y and $\hat{y}$ and 0 elsewhere. The integral equals the length of that interval, $|y - \hat{y}|$. $\square$

Operational consequence. A point forecaster (B1, B3 baselines in Section 5.5) and a distributional forecaster (PATHCAST) are directly comparable on the same scale. CRPS reduces to MAE for the point baselines and accounts for distributional sharpness for the PATHCAST simulation, eliminating the need for separate metrics.

B.4 CALIBRATION AND THE AGGREGATE CALIBRATION ERROR

Definition B.3 (Calibration). *A forecaster is calibrated if, for every $\alpha \in (0, 1)$, $P(Y \leq \hat{Q}_\alpha) = \alpha$, where $\hat{Q}_\alpha$ is the forecaster's α -quantile.*

The empirical version replaces the population probability by the observed frequency $\hat{p}_\alpha$ across the test set.

Calibration error and ACE. The pointwise calibration error is $\text{CalErr}(\alpha) = |\hat{p}_\alpha - \alpha|$. The Aggregate Calibration Error averages across decile checkpoints (Equation 2.12): $\text{ACE} = \frac{1}{|\mathcal{A}|} \sum_{\alpha \in \mathcal{A}} |\hat{p}_\alpha - \alpha|$.

Calibration vs. sharpness. A forecast may be sharp (low spread) but miscalibrated, or calibrated but diffuse. CRPS combines both; ACE isolates calibration. Reporting both prevents Goodhart-style optimization of one at the expense of the other.

B.5 WORKED NUMERICAL EXAMPLES

Example 1 (Point forecast). $\hat{F} = \delta_{10}$, observed $y = 12$. By Theorem B.3, $\text{CRPS} = |12 - 10| = 2$.

Example 2 (Empirical forecast). An MC run produces lead-time samples $\{8, 9, 10, 11, 12, 14, 15\}$; observed $y = 13$. Using the kernel form (Equation B.2): $E|X - y| = \frac{1}{7}(5 + 4 + 3 + 2 + 1 + 1 + 2) = 2.571$; $E|X - X'| \approx 2.612$ (computed across all $7^2 = 49$ pairs); $\text{CRPS} \approx 2.571 - \frac{1}{2}(2.612) = 1.265$. The distributional forecast achieves a CRPS lower than the worst point error of 5, illustrating the sharpness reward.

Example 3 (Calibration over a test set). Decile coverage on 40 test cases yields observed frequencies $\hat{p}_{0.1} = 0.08$, $\hat{p}_{0.5} = 0.55$, $\hat{p}_{0.9} = 0.85$. Pointwise errors: 0.02, 0.05, 0.05. ACE (averaged across all nine deciles) computed analogously, summarizing systematic over- or underconfidence.



Process Mining Algorithmic Details

This appendix expands the algorithmic content sketched in Section 2.1.

C.1 INDUCTIVE MINER INTERNALS

The Inductive Miner (IM) (??) discovers a process tree by recursively partitioning the directly-follows graph (DFG) extracted from the event log.

Directly-follows graph. For event log $\mathcal{L}$, the DFG is a graph $G_{\mathcal{L}} = (V, E)$ with $V = \mathcal{A}_{\mathcal{L}}$ and edge $(a, b) \in E$ iff there exist consecutive events e_k, e_{k+1} in some trace with $\alpha(e_k) = a$ and $\alpha(e_{k+1}) = b$.

Recursive partitioning. At each recursion level, IM identifies the most prominent control-flow operator in the DFG and partitions the activity set accordingly:

- **Sequence ($\rightarrow$):** $\mathcal{A}$ partitions into $\mathcal{A}_1, \mathcal{A}_2$ such that all edges from $\mathcal{A}_1$ to $\mathcal{A}_2$ exist and no reverse edges exist.
- **Exclusive choice ($\times$):** $\mathcal{A}$ partitions into disjoint subsets with no edges between them.
- **Parallel ($\wedge$):** $\mathcal{A}$ partitions into subsets with edges in both directions between every pair of subsets.
- **Loop ($\cup$):** a body subset and a redo subset, with edges from body end to redo start and from redo end to body start.

Recursion terminates at single-activity leaves or empty sequences.

IMf variant. Inductive Miner—Infrequent (IMf) filters edges with frequency below a threshold θ_{noise} before partition detection, yielding a model that ignores rare patterns while preserving soundness.

Soundness guarantee. The block-structured nature of process trees guarantees soundness: every reachable marking can reach the final marking, and the final marking is reachable. The proof reduces to structural induction over tree operators.

Complexity. The DFG construction is $O(|\mathcal{L}|)$. The recursive partitioning is $O(|\mathcal{A}|^3)$ per level with at most $|\mathcal{A}|$ levels, giving overall $O(|\mathcal{L}| + |\mathcal{A}|^3 \log |\mathcal{A}|)$.

C.2 ALIGNMENT-BASED CONFORMANCE CHECKING

Synchronous product. Given trace σ and model M , the synchronous product is a Petri net (or transition system) whose transitions are labeled with the moves of Definition 2.2. The cost function assigns 0 to synchronous moves and a positive cost (typically 1) to log moves and model moves.

A^* search. The optimal alignment γ^* is the minimum-cost path from the initial marking to a final marking. A^* expands states in order of $f(n) = g(n) + h(n)$, where $g(n)$ is the accumulated cost and $h(n)$ is an admissible heuristic (a lower bound on the remaining cost). A typical heuristic is the number of remaining non-synchronous moves implied by the unmatched portion of the trace.

Complexity. Alignment is NP -hard in the general case; efficient implementations (PM4Py, ProM) handle SDL logs in practice because the state spaces are small (typically $|S| \leq 20$).

C.3 FITNESS, PRECISION, AND GENERALIZATION

Fitness. As stated in Equation 2.1, fitness is $1 - \text{cost}(\gamma^*)/\text{cost}(\gamma_{\text{worst}})$, where γ_{worst} is the alignment with all moves non-synchronous (cost equal to $|\sigma| + |M_{\min}|$, where $|M_{\min}|$ is the length of the shortest model trace).

ETC precision. Escaping Edges precision counts model states where transitions allowed by the model were never observed in the log. For each state s visited during alignment, let $\text{en}_M(s)$ be the set of model-allowed activities and $\text{en}_\mathcal{L}(s)$ the set of log-observed activities. Then

$$\text{Precision}(M, \mathcal{L}) = 1 - \frac{\sum_s w_s \cdot |\text{en}_M(s) \setminus \text{en}_\mathcal{L}(s)|}{\sum_s w_s \cdot |\text{en}_M(s)|}, \quad (\text{C.1})$$

where w_s is the visit frequency of s .

Generalization. Quantifies the extent to which the model captures behavior beyond observed traces. A common formulation uses the expected fitness on cases generated by a hold-out replication of the underlying process; the empirical estimator weighs states by visit frequency, penalizing states visited by many distinct traces less than states visited by few.

C.4 WORKED EXAMPLE

Toy log. Three traces over four activities: $\sigma_1 = \langle A, B, C, D \rangle$, $\sigma_2 = \langle A, B, C, D \rangle$, $\sigma_3 = \langle A, C, B, D \rangle$.

DFG. Edges $A \rightarrow B$ (count 2), $A \rightarrow C$ (count 1), $B \rightarrow C$ (count 2), $C \rightarrow B$ (count 1), $C \rightarrow D$ (count 2), $B \rightarrow D$ (count 1).

Discovered process tree. IM identifies A as a sequence predecessor to $\{B, C\}$ and D as a sequence successor of $\{B, C\}$, with $\{B, C\}$ in parallel ($B \parallel C$). Resulting tree: $\rightarrow (A, \wedge(B, C), D)$.

Quality metrics. Replaying the three traces against the tree yields fitness = 1.0 (all traces are valid sequences of the parallel construct); precision ≈ 0.5 (the parallel construct admits two permutations of B, C , but only two of them are observed); generalization ≈ 0.7 (depends on weighting; full computation in PM4Py).



End-to-End Walkthrough Example

This appendix presents a complete worked example of the PATHCAST pipeline on a small synthetic project, making the data-flow concrete from raw events through probabilistic forecast.

D.1 SYNTHETIC EVENT LOG

The synthetic dataset contains 10 issues across 35 days, with events from three sources (Jira, Git, GitHub Actions). Issue keys: PROJ-1 to PROJ-10. Total raw events: 58.

Raw events (excerpt for PROJ-1).

Source	Type	Timestamp
Jira	status_change (Backlog → In Progress)	day 1, 09:00
Git	commit (msg: "PROJ-1 initial impl")	day 1, 14:30
Jira	status_change (In Progress → Review)	day 2, 11:00
PR	pr_opened (PROJ-1)	day 2, 11:05
Git	commit (msg: "PROJ-1 review fix")	day 3, 10:00
PR	pr_merged	day 3, 16:00
Jira	status_change (Review → Testing)	day 4, 09:30
CI	build_end (status: pass)	day 4, 09:45
Jira	status_change (Testing → Done)	day 5, 14:00

Case correlation. The cascade $\kappa_1 \circ \kappa_2 \circ \kappa_3$ assigns 56 of 58 events to one of the 10 issues; 2 unrelated commits are discarded. Coverage = $56/58 = 0.966$.

Activity mapping. The mapping μ projects raw events onto $\mathcal{A} = \{\text{backlog}, \text{in_progress}, \text{review}\}$. Build events are absorbed into the surrounding state to avoid spurious transitions.

D.2 PROCESS DISCOVERY

Inductive Miner with $\theta_{\text{noise}} = 0.2$ produces the process tree $\rightarrow (\text{backlog}, \text{in_progress}, \text{review})$ with an optional rework loop between review and in_progress.

Quality metrics on the synthetic log.

- Fitness: 0.93 (one trace contains an unmatched review $\rightarrow$ testing loop)
- Precision: 0.81
- Generalization: 0.74

D.3 MARKOV ESTIMATION

Transition counts (transient block).

$$C_T = \begin{matrix} & \begin{matrix} \text{IP} & \text{R} & \text{T} \end{matrix} \\ \begin{matrix} \text{IP} \\ \text{R} \\ \text{T} \end{matrix} & \begin{pmatrix} 0 & 11 & 0 \\ 2 & 0 & 9 \\ 1 & 0 & 0 \end{pmatrix} \end{matrix}.$$

MLE transition matrix.

$$\hat{Q} = \begin{pmatrix} 0 & 1.000 & 0 \\ 0.182 & 0 & 0.818 \\ 1.000 & 0 & 0 \end{pmatrix}, \quad \hat{R} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}_{\rightarrow \text{done}}.$$

Fundamental matrix. $N = (I - \hat{Q})^{-1} \approx \begin{pmatrix} 1.22 & 1.22 & 1.00 \\ 0.22 & 1.22 & 1.00 \\ 1.22 & 1.22 & 1.00 \end{pmatrix}.$

Expected steps to absorption. $\mathbf{t} = N\mathbf{1} \approx (3.44, 2.44, 3.44)^\top$.

Empirical sojourn distributions. Per state, in days: $\hat{F}_{IP} = \{1.0, 1.5, 2.0, 2.0, 3.0\}$, $\hat{F}_R = \{0.5, 1.0, 1.0, 2.0\}$, $\hat{F}_T = \{0.5, 0.8, 1.5\}$.

D.4 MONTE CARLO SIMULATION

Run $M_{\text{sim}} = 10,000$ trajectories starting from `in_progress`, sampling sojourns from $\hat{F}_s$ and next states from $\hat{Q}$.

Forecasts.

- Median lead time $\hat{Q}_{0.5} \approx 5.4$ days
- P85 forecast $\hat{Q}_{0.85} \approx 7.9$ days
- P95 forecast $\hat{Q}_{0.95} \approx 9.7$ days
- Completion probability (Done): 1.00 (no Cancelled in this synthetic example)

Top-3 paths.

- $IP \rightarrow R \rightarrow T \rightarrow \text{Done}$ (frequency 0.81)
- $IP \rightarrow R \rightarrow IP \rightarrow R \rightarrow T \rightarrow \text{Done}$ (frequency 0.15)
- Longer rework paths (frequency 0.04)

D.5 CROSS-VALIDATION AGAINST ANALYTICAL RESULTS

Mean lead time, computed analytically with mean sojourns $\bar{T}_{IP} = 1.9$, $\bar{T}_R = 1.125$, $\bar{T}_T = 0.93$:

$$E[\text{LT} \mid s_0 = IP] = 1.22 \cdot 1.9 + 1.22 \cdot 1.125 + 1.00 \cdot 0.93 \approx 4.62 \text{ days.} \quad (\text{D.1})$$

Monte Carlo mean: 4.59 days. Relative deviation: 0.6%, confirming consistency between analytical and simulation paths.

D.6 ML FEATURE CONSTRUCTION

For each of the 10 cases, the 15 features defined in Table 4.4 are constructed at the half-trace observation point. Random Forest with 5-fold temporal cross-validation yields:

- AUC-ROC (PM-only features): 0.79
- AUC-ROC (MSR-only features): 0.71
- AUC-ROC (Combined): 0.84

SHAP analysis ranks Markov-expected remaining transitions and rework count among the top three predictors, illustrating the predictive value of process-derived features hypothesized in Section 2.5 and formally evaluated in Chapter 7.

Disclaimer. Numbers above are illustrative for a synthetic $n = 10$ dataset. Statistical significance and external validity are not established at this scale; the example serves to demonstrate the end-to-end data flow, not to evidence empirical claims.



Search Protocol: Search Strings

This appendix presents the four search strings executed across the five digital libraries during the search phase of the PATHCAST SLR. Each string was adapted to the syntax and field conventions of the target database; the block structure (P and I) is preserved across all adaptations. Strings S1–S3 correspond to the principal, complementary (MSR), and frontier (stochastic $\wedge$ PM) blocks, respectively; S4 is a supplementary string executed in Scopus only, targeting Markov models in software testing.

E.1 SEARCH STRING SUMMARY BY DATABASE

Table E.1: Search strings by database and field scope.

Database	Strings	Field / syntax notes
Scopus	S1, S2, S3, S4	TITLE-ABS-KEY(...); DOCTYPE(ar OR cp); LANGUAGE(english); years 1994–2026
IEEE Xplore	S1 (split)	All Metadata; boolean AND of sub-blocks; Conference + Journals; 1994–2026
Web of Sci.	S1, S2, S3	Topic TS=(...); Article + Proceedings; English; 1994– 2026
SpringerLink	S1 (simplified)	Title + Abstract; 8 separate exports; Chapter + Article; English
ACM DL	S1, S2	Abstract; 2 queries; date 1994–2026

E.2 STRING S1 — PRINCIPAL ($P \wedge I$)

Block P (software engineering context):

*("software engineering" OR "software development" OR "software process"
OR "software lifecycle" OR "software life cycle" OR "SDLC" OR "development
lifecycle" OR "development life cycle" OR "DevOps" OR "CI/CD" OR
"continuous integration" OR "continuous delivery" OR "agile development"
OR "agile process" OR "issue tracking" OR "bug tracking" OR "version
control" OR "code review" OR "pull request" OR "software project"
OR "software repository" OR "software maintenance" OR "software
evolution")*

Block I (techniques):

*("process mining" OR "process discovery" OR "conformance checking"
OR "workflow mining" OR "predictive process monitoring" OR "process
prediction" OR "remaining time prediction" OR "process forecasting"
OR "event log" OR "process simulation" OR "Monte Carlo simulation"
OR "Markov chain" OR "Markov model" OR "stochastic process model"
OR "transition probability" OR "absorbing chain" OR "lead time prediction"
OR "cycle time prediction")*

E.3 STRING S2 — COMPLEMENTARY ($MSR \rightarrow PROCESS$
MODELLING)

*("mining software repositories" OR "software repository mining"
OR "GitHub mining" OR "Jira mining") AND ("process model" OR "Markov"
OR "Monte Carlo" OR "stochastic" OR "lead time" OR "cycle time"
OR "throughput" OR "transition matrix")*

E.4 STRING S3 — FRONTIER ($STOCHASTIC \wedge PM \wedge SE$)

*("Monte Carlo" OR "Markov chain" OR "stochastic simulation" OR "probabilistic
forecast" OR "transition matrix") AND ("process mining" OR "event
log" OR "workflow mining" OR "process model" OR "business process")
AND ("software development" OR "software engineering" OR "software*

*process" OR "DevOps" OR "SDLC" OR "issue tracker" OR "commit log"
OR "build pipeline")*

E.5 STRING S4 — MARKOV IN SOFTWARE TESTING (SCOPUS SUPPLEMENT)

*("Markov chain" OR "Markov model" OR "hidden Markov" OR "stochastic
automata") AND ("software testing" OR "test coverage" OR "software
reliability" OR "defect prediction" OR "fault prediction")*



LLM Screening Prompts

The review employed the *claude-haiku-4-5-20251001* model as the primary screener in two phases: title/abstract (T/A) and full text (FT). Each phase uses a *system prompt* that encodes the scope, criteria, and output schema, combined with a per-paper *user prompt* that supplies bibliographic data. All prompts are versioned in `config/screening_criteria.py` in the review repository.

F.1 PHASE 1: TITLE/ABSTRACT SCREENING

SYSTEM PROMPT (T/A PHASE)

You are an expert in systematic literature review (SLR) in the area of process mining and software engineering. Your task is to perform the title/abstract (T/A) screening for SLR PATHCAST.

SLR PATHCAST scope: *Intersection of (1) process mining, (2) stochastic modelling, (3) forecasting/prediction, and (4) software development processes. Focus: studies that analyse, discover, or predict software process characteristics using event logs, repositories, or stochastic models.*

INCLUSION CRITERIA *(at least ONE must be satisfied to include):*

IC1 - Process Mining in Software: applies PM (discovery, conformance, workflow mining, event log analysis, PPM) to SW artefacts (commits, issues, PRs, CI/CD). IC2 - Stochastic Modelling of SW Processes: uses Markov chains, Monte Carlo, stochastic Petri nets, or transition

matrices to model SW dev processes. IC3 - Process Metric Forecasting: predicts lead time, cycle time, remaining time, throughput, or defect rates in SW processes using data or event logs. IC4 - Repository Mining for Process: mines repositories (GitHub, Jira, VCS) to discover or improve SW development process models.

EXCLUSION CRITERIA (any ONE is sufficient to exclude): EC1 - Non-SW domain: applied to healthcare, manufacturing, finance, etc., with no link to SW processes. EC2 - SW as tool: "software" is the implementation platform, not the process under study. EC3 - Algorithm without SW application: purely theoretical method, no evaluation in SW processes. EC4 - Out-of-scope secondary study: survey/SLR not specifically addressing PM or stochastic methods in SE.

UNCERTAIN CASES (maybe): Relevant title without abstract; generic BPM article with possible SW application; ambiguous terminology.

STRUCTURED FIELDS (use only the allowed enums): evidence_tags, software_context, stochastic_method, forecast_target, process_data_source, confidence.

Respond ONLY with valid JSON, no additional text.

USER PROMPT TEMPLATE (T/A PHASE)

Evaluate the paper below for SLR PATHCAST.

Title: {title} **Abstract:** {abstract} **Venue/Type:** {venue} | {doc_type} | {year} **Source database:** {source_db} **Abstract origin:** {abstract_source}

Respond with JSON in the exact format: { "decision": "include" | "exclude" | "maybe", "rationale": "<1-2 sentences justifying the decision>", "matched_ic": ["IC1", "IC2"], "matched_ec": ["EC1"], "evidence_tags": ["process_mining", "forecasting"], "software_context": "software_development_process", "stochastic_method": "markov_chain", "forecast_target": "remaining_time", "process_data_source": "event_logs", "confidence": "medium" }

Rules: - "include" if ≥ 1 IC met and no decisive EC - "exclude" if an EC clearly applies, or paper clearly meets no IC - "maybe" if genuine doubt (title without abstract, domain ambiguity) - matched_ic and matched_ec must be [] when not applicable

F.2 PHASE 2: FULL-TEXT SCREENING

SYSTEM PROMPT (FT PHASE)

The FT-phase system prompt is identical to the T/A prompt, augmented with three instructions that encode the stricter full-text screening pattern:

THIS IS THE FULL-TEXT PHASE - apply criteria rigorously: - If the abstract clearly confirms ≥ 1 IC with no decisive EC $\rightarrow$ "include"
- If the abstract clearly confirms no IC applies, or an EC applies $\rightarrow$ "exclude"
- Use "pending" ONLY if the abstract is genuinely insufficient for a decision (absent or extremely short) AND the title suggests marginal relevance
When in doubt between include and exclude, prefer exclude - this phase requires confidence.
YOU MUST NEVER invent, complete, or infer a missing abstract.

USER PROMPT TEMPLATE (FT PHASE)

The FT user prompt extends the T/A template with a **T/A context block** that supplies the prior decision, rationale, and ICs identified in phase 1:

Evaluate the paper below for SLR PATHCAST - FULL-TEXT screening (phase 2).

Title: {title} **Abstract:** {abstract} **Venue/Type:** {venue} | {doc_type} | {year} **Source database:** {source_db} **Abstract origin:** {abstract_source}

T/A phase context: - Previous decision: {ta_decision} - Previous rationale: {ta_rationale} - ICs identified in phase 1: {ta_matched_ic}

Re-evaluate rigorously for the full-text phase. Respond with JSON in the exact format: { "decision": "include" | "exclude" | "pending", [... same fields as T/A phase ...] }

F.3 PHASE 2B: SCREEN-BLANKS PASS

A third pass was applied to the 186 papers without a decision after the two prior phases (mainly Band E papers with no abstract available). The *screen-blanks*

pass reuses the same FT system and user prompts, with the abstract field set to “Abstract not available” and the T/A context block showing `ta_decision = maybe` and `ta_matched_ic = none`. Papers for which the LLM returned `pending` and whose title analysis indicated low relevance were closed under EC5 (full text inaccessible; $n = 148$).

F.4 MODEL PARAMETERS BY PHASE

Table F.1 summarises the model parameters used in the three screening phases and in data extraction.

Table F.1: LLM parameters in each screening phase.

Phase	Model	Max tok.	Batch	API
T/A screening	claude-haiku-4-5-20251001	512	500	Anthropic Batches
FT screening	claude-haiku-4-5-20251001	512	100	Anthropic Batches
Screen-blanks	claude-haiku-4-5-20251001	512	186	Anthropic Batches
Data extraction	claude-haiku-4-5-20251001	1024	100	Anthropic Batches
Synthesis (ch. 3)	claude-sonnet-4-6	2048	—	Anthropic Messages



Data Extraction Form

Data extraction for the 169 included studies was performed with the *claude-haiku-4-5-20251001* model via the Anthropic Batches API. Papers with a locally available PDF had text pre-extracted with *pdfplumber* (first 8 pages, maximum 6,000 characters). Papers without a PDF used the enriched abstract from Semantic Scholar. The prompt below was submitted once per paper.

G.1 EXTRACTION PROMPT

You are a researcher performing data extraction for SLR PATHCAST on Process Mining and Stochastic Modelling in Software Processes.

Title: {title} **Authors:** {authors} **Year:** {year} | **Venue:** {venue}
Inclusion criteria met: {ics} {content_block}

Return ONLY a valid JSON object with exactly these fields:

```
{ "research_question": "main research question (1-2 sentences)",  
  "study_type": "case_study | experiment | survey | tool | theoretical  
| simulation | mixed", "research_contribution": "discovery | conformance  
| enhancement | prediction | simulation | framework | hybrid", "pm_technique":  
"alpha | inductive_miner | heuristic | conformance_checking | social_network  
| declarative | other | none", "stochastic_technique": "markov_chain  
| stochastic_petri_net | monte_carlo | system_dynamics | other |  
none", "software_artifact": "commits | issues | ci_cd | ide_logs  
| vcs | jira | github | other | none", "software_process": "development  
| testing | maintenance | code_review | bug_fixing | deployment
```

```
| requirements | other", "dataset_source": "open_source | industrial
| academic | synthetic | not_specified", "dataset_public": "yes
| no | partial | not_mentioned", "tool_used": "tool names (e.g.
ProM, pm4py, Disco) or not_mentioned", "main_finding": "main result/contribution
(2-3 sentences)", "limitations": "reported limitations (1-2 sentences)
or not_mentioned", "replication_package": "yes | no | partial |
not_mentioned" }
```

Respond ONLY with JSON, no additional text.

G.2 DATA EXTRACTION FORM FIELDS

Table G.1 describes all fields in the structured extraction form stored in `extraction_template.csv`.

G.3 CONTROLLED VOCABULARY (ENUMS) FOR STRUCTURED OUTPUT

Table G.2 defines the full controlled vocabulary for structured fields returned by the LLM in both screening phases. These enums are versioned in `config/screening_criteria.py` and were not changed between screening rounds, ensuring consistency.

Table G.1: Data extraction form: all fields in `extraction_template.csv`.

Field	Description	Allowed values / format
<i>Identification</i>		
<code>internal_id</code>	8-char hash (dedup key)	—
<code>title</code>	Paper title	free text
<code>doi</code>	Digital identifier	URI
<code>year</code>	Publication year	YYYY
<code>source_db</code>	Source database	scopus, ieee, acm, springer, wos, snowball
<code>ft_matched_ic</code>	Inclusion criteria met	IC1 IC2 IC3 IC4
<code>ft_screened_by</code>	FT decision responsible	llm, manual, llm_pdf
<i>Enriched metadata (Semantic Scholar)</i>		
<code>authors</code>	Author list	semicolon-separated
<code>venue</code>	Publication venue	free text
<code>journal_name</code>	Journal/conference name	free text
<code>abstract</code>	Paper abstract	free text
<code>pdf_available</code>	PDF available locally	yes / no
<i>LLM-extracted fields</i>		
<code>research_question</code>	Main RQ	1–2 sentences
<code>study_type</code>	Study type	case_study experiment survey tool theoretical simulation mixed
<code>research_contribution</code>	Contribution type	discovery conformance enhancement prediction simulation framework hybrid
<code>pm_technique</code>	PM algorithm used	alpha inductive_miner heuristic conformance_checking social_network declarative other none
<code>stochastic_technique</code>	Stochastic method	markov_chain stochastic_petri_net monte_carlo system_dynamics other none
<code>software_artifact</code>	Data source	commits issues ci_cd ide_logs vcs jira github other none
<code>software_process</code>	SDLC phase	development testing maintenance code_review bug_fixing deployment requirements other
<code>dataset_source</code>	Dataset provenance	open_source industrial academic synthetic not_specified
<code>dataset_public</code>	Data availability ¹²¹	yes no partial not_mentioned
<code>tool_used</code>	Tools/platforms	free text
<code>main_finding</code>	Main result	2–3 sentences
<code>limitations</code>	Study limitations	1–2 sentences

Table G.2: Controlled vocabulary for LLM structured output.

Field	Allowed values
decision (T/A)	include, exclude, maybe
decision (TC)	include, exclude, pending
software_context	software_development_process, repository_mining, ci_cd, issue_bug_workflow, software_testing, requirements_engineering, software_project_management, unclear, not_software_process
stochastic_method	none, markov_chain, hidden_markov_model, monte_carlo, stochastic_petri_net, bayesian_model, probabilistic_model, simulation, queueing_model, other_stochastic, unclear
forecast_target	none, lead_time, cycle_time, remaining_time, throughput, defect_rate, build_outcome, reliability, completion_time, other_process_metric, unclear
process_data_source	none, event_logs, version_control, issue_tracker, pull_requests, ci_cd_logs, software_repository_mixed, synthetic_data, simulated_process, survey_or_secondary, unclear
confidence	low, medium, high
evidence_tags	process_mining, software_process, repository_mining, stochastic_modeling, forecasting, event_log, version_control, issue_tracking, pull_requests, ci_cd, markov, hidden_markov_model, monte_carlo, stochastic_petri_net, bayesian_model, simulation, lead_time, cycle_time, remaining_time, throughput, defect_prediction, build_prediction, reliability, insufficient_abstract, full_text_required



Assessment and Classification Instruments

This appendix gathers the instruments used to classify, enrich, and assess papers throughout screening and selection: the priority-band taxonomy, the abstract-enrichment algorithm, the quality-assessment checklist, the control paper set, and the SPMF integration-level taxonomy.

H.1 PRIORITY BANDS FOR THE FULL-TEXT QUEUE

Before full-text screening, the queue of 886 papers was stratified into five priority bands (A–E) to optimise resource allocation. Abstract enrichment and LLM screening were applied in band order. Table H.1 defines the classification rules.

Band D papers with a DOI were prioritised for manual PDF recovery. Band E papers without a DOI or abstract were closed as EC5 (full text inaccessible) at the end of the *screen-blanks* pass.

H.2 ABSTRACT ENRICHMENT ALGORITHM

To increase the number of papers assessable by the LLM, a four-source cascade was executed before full-text screening. Algorithm H.1 describes the enrichment procedure.

Table H.1: Priority-band definitions for the full-text review queue.

Band	<i>n</i>	Classification rule
A	95	T/A decision = include and ≥ 2 distinct ICs met
B	20	T/A decision = include and exactly 1 IC met
C	39	T/A decision = maybe and abstract available
D	152	T/A decision = maybe, at least 1 IC signal in title, no abstract
E	584	T/A decision = maybe, no IC signal in title, no abstract
Total	886	

Algorithm 1 Abstract Enrichment Cascade**Input:** Record p with doi and/or title**Output:** p updated with abstract, authors, venue

1. If p .abstract is non-empty $\rightarrow$ **return** p
2. If p .doi is present:
 - 2a. Query **Semantic Scholar** Batch API (DOI:{doi})
If found: update abstract, authors, venue; **return**
 - 2b. Query **OpenAlex** (filter=doi:{doi})
If found: update abstract; **return**
 - 2c. Query **CORE** (/search/works?q=doi:{doi})
If found: update abstract; **return**
 - 2d. Query **Crossref** (/works/{doi})
If abstract present: update; **return**
3. If p .doi is absent:
 - 3a. Query **Semantic Scholar** by title (fuzzy)
If similarity ≥ 0.85 : update; **return**
4. Set p .abstract_source = missing_or_unverified

Figure H.1: Abstract enrichment cascade applied to the 886-paper FT queue.

The cascade raised abstract coverage from 16.6% (147/886) to 72.6% (643/886) before Round 2 screening. Of the 643 enriched abstracts, 590 were obtained from Semantic Scholar, 31 from OpenAlex, 13 from CORE, and 9 from Crossref or the title fallback. Table H.2 summarises results by band.

Table H.2: Abstract enrichment results by priority band.

Band	Papers	With abstract (pre)	Obtained	Coverage (post)
A	95	91 (95.8%)	4	95 (100.0%)
B	20	18 (90.0%)	2	20 (100.0%)
C	39	39 (100%)	0	39 (100.0%)
D	152	0 (0.0%)	118	118 (77.6%)
E	584	0 (0.0%)	371	371 (63.5%)
Total	886	148 (16.7%)	495	643 (72.6%)

H.3 QUALITY ASSESSMENT CHECKLIST

The eight-item checklist (Table H.3) is applied to each included primary study during manual reading. Items QA1–QA4 assess methodological rigour; QA5–QA7 assess empirical robustness and reproducibility; QA8 specifically checks reporting of process-model quality, which is central to the PATHCAST evaluation strategy. Studies scoring below 4/8 are excluded from synthesis.

Current QA status: 76 of 169 studies assessed (complete Round 1 set); mean = 5.18 (SD = 1.13), median = 5.0 (IQR: 4–6); 5 studies scored below the threshold and were excluded from synthesis. The remaining 93 studies (Rounds 2+) have QA assessment pending.

H.4 CONTROL PAPER SET

The set of 12 control papers was used to validate search strings before execution. Table H.4 lists all control papers with their IC profile and the source by which each was captured.

Papers V4, V5, V8, and V9 were not in the initial working set but were recovered via *snowballing* during the FT phase and are included in the final corpus.

Table H.3: Quality assessment checklist with scoring guide.

ID	Criterion	Scoring guide
QA1	Are the research objectives clearly stated?	Score 1 if RQs or hypotheses are explicitly enumerated; 0 otherwise.
QA2	Is the software engineering context described in sufficient detail?	Score 1 if SDLC phase, artefact type, and process scope are defined.
QA3	Is the data source (event log or repository) described reproducibly?	Score 1 if dataset origin, collection method, and key statistics are provided.
QA4	Is the PM or stochastic technique formally defined?	Score 1 if algorithm, parameters, and configuration are specified.
QA5	Are results validated empirically?	Score 1 if validation uses real or semi-real data; 0 for pure simulation or no validation.
QA6	Are threats to validity discussed?	Score 1 if at least internal and external validity are addressed.
QA7	Is the study reproducible (data and/or code available)?	Score 1 if a replication package or open dataset is described or referenced.
QA8	Are process-model quality metrics reported (fitness, precision)?	Score 1 if at least one formal quality metric is quantified.
Score range		0–8 (threshold: ≥ 4)

Table H.4: Complete control-paper set for string validation.

#	Paper	Rationale	ICs	Captured via
V1	Cook & Wolf (1998) (??)	Seminal PM + SE	IC1, IC2	ACM
V2	Rubin et al. (2007) (??)	First named PM framework for SW	IC1	Scopus
V3	Poncin et al. (2011) (??)	PM + MSR intersection	IC1, IC4	IEEE
V4	Caldeira & Cardoso (2019)	PM + SW process assessment	IC1	IEEE
V5	Lemos et al. (2011)	PM for SW management	IC1	IEEE
V6	Whittaker & Thomason (1994) (??)	Markov + SW testing (seminal)	IC2	Scopus
V7	Tax et al. (2017) (??)	Seminal PPM (BPM baseline)	IC3	Control
V8	Rogge-Solti & Weske (2015)	Stochastic + PM forecasting	IC2, IC3	Scopus
V9	Rozinat et al. (2009)	PM → Simulation	IC1, IC2	Control
V10	Gupta (2017) (??)	Multi-repository SE PM	IC1, IC3	ACM
V11	Jalote & Kamma (2021) (??)	IDE logs + Markov chains	IC1, IC2	Scopus
V12	Ortu et al. (2023) (??)	GitHub Markov + SNA	IC2, IC4	Scopus
Capture rate from primary queries only				10/12 (83.3%)
Capture rate including snowballing				12/12 (100%)

H.5 SPMF INTEGRATION-LEVEL TAXONOMY

The *Software Process Mining and Forecasting* (SPMF) taxonomy, used throughout Chapter 3, classifies studies along three orthogonal dimensions (Table H.5).

Table H.5: SPMF taxonomy: three dimensions with level definitions.

Dimension	Level	Definition
Analytical Depth	AD1 – Descriptive	As-is model discovery; conformance checking
	AD2 – Diagnostic	Bottleneck detection; rework quantification; root-cause analysis
	AD3 – Predictive	Outcome prediction; remaining-time estimation
	AD4 – Forecasting	Distributional forecasts; probabilistic completion estimates
SDLC Coverage	SC-R	Requirements and design
	SC-D	Development (coding, commits, pull requests)
	SC-Q	Quality (testing, reviews, defects)
	SC-C	CI/CD and delivery
	SC-O	Operations and maintenance
Technique Integration	TI-L0	Single technique (PM, ML, or stochastic in isolation)
	TI-L1	Two techniques in the same study without an explicit pipeline contract
	TI-L2	Partial bridge: two techniques connected, but no ML closure or contract guarantee
	TI-L3	Unified pipeline: PM → Markov → MC → ML with formal inter-stage contracts

The L0–L3 scale on the TI dimension corresponds to the integration-level code used in Sections 3.4.7 and I.2. PATHCAST is positioned at TI-L3, AD4, with SC coverage spanning SC-D, SC-Q, and SC-C.



Reading List and Pipeline Artefacts

This appendix presents the ranked list of 32 primary studies selected for in-depth synthesis and documents the persistent scripts and artefacts of the review pipeline.

I.1 MULTI-CRITERION SCORING RUBRIC

Table I.1 presents the 15-point rubric used to rank studies. Three seminal papers (Cook 1995, Cook 1998, Rubin 2007) are included regardless of score.

Table I.1: Multi-criterion scoring rubric for the top-32 reading list.

Criterion	Points
IC2 present (stochastic modelling in SE)	+3
IC3 present (process metric forecasting)	+3
IC1 present (process mining in SE)	+2
IC4 present (repository mining for process)	+1
PDF available locally	+2
Published in 2020 or later	+1
High-impact venue (IEEE/ACM/IST/JSS/IS/TSE)	+1
Empirical type (case_study or experiment)	+1
Contribution = prediction or simulation	+1
Maximum score	15

I.2 TOP-32: RANKED READING LIST

I.3 REVIEW PIPELINE SCRIPTS

The review was conducted with a custom Python pipeline (`pipeline/`) operating on structured CSV storage. Table I.3 documents each script with its function and main invocation command.

I.4 PERSISTENT ARTEFACT REGISTRY

Table I.4 lists persistent artefacts produced by the review, with their role, location, and current status.

Table I.2: Top-32 reading list with score, year, IC cluster, and summary of the main finding. Papers marked with † are forced inclusions due to seminal status.

Rank	Pts	Authors (year)	ICs	PDF	Main finding (summary)
1	14	Joshi & Kahani (2024) (??)	IC2,IC3,IC4	✓	RL/MDP for GitHub PR outcome prediction
2	14	Li et al. (2020) (??)	IC2,IC3	✓	Hybrid SD+DES simulation for industrial forecasting
3	13	Guinea-Cabrera et al. (2025) (??)	IC1,IC2,IC3	✓	PRIMAD: Digital Twin for SDLC (pm4py+Akka+LightGBM), L2
4	13	Ortu et al. (2023) (??)	IC2,IC4	✓	Markov+SNA for fault insertion/correction patterns (Apache)
5	13	Ben Mesmia et al. (2021) (??)	IC2,IC3	✓	Non-Markovian SPN for DevOps duration prediction
6	12	Buliga et al. (2025) (??)	IC1,IC3	✓	Generative AI + process simulation for what-if scenarios
7	12	Bhadra et al. (2023) (??)	IC2,IC3	✓	CI/CD SPN with reliability sensitivity analysis
8	12	Bhadra (2022) (??)	IC2,IC3	✓	CI/CD pipeline SPN for delivery probability
9	12	Massitela et al. (2018) (??)	IC2,IC3	✓	Stochastic Automata Network for Waterfall estimation
10	11	Incerto et al. (2025) (??)	IC1,IC2	✓	VLMC for stochastic conformance; outperforms 18 techniques
11	11	Jo & Kwon (2024) (??)	IC2,IC4	✓	Probabilistic model checking (CTL) on GitHub repositories
12	11	Jo et al. (2023) (??)	IC2,IC4	✓	DTMC for GitHub collaboration patterns (5 repos)
13	11	López-Pintado & Dumas (2023) (??)	IC1,IC2	—	Probabilistic resource calendars discovered from logs
14	11	Caldeira et al. (2022) (??)	IC1,IC3	✓	Process complexity → cyclo-matic complexity ($r \approx 0.43$)
15	11	Jalote & Kamma (2021) (??)	IC1,IC2	✓	Markov chains on IDE logs for programmer productivity
16	11	Lunesu et al. (2021) (??)	IC2,IC3	✓	Monte Carlo for risk assessment in agile development
17	11	Nafreen et al. (2020) (??)	IC2,IC3	—	SRGM + defect tracking for hybrid forecasting
18	11	Gupta (2017) (??)	IC1,IC3,IC4	—	Inductive miner + predictive analytics for SW maintenance
19	10	Salmani et al. (2022)	IC1,IC3	✓	Process mining for requirements engineering in MDPs
20	10	Jeon et al. (2015) (??)	IC2,IC3,IC4	—	Probabilistic risk prediction via Markov on repositories
21	10	Tian & Noore (2005) (??)	IC2,IC3	—	Markov modelling of defect removal under code churn
22	10	Cook & Wolf (1995) [†] (??)	IC1,IC2	✓	First automated process discovery from traces
23	9	Pourbafrani et al. (2021) (??)	IC1,IC3	✓	SIMPT: process tree + what-if simulation tool
24	9	Tyagi et al. (2021) (??)	IC2,IC3	—	Markov state-space framework for agile planning

Table I.3: Review pipeline scripts and their functions.

Script	Function	Main command
pipeline/dedup.py	Cross-source deduplication (DOI + fuzzy title)	python main.py dedup
pipeline/enrich.py	Abstract enrichment cascade (4 sources)	python main.py enrich
pipeline/screening.py	LLM T/A screening via Batches API	python main.py screen --run
pipeline/fulltext.py	LLM FT screening + manual review	python main.py fulltext --run
pipeline/pdf_download.py	PDF recovery (Unpaywall, S2, OA)	python main.py pdf
pipeline/extract_prep.py	Copy for extraction	python pipeline/extract_prep.py
pipeline/extract_llm.py	Structured data extraction (LLM, 169 papers)	python pipeline/extract_llm.py --run
pipeline/synth_llm.py	Synthesis paragraph generation per IC cluster	python pipeline/synth_llm.py
pipeline/finalization.py	PRISMA snapshot, EC5 closure, export	python main.py finalize

Table I.4: Persistent review artefacts.

File	Role	Status
results/screening/ft_screening_decision_log.csv	Decision log (886 papers)	Final (no blanks)
results/extraction/extraction_data.csv	Extraction data (169 papers)	169/169 main_finding
results/final_review/included_included_current.csv	Included in current	169 rows
results/final_review/top30_ranking_ranking.csv	Ranking leading list	32 rows
results/final_review/cap3_synthesis_synthesis_paragraphs	LLM synthesis paragraphs	For review
results/final_review/missing_bibtex_for_s2_bibtex_keys	Bibtex for S2 missing keys	Ready to merge
results/final_review/pending_for_closure_for_closure.csv	For closure or closure	118 rows
results/final_review/prisma_snapshot_snapshot.txt	PRISMA snapshot	Final
results/SLR_EXECUCAO_ATE_AGORA_execution_log	Execution log (all sessions)	Updated



Evaluation Dataset: Repository List

This appendix lists the 48 open-source repositories selected for the empirical evaluation described in Chapter 5 (Section 5.2). Selection followed the stratified sampling protocol of Section 5.2.2: 150 candidates from SEART GitHub Search (??), criterion verification via the GitHub REST API (??), and stratification across primary language, project scale, governance model, and workflow maturity (Table 5.3).

Forty-four repositories had completed data collection at the time of writing; the remaining four (marked *Pending*) complete the stratification targets and are scheduled for collection prior to Chapter 7. Apache and Spring projects use Jira issue transition history from Montgomery et al. (??) in place of GitHub Issues where noted.

J.1 STRATIFICATION SUMMARY

J.2 COMPLETE REPOSITORY LIST

Table J.1: Repository count by primary language stratum ($n = 48$).

Primary language	Count	Issue source mix
Python	11	GitHub Issues
JavaScript/TypeScript	11	GitHub Issues
Java	7	Jira (Montgomery et al.)
Go	7	GitHub Issues
Rust	5	GitHub Issues
C/C++	7	GitHub Issues
Total	48	41 GitHub; 7 Jira

Table J.2: Complete list of 48 evaluation repositories with stratification metadata. Abbreviations: Scale — S = Small (< 5K commits), M = Medium (5–20K), L = Large (> 20K); Gov. — Fdn = Foundation, Corp = Corporate, Comm = Community; WF — B = Basic, St = Standard, A = Advanced.

#	Repository	Lang.	Domain	Scale	Gov.	WF	Issues
1	angular/angular	JS/TS	Web framework	L	Corp	A	GitHub
2	apache/airflow	Python	Data / workflow	L	Fdn	A	Jira
3	apache/commons-lang	Java	Library	M	Fdn	St	Jira
4	apache/flink	Java	Data processing	L	Fdn	A	Jira
5	apache/kafka	Java	Messaging	L	Fdn	A	Jira
6	apache/maven	Java	Build tool	L	Fdn	St	Jira
7	BurntSushi/ripgrep	Rust	CLI	M	Comm	St	GitHub
8	cli/cli	Go	CLI	L	Corp	A	GitHub
9	curl/curl	C	Library / DevOps	L	Comm	St	GitHub
10	dbt-labs/dbt-core	Python	Data processing	M	Corp	A	GitHub
11	denoland/deno	JS/TS	Runtime	L	Fdn	A	GitHub
12	django/django	Python	Web framework	L	Fdn	A	GitHub
13	expressjs/express	JS/TS	Web framework	M	Comm	St	GitHub
14	facebook/react	JS/TS	UI library	L	Corp	A	GitHub
15	fastapi/fastapi	Python	Web framework	M	Comm	A	GitHub
16	git/git	C	DevOps	L	Comm	St	GitHub
17	golang/go	Go	Language / runtime	L	Corp	A	GitHub
18	grafana/grafana	Go	DevOps / monitoring	L	Corp	A	GitHub

Continued on next page

Table J.2 — continued from previous page

#	Repository	Lang.	Domain	Scale	Gov.	WF	Issues
19	hashicorp/terraform	Go	DevOps / IaC	L	Corp	A	GitHub
20	helm/helm	Go	DevOps	M	Fdn	A	GitHub
21	huggingface/transformers	Python	ML library	L	Corp	A	GitHub
22	kubernetes/kubectl	Go	DevOps	L	Fdn	A	GitHub
23	llvm/llvm-project	C/C++	Compiler	L	Fdn	A	GitHub
24	microsoft/TypeScript	JS/TS	Language	L	Corp	A	GitHub
25	microsoft/vscode	JS/TS	IDE	L	Corp	A	GitHub
26	neovim/neovim	C	Editor	L	Comm	St	GitHub
27	nestjs/nest	JS/TS	Web framework	M	Comm	St	GitHub ⁺
28	openssl/openssl	C	Security library	L	Comm	St	GitHub
29	pallets/click	Python	CLI library	M	Comm	St	GitHub
30	pallets/flask	Python	Web framework	M	Comm	St	GitHub
31	pandas-dev/pandas	Python	Data processing	L	Fdn	A	GitHub ⁺
32	postgres/postgres	C	Database	L	Comm	St	GitHub
33	prometheus/prometheus	Go	DevOps / monitoring	L	Fdn	A	GitHub
34	psf/requests	Python	Library	M	Fdn	St	GitHub
35	python/cpython	Python	Language / runtime	L	Fdn	A	GitHub
36	redis/redis	C	Database	L	Corp	A	GitHub
37	rust-lang/rust	Rust	Language	L	Fdn	A	GitHub
38	scikit-learn/scikit-learn	Python	ML library	L	Fdn	A	GitHub
39	scipy/scipy	Python	Scientific comp.	L	Fdn	St	GitHub

Continued on next page

Table J.2 — continued from previous page

#	Repository	Lang.	Domain	Scale	Gov.	WF	Issues
40	serde-rs/serde	Rust	Library	M	Comm	St	GitHub
41	sharkdp/bat	Rust	CLI	S	Comm	St	GitHub
42	spring-projects/spring-boot	Java	Web framework	L	Corp	A	Jira [†]
43	spring-projects/spring-framework	Java	Web framework	L	Corp	A	Jira
44	sveltejs/svelte	JS/TS	Web framework	M	Comm	A	GitHub
45	tokio-rs/tokio	Rust	Async runtime	M	Comm	A	GitHub
46	torvalds/linux	C	OS kernel	L	Fdn	A	GitHub
47	vitejs/vite	JS/TS	Build tool	M	Comm	A	GitHub [†]
48	vuejs/vue	JS/TS	Web framework	L	Comm	A	GitHub

[†] *Pending*: selected by stratified sampling; data collection scheduled prior to Chapter 7. The remaining 44 repositories have confirmed data in the evaluation pipeline.